neURROn Design Manifest

C. Niederberger
and
the neURROn group

Revised July 2026

Contents

Chapter 1

Introduction and History

The original neUROn was coded in C in 1992 when I was at Baylor College of Medicine. It was named "neural computational for UROlogical numericals," a somewhat mercurial reference to its application domain, the medical field of Urology. In actuality, neUROn is a general purpose neural computational modeling environment. neUROn has been, and will continue to be used to model many problems beyond the Urological domain.

Between 1992 and 2000, neUROn evolved in fundamental ways. In 1996, Young Hong migrated neUROn to C++, and the resulting environment was named "neUROn++". Many have coded these fundamental changes, including Vinod Kutty, Yuan Qin, Luke Cho, Joe Jovero, and Kai Liu. In general, beyond migrating neUROn to object-oriented programming, these fundamental changes have taken 2 forms: programming novel neural computational training algorithms and programming statistical analysis of training behavior. Throughout this project, Richard Golden Ph.D. at the University of Texas at Dallas has suggested algorithms which the team has implemented.

The impetus for programming novel training algorithms should be obvious. Neural computational algorithms are iterative, and problems modeled by the neural computational approach can easily become intractable, even with modern computational hardware. A single training session may require several hours, and multiple training sessions may require days or even weeks. There are 3 basic ways in which training sessions may become shorter:

1. Faster hardware may be used. This is clearly the way in which the programmer has the least control.

2. More efficient code may be written. The importance of this cannot be stessed enough, and it will serve as a basic design principle for the neUROn project. In the original neUROn code, for example, arrays were expressed entirely as pointer operations. It was not until the compiler benchmarks demonstrated similar outcomes for array constructs that arrays were used in the code.

3. Faster training algorithms may be encoded. A substantial part of the neURON project is devoted to investigating novel training algorithms, and this has paid handsomely. Investigated algorithms have been demonstrated to speed training by 2 orders of magnitude or more.

The impetus for programming statistical analysis of network behavior was less obvious, but perhaps more significant. When we began reporting results to the Urological community, we were surprised to find a universal response was, "so the network accurately predicts X, but what were the important variables?" In retrospect, this response is unsurprising. The majority of our audience, our potential users, were physicians or scientists involved in the medical field. It is not enough to accurately model an outcome for these users, it is critical to find the significant input features so that these features may be altered by the user in the clinical environment. For example, if it is determined that medication X is statistically significant in the model Y that predicts cancer recurrence, the user will put that information to use in the clinical domain.

Beginning in 2000, neUROn was entirely redesigned using this document as a specification, and taking full advantage of the object-oriented nature of C++. Whereas the migration from 1996 to 2000 was an incorporation of C code into the C++ paradigm, the hybrid code which resulted was brittle, with previous features breaking as new features were added. In 2000, the code was redesigned from the "ground up". This document is intended to be a description of neUROn programming, past, present, and future. As such, the document is a plan that is expected to evolve as neUROn is redesigned and recoded in the future.

The current neuron 3.0 work resumes that design discipline around a modern C++ engine and a loopback web interface. It preserves the original numerical, model, and statistical object hierarchy while adding reproducible scripted sessions; a GUI and REST API with command-line parity; asynchronous training and cancellation; explicit convergence semantics; automatic optimizer probing; plateau stopping; validation-based OBD architecture selection; shared, stratified, and group-disjoint cross-validation plans; untouched locked-test evaluation; and machine-readable prediction, metric, and design artifacts.

Statistical reporting has likewise been made explicit about its sampling unit. Independent-row locked tests use paired DeLong covariance; clustered locked tests use Obuchowski's nonparametric clustered covariance. Both share the same Mann–Whitney point AUC, placements, tie handling, interval algebra, and paired contrast rules. Group-aware partitioning and clustered inference remain separate mechanisms: the first prevents leakage, while the second estimates uncertainty when rows within a sampling unit are correlated.

Part I describes these current interfaces and workflows. Part II retains the mathematical derivations, documentation rules, and object specifications which make the manifest a programming plan rather than a user manual. Chapter 12 now connects the established objects with the newer partition, training-control, selection, covariance, cross-validation, and reporting services.

Part I

Current neuron 3.0

Chapter 2

neuron 3.0 Architecture

The 2026 reanimation preserves the object design described in this Manifest while giving it three contemporary ways to be used: the original menu interface, a local graphical interface, and a loopback HTTP interface. The same C++ model and statistical classes serve all three. Python tools prepare data and export trained models, but do not duplicate the engine.

The central architectural rule is that every mechanism has one authoritative implementation in the class that conceptually owns it. The graphical interface may call that implementation, and a cross-validation run may call it repeatedly, but neither copies its mathematics. Dependencies point down through the layers:

Human and programmatic interfaces: GUI, REST API, frozen CLI menus
Experiment orchestration: cross-validation, OBD sizing, reports, automatic optimizer selection
Model and data interfaces: Model, DataSet, TwoSet, split plans, model factory and clone helpers
Training and concrete methods: Iterative, Network, Logistic, SimpleProp, BareProp, BackProp, LDFA, QDFA, RegressNet
Numerical foundation: Matrix, vector operations, statistics and utility functions

A lower layer never learns a higher concern. Matrix can gather rows but does not know what a fold is. DataSet can materialize a fold but does not choose a model. A model fits itself but does not run a cross-validation loop. OBD chooses a hidden-layer size but does not own final evaluation. Cross-validation owns repetition, not training. This separation keeps the matrix notation, the implementation, and the statistical report traceable to one another.

2.1 Legibility and speed as joint requirements

Two constraints govern every line of engine code, and neither may be traded for the other or for a smaller source file.

The first is legibility. The computational core was built on Matrix, the vector operations and Population so that a reader can move between a published equation and its implementation in either direction. A gradient in this Manifest and the statement that computes it should be recognizably the same object. An abstraction that hides a published formula has failed even when it removes duplication: the biased and unbiased forms of a gradient are two equations, and keeping both visible is worth the repetition that a flag would remove.

The second is speed. Neural computation is compute-bound, and the engine is compiled C++ for that reason. The numerical classes therefore offer each operation twice: a form that fills a destination the caller supplies, and a form that returns a new object. That pair is deliberate and is not duplication. The returning form is for formulae written once; the destination form is what belongs inside a training loop, where a fresh allocation per exemplar per iteration is the difference between a run and a wait. The compound operators and the range overloads exist for the same reason, the latter also because a range expresses a mathematical domain – every hidden unit except the bias slot – more directly than computing the whole vector and repairing it afterwards.

It follows that removing duplicated source is not, by itself, an improvement. Indirection through a function object, a virtual call, or a copy introduced into a per-exemplar path to unify two loops trades a measurable cost for a cosmetic gain. In cold paths – reporting, file writing, one-time setup – those tools are unobjectionable. A suspected performance problem is measured before an algorithm changes in response to it.

The operating form of this section, with the specific idioms and the episode that prompted writing it down, is standing rule 7 in `CLAUDE.md`; rule 4 states the legibility half and rule 6 the ownership half.

2.2 Interfaces

2.2.1 Command line

The interactive menus remain the authoritative feature inventory and are intentionally frozen. Reproducible command-line runs feed a complete answer file to the engine and always provide a seed:

```
./build/neuron --seed 42 < session.in > session.out
```

The transcript is the permanent record of the run. The program is not intended to be driven by keystrokes in an automated experiment.

2.2.2 Graphical interface

```
./build/neuron --gui
```

This starts an embedded cpp-httplib server on `127.0.0.1` at an operating-system-assigned port and opens a single local web page. The page provides the full load, model, train, analysis, report, and save workflow. It includes realtime training progress and ROC plots. The server is loopback-only: it is a local application, not a network service.

GUI and CLI parity is a contract. Every CLI capability must have both a page control and an HTTP parameter. The maintained option-by-option map is `docs/gui_cli_parity.md`. Features added beyond the frozen CLI, including automatic optimizer selection, OBD sizing, and cross-validation, are available through both the GUI and the HTTP interface.

2.2.3 Python tools

`tools/mkdataset.py` turns raw CSV data into the numeric, outcome-last format used by the engine and writes the key and input-group files needed to interpret the encoded columns. `tools/neuron2web.py` combines a saved network, scaling factors, and a human-readable label specification into a self-contained HTML calculator. The deployed calculator performs inference entirely in the browser.

2.3 Data ownership and splitting

`DataSet` owns loaded and normalized observations. `TwoSet` owns classification and ROC reporting for a modeled set. The dedicated `split` layer owns index plans and provides:

- outcome-stratified train and test splits;
- optional covariate stratification with quantile cells and Hamilton apportionment;
- group-aware splits that keep an entire cluster on one side;
- three-way train, validation, and untouched test splits;
- outcome-stratified and outcome-by-covariate folds for cross-validation; and
- outcome-balanced group folds that never divide a cluster.

These mechanisms solve different problems. Covariate stratification makes two samples more alike and is useful for subgroup coverage. Grouping makes evaluation harder by preventing site or cluster leakage. In a grouped comparison, one group key governs the outer folds, the locked holdout, and nested OBD's inner validation split. No model-selection stage can therefore see rows from a cluster that it later treats as unseen. A validation set permits model selection without repeatedly looking at the final test set.

2.4 Models and training

`Model` defines the common modeling interface. `Network` adds connected layers and weights. `Iterative` owns the training loop, stopping conditions, progress callbacks, and convergence state. Concrete model classes own their distinct mathematics:

SimpleProp One hidden layer with bias nodes.

BareProp One hidden layer without bias nodes. `SimpleProp` and `BareProp` share their state, training, and forward equations through `OneHiddenNet` and remain distinct concrete types; only the biased model establishes a bias slot.

BackProp Multiple hidden layers and multi-output networks.

Logistic Binary logistic regression, including Wald tests and the design-matrix condition number.

LDFA and QDFA Linear and quadratic discriminant analysis.

RegressNet Forward and reverse stepwise variable selection using the Wilks likelihood-ratio test.

Training supports canonical gradient descent, conjugate gradient descent, and Shanno's method. The automatic selector probes all three from cloned starting weights for a short wall-clock budget, adopts the lowest-error probe, and continues from the progress it made.

Stopping conditions are evaluated every iteration, independently of how often an iteration row is printed. Reaching the iteration ceiling means that the requested operation ran, but the fit did not converge. The weights remain available for continuation; an unfinished fit is not eligible for model selection or cross-validation scoring.

2.5 Model selection and evaluation

2.5.1 OBD hidden-layer sizing

The `obd` layer grows a SimpleProp hidden layer while held-out error improves, then prunes by saliency. Each candidate size is stopped at its held-out minimum or by a valid training stopping condition. With a three-way split, the search monitors validation and leaves test untouched. A successful standalone OBD run replaces the current model with the selected, trained network. The published saliency idea, neuron's validation-driven adaptation, and the complete object contract are explained in section 12.8.

2.5.2 Cross-validation

`crossval` applies logistic regression, LDFA, QDFA, a fixed neural architecture, or nested OBD to one shared fold plan. `cvadapters` connects those procedures to their existing model implementations. `cvreport` renders a three-tier result:

1. a headline comparison suitable for decisions;
2. a detailed human-readable audit; and
3. machine-readable prediction, metric, and run files.

Nested OBD repeats architecture selection inside each training fold. It therefore estimates the performance of the selection procedure, not the performance of an architecture chosen using all observations. Cross-validation is a standalone analysis and does not replace the current model. Section 12.9 explains why selection must be nested inside each assessment fold and cites the supporting model-selection evidence.

The default plan is outcome-stratified. A request may instead balance folds on outcome-by-covariate cells or assign whole groups with a seeded, outcome-balanced bin-packing plan. Covariate stratification and grouping are alternatives: no untested joint objective is silently invented. The completed design records achieved rows, events, and groups per fold, its largest group and imbalance, and a zero-leakage check in the Tier 3 run artifact.

2.5.3 Locked-test inference

A locked test set may be removed before cross-validation. Procedures are compared on the development rows, refitted by their own rules, and scored once on the locked rows. The evaluation design is structured by `evaldesign`: the declared sampling unit and achieved partition select the only permitted covariance estimator.

For genuinely independent rows, `delong` supplies ordinary paired AUC intervals and the prespecified contrast. For rows clustered by the request's group key, `clustered_auc` supplies Obuchowski's non-parametric clustered ROC covariance. Both use the same `auccov` placements, mid-ranks, point AUC, and contrast algebra. They can therefore differ only in the covariance appropriate to the sampling unit. Grouping prevents leakage but does not make rows independent; an independent-row declaration over a group-disjoint design is refused rather than passed to ordinary DeLong. Section 12.6 derives the distinction, defines the paired contrast, and cites both estimators' primary papers.

The point AUC remains the row-level Mann–Whitney probability under both methods. Clustering changes uncertainty, not the point estimand. The effect has no fixed direction: ordinary row-based standard errors can be too small or too large, depending on the within-cluster covariance.

2.6 Statistical reporting

Binary models report classification accuracy, sensitivity, specificity, and two complementary ROC estimates. The trapezoidal area is the exact non-parametric AUC over every distinct operating point. The primary binormal A_z is fitted to the z-ROC curve and is reported with a stratified bootstrap confidence interval. Reports state the number of fitted operating points, bootstrap resamples, and failures. Logistic models additionally report coefficient-level Wald tests and the condition number.

2.7 Run artifacts and audit trail

The directory of the first loaded dataset is the run directory. Networks, scaling factors, normalized sets, guesses, reports, `neuron.log`, and `neuron_actions.log` are written there. The latter records every GUI or API action with a timestamp and its parameter values. Long operations publish structured progress through one shared status channel and honor graceful cancellation at an iteration boundary.

2.8 Source map for the modern layers

Files	**Responsibility**
`gui.*`, `gui_page.html`	Local server, REST handlers, and browser interface.
`asyncjob.*`, `procguard.*`	The asynchronous-job lifecycle every long operation shares, and the command-line program's top-level exception boundary.
`split.*`	Stratified, group-aware, three-way, arbitrary-stratum k-fold, and whole-group k-fold index plans.
`crossval.*`, `cvadapters.*`, `cvreport.*`	Procedure comparison, connections to model implementations, and human plus machine-readable reports.
`obd.*`	Grow-and-prune hidden-layer selection.
`autoalgo.*`, `plateau.h`	Optimizer probing and plateau detection shared by training workflows.
`evaldesign.*`	Typed partition, sampling-unit, and inference policy metadata.
`auccov.*`, `delong.*`, `clustered_auc.*`	Shared AUC placements and contrast algebra, ordinary independent-row DeLong covariance, and Obuchowski clustered covariance.
`modelfactory.*`, `netclone.*`	Construction and faithful cloning through the common model interfaces.

Files	Responsibility
`dataset.*`, `twoset.*`	Observation ownership, normalization, classification, ROC, and statistical reporting.
`model.*`, `network.*`, `iterative.*`	Core model contract, network structure, and authoritative training loop.
`onehidden.*`	State, training, and forward equations shared by the two one-hidden-layer models.
`matrix.*`, `vector_ops.*`, `stats.*`	Bounds-checked numerical and statistical foundation.

Chapter 3

Command-line interface

The command-line menus remain fully supported and are the authoritative feature inventory, but they are frozen. New interactive work normally uses the GUI described in Chapter 2; new programmatic work uses the REST API in Chapter 4.

3.1 Reproducible sessions

Automated runs write every menu answer to a text file, one answer per line, and feed that file through standard input. Always supply a seed and capture standard output:

```
./build/neuron --seed 42 < session.in > session.out
```

Read the completed transcript and verify that it ends after the quit confirmation, not at an error or unanswered prompt. Do not automate the menus by typing into a live terminal.

3.2 Menu organization

Specify dataset Set input and output counts; load raw data or a normalized training and test pair; create a seeded train and test split; set outcome type, normalization bounds, and ROC reporting; save normalized sets and scaling factors.

Specify model Create a feed-forward network or binary logistic model; set bias nodes, hidden layers, and error function; load a saved network; set logging.

Use model Randomize weights; configure learning rate, weight decay, batch or epoch behavior, printing, and stopping conditions; train with canonical gradient descent, conjugate gradient descent, or Shanno's method; run stepwise selection; save the network and predictions.

Discriminant function analysis Run linear or quadratic discriminant analysis and save its predictions.

The exact menu-to-GUI-to-API mapping is maintained in `docs/gui_cli_parity.md`. That file is the current option-level reference and is updated whenever an interface changes.

3.3 Dataset and run files

Numeric data place the outcome in the final column. Binary outcomes are encoded as 0 and 1. The first loaded dataset establishes the run directory, where the engine writes networks, scaling factors, reports, predictions, `neuron.log`, and `neuron_actions.log`.

For raw CSV data, use `tools/mkdataset.py` as described in Chapter 5. When an intercept-bearing model will be trained, reference-category coding avoids collinearity between a full one-hot group and the intercept.

3.4 Convergence and reporting

The iteration ceiling is a safety limit, not evidence of convergence. If training reaches it, the current weights can be continued, but the fit is unfinished. Model-selection and cross-validation procedures refuse unfinished candidate fits.

For binary outcomes, interpret held-out performance through classification measures and ROC areas, not accuracy alone. The primary fitted binormal A_z is reported with its bootstrap 95% confidence interval, operating-point count, resample count, and any failures. The exact empirical AUC and Hanley–McNeil interval are an independent cross-check. Logistic reports additionally provide Wald tests and a condition number.

3.5 Current command-line options

```
Usage: neuron [--seed N] [--gui [--no-browser]] [--version]
```

`--gui` starts the local browser interface. `--no-browser` leaves it running without opening a browser. `--version` prints the package version and exits.

Chapter 4

Loopback REST API

The graphical interface is a client of the same HTTP endpoints that are available to scripts. Start the service without opening a browser:

```
./build/neuron --gui --no-browser
```

The engine prints a URL such as `http://127.0.0.1:54321/`. The port is assigned at runtime, so clients should read it from standard output. Requests use ordinary form fields or multipart file uploads. Successful JSON responses contain `"ok":true`; failures contain `"ok":false` and a human-readable `message`.

The server deliberately binds only to loopback and has no authentication. It is intended for a user or local agent on the same computer. It must not be exposed as a public web service.

4.1 How a field's value is read

A request field arrives as text, and the endpoint has to decide what number, count or flag it names. Until 2026-08-03 that decision was made by the C conversions `atol` and `atof`, which consume as much of the text as they understand, stop, and report nothing. A caller who wrote `folds=5junk` got five folds; `fraction=abc` got zero, which is a request for a train/test split that produces no test set and still answers `"ok":true`; and `async=true` ran synchronously, because the only token compared was `1`. A local agent driving this API has no prompt to re-read and no chance to notice.

Every field is now read through the strict parsers described in Section 12.11, and the rules below are the whole contract.

Whitespace and completeness

Leading and trailing spaces and tabs are removed, and the rest of the text must be consumed entirely. `fraction= 0.3 ` is accepted, because the old conversions accepted it too; `fraction=0.3junk` is refused. There is no partial reading of any field.

Whole numbers

Optional sign `+`, then decimal digits. No hexadecimal, no exponent, no decimal point. A negative value is refused as a sign error rather than wrapping to an enormous unsigned one, and a value above the width of the type is refused as out of range rather than truncating: before this change `maxiter=4294967297` trained for one iteration and then reported that it had reached the maximum iteration count.

Numbers

Anything `strtod` accepts in the C locale, consumed entirely, and finite. `nan`, `inf` and their signed and mixed-case spellings are refused for every field: an infinite gradient limit is a stopping rule that can never fire, and an infinite variate bound is not a bound. Overflow to infinity is refused as out of range; underflow yields the closest representable value, which may be zero, and the field's own domain check then decides whether zero is allowed.

Flags

Exactly `1` and `0`, case-sensitively, on every endpoint. No other spelling is accepted anywhere, so a misspelled flag can no longer silently mean false. Until 2026-08-03 the five procedure flags of `/api/cv` also accepted lowercase `true`; they no longer do, and that is the one previously valid request this change refuses.

Omitted, and present but empty

A field that is absent leaves the value it configures alone: the model's current setting, the engine's own default, or the documented default for that endpoint. A field that is present with an empty value does the same for numbers and counts, because the page submits several of them unconditionally — `fraction`, `seed` and `decay` arrive on every request whether or not the user filled them in.

Two exceptions, both deliberate. On `/api/train` a present but empty `minerr`, `change`, `errwindow` or `gradmax` *disables* that stopping condition, while an empty `printcount` is simply ignored. And an empty *flag* is refused: there is no field on this API where a blank checkbox means anything.

What a refusal looks like

A malformed field is refused with `"ok":false` and a message naming the field and quoting the text it was given, so that a caller can tell which of twenty parameters was wrong:

```
folds: '5junk' is not a whole number
maxiter: '4294967297' is out of range
test_n: '-5' cannot be negative
```

```
gradmax: 'inf' is not a finite number
async: 'true' must be 1 or 0
logistic is empty
```

A syntax fault and a domain fault read differently on purpose. `strata_bins=abc` once reported "strata_bins must be at least 2", which sends the reader looking for a value problem in a value that was never read; it now reports that the text is not a whole number, while `strata_bins=1` still reports the domain.

Refusal is atomic

Every handler reads all of its fields before it changes anything, so a request refused for its twentieth parameter has not applied its first. A refused `/api/train` carrying a new learning rate leaves the model training at the old one.

4.2 A complete minimal session

```
URL=http://127.0.0.1:54321

curl -s -X POST "$URL/api/load" \
  -d "mode=raw&path=data.txt&fraction=0.25&seed=42"

curl -s -X POST "$URL/api/model" \
  -d "type=logistic"

curl -s -X POST "$URL/api/train" \
  -d "algorithm=1&maxiter=20000&seed=42"

curl -s "$URL/api/stats"
curl -s "$URL/api/save/network" -o network.txt
curl -s "$URL/api/save/scales" -o scales.txt
```

File fields accept either a path for a local script or an uploaded multipart part for a browser. Uploaded files are copied into the run directory using the safe final component of their filename.

4.3 Endpoint summary

Verb	Path	Purpose
GET	`/api/version`	Engine package and version string.
POST	`/api/load`	Load, normalize, and optionally split a dataset.

Verb	Path	Purpose
POST	`/api/model`	Create a model or load a saved network.
POST	`/api/randomize`	Reset the current model's weights from a seed.
POST	`/api/train`	Train synchronously or start an asynchronous run.
GET	`/api/train/status`	Read progress and the completed result of the active or most recent long job.
POST	`/api/train/stop`	Request graceful cancellation at an iteration boundary.
GET	`/api/stats`	Return the current trained model's ROC and classification statistics.
POST	`/api/regress`	Run forward or reverse stepwise regression.
POST	`/api/dfa`	Run linear or quadratic discriminant analysis.
POST	`/api/obd`	Start asynchronous hidden-layer sizing.
POST	`/api/cv`	Start asynchronous procedure comparison and optional locked-test inference.
GET	`/api/save/:what`	Download a session artifact.

4.4 Dataset loading: POST /api/load

`mode` `raw` to normalize and split one file; `train` to load an already normalized training set and optional matched test set.

`path` **or** `file` Dataset path or multipart upload.

`testpath` **or** `testfile` Optional matched test set in `train` mode.

`inputs`, `outputs` Optional column-count overrides. Outputs default to 1; multiple outputs select BackProp and accuracy-only reporting.

`discrete` `1` for classification or `0` for a continuous outcome.

`fraction`, `test_n` Test share or exact test count. Use one form. A value of 0 keeps all rows in training.

`val_fraction`, `val_n` Optional validation share or count for a three-way split. The count form accompanies `test_n`.

`seed` Reproducible split seed.

`strata`, `strata_bins` Comma-separated one-based input columns for covariate stratification and the number of quantile bins for continuous columns.

`group` Comma-separated one-based input columns whose joint values define indivisible clusters. Grouping takes precedence over covariate stratification.

`threshold` Classification threshold between 0 and 1.

`in_lower`, `in_upper` Input normalization bounds.

`out_lower`, `out_upper` Output normalization bounds for a continuous outcome.

`roc_report` `both` or `either`, defining the class-availability rule for ROC reporting.

`roc_min` Minimum observations needed for statistical ROC reporting.

`history` `1` or `0` for dataset-operation logging.

Outcome stratification is always applied to ordinary random splits. Covariate stratification and grouping are deliberate alternatives. A three-way split cannot currently be combined with either of them. The response includes achieved row counts and, when appropriate, a representativeness or zero-leakage diagnostic.

4.5 Model construction: POST /api/model

`type` `logistic` or `simpleprop`.

`hidden` A comma-separated layer list such as `5` or `8,4`. Several layers create BackProp.

`bias` `0` creates BareProp for a one-hidden-layer network; `1` keeps bias nodes.

`errfunc` `auto`, `xentropy`, or `lms`.

`mode` `load` reads the network type and architecture from a saved network.

`path` **or** `file` Saved network path or multipart upload when `mode=load`.

`log_lastop`, `log_history` `1` or `0` logging controls.

Model construction requires a loaded dataset because the input and output dimensions are data-dependent. A loaded network is checked against those dimensions before it becomes current.

4.6 Weight reset: POST /api/randomize

The optional `seed` field resets the current trainable model to a reproducible starting point. Training itself continues from current weights; repeated calls to `/api/train` do not randomize implicitly.

4.7 Training: POST /api/train

`algorithm` `1` for canonical gradient descent, `2` for conjugate gradient, `3` for Shanno, or `auto` to probe and choose.

`maxiter` Required positive iteration ceiling.

`seed` Seed used by training and automatic selection.

`async` `1` returns immediately and publishes progress through `/api/train/status`.

`eta`, `autostep` Manual learning rate and automatic-step toggle.

`batch_epoch` Batch or epoch mode. Logistic regression requires it to remain on.

`weight_decay`, `decay` Weight-decay toggle and lambda.

`minerr`, `change`, `errwindow`, `gradmax` Classic stopping conditions. Omitting a field keeps its current value; sending an empty field disables that condition.

`autostop`, `autostop_tol`, `autostop_window` Plateau detector and its tolerance and moving-window width.

`logprint`, `printcount` Iteration-table presentation only. These fields never change the fitted endpoint.

A blocking call returns the complete report, ROC series, and structured statistics. An asynchronous call returns acceptance immediately; the same complete result later appears in status. The response separates operation success from fit validity with fields including `converged`, `ceilingExhausted`, and `stopReason`.

4.8 Long-job control

4.8.1 GET /api/train/status

Training, stepwise regression, OBD, and cross-validation share one job channel. Status returns `running`, the current phase, decimated training and held-out error series, and procedure-specific progress. OBD adds its phase and hidden-node count. Stepwise adds the current pass and candidate. Cross-validation adds stage, procedure, run, fold, and nested-search detail when present. When the job finishes, `result` contains the same object its blocking form would have returned.

4.8.2 POST /api/train/stop

Cancellation is graceful: the engine finishes the current iteration and produces the appropriate partial audit. Bodyless POST requests must still carry a zero-length body:

```
curl -s -X POST -d "" "$URL/api/train/stop"
```

A bare `curl -X POST` sends no Content-Length header and can make the server wait for its read timeout before dispatching.

Only status and stop remain available while a job owns the engine. Other engine-touching endpoints return HTTP 409 with `"busy":true`; there is no hidden request queue.

4.9 Standalone analyses

4.9.1 POST /api/dfa

`type=linear` runs LDFA and `type=quadratic` runs QDFA. Optional `log_lastop` and `log_history` fields control logging. The response includes a captured report and, for a discrete single-output dataset, ROC series and structured statistics. DFA does not replace the current model.

4.9.2 POST /api/regress

`structure` Semicolon-separated conceptual variables; commas join nodes and hyphens denote inclusive ranges, for example `0;1-4;5;6,7`.

`direction` `forward` or `reverse`.

`threshold` Selection p-value threshold.

`async` `1` for status, progress, and Stop.

Stepwise requires a trained cross-entropy model. Candidate subnetworks are fitted on clones, never mutate the current model, and must converge before entering a Wilks comparison. The result contains the selection path, every candidate audit, the final variables settled by completed passes, and a `complete` flag.

4.9.3 POST /api/obd

`hidden_start`, `hidden_max` Starting size and safety ceiling.

`iter_budget`, `sample_every` Per-size ceiling and held-out sampling cadence.

`early_stop_tol`, `early_stop_patience` Within-size held-out reversal rule.

`grow_patience`, `prune_tol` Growth and pruning rules.

`autostop_tol`, `autostop_window` Per-size training plateau backstop.

`algorithm` `1`, `2`, `3`, or `auto`.

`seed` Reproducibility seed.

OBD is asynchronous-only and requires a discrete, single-output dataset with a held-out monitoring set. Its result names that set as `validation` or `test`, records every size trial, and replaces the current model only after a successful search.

4.9.4 POST /api/cv

`folds`, `seed` Fold count and reproducibility seed.

`logistic`, `ldfa`, `qdfa`, `neural` Procedure-selection toggles.

`neural_obd`, `neural_hidden` Nested OBD or a fixed neural size.

`maxiter`, `hidden_max`, `iter_budget` Ordinary-fit and nested-search ceilings.

`algorithm` Nested OBD optimizer, including `auto`.

`autostop_tol`, `autostop_window`, `inner_val` Nested-fit plateau rule and inner validation share.

`strata`, `strata_bins` Optional one-based input columns for outcome-by-covariate fold balance and the number of quantile bins for continuous columns.

`group` Optional one-based input columns whose joint values define indivisible sampling groups. The same key governs the outer folds, locked holdout, and nested OBD inner validation split.

`locked_fraction`, `locked_n` Optional locked test share or count. Use one form.

`primary`, `reference` Prespecified procedure tokens for the signed AUC contrast.

`independence` `rows` opts into ordinary paired DeLong inference for a row-wise design. `cluster` selects Obuchowski clustered covariance and requires `group` plus a locked test. Omit the field for point estimates only.

Cross-validation is asynchronous-only, requires raw discrete single-output data, and does not alter the current model. The result contains headline and detailed reports plus paths to Tier 3 CSV and JSON artifacts.

Without `strata` or `group`, folds are outcome-stratified. `strata` balances outcome-by-covariate cells; `group` makes whole-cluster folds for evaluation on unseen groups. The two cannot currently be combined. These fields belong to the CV request and are never inherited from `/api/load`. A grouped request also makes the locked holdout and nested inner validation group-disjoint, with no row-wise fallback when a whole-group partition is infeasible.

A locked test adds original-row predictions and a signed, prespecified AUC contrast. Independent-row DeLong and clustered Obuchowski inference are explicit alternatives selected by the declared sampling unit; neither ever falls back to the other. The clustered preflight requires at least two locked-test clusters carrying each outcome class. Tier 1 names the estimator and the number of independent clusters. `cv_run.json` records the typed fold and locked-test designs, including per-fold rows, events, groups, imbalance, largest group, and leakage. The two prediction CSV files use the original raw-row identity and, for grouped designs, stable cluster identifiers, so they join without reconstructing either key.

4.10 Statistics and downloads

4.10.1 GET /api/stats

Returns structured train and optional test statistics for the current trained classification model. Each set includes classification measures, ROC estimates, confidence intervals, and curve data. The training endpoint also embeds these values with the captured human-readable report.

4.10.2 GET /api/save/:what

Replace `:what` with one of:

`network`	saved model weights and architecture
`scales`	natural-to-normalized scaling factors
`train_set`, `test_set`	normalized datasets
`train_guesses`, `test_guesses`	model predictions
`dfa_train_guesses`, `dfa_test_guesses`	DFA predictions
`report`	captured report from the last training operation

The engine writes the artifact in the run directory and returns the same bytes as a download. Missing prerequisites return HTTP 400 with a JSON explanation.

4.11 Response and audit conventions

The API returns machine-readable values together with the engine's captured report, rather than asking clients to reconstruct statistical meaning from a few scalars. JSON numeric fields that cannot be calculated are null, not invented sentinel estimates.

Every API action, including save and stop, is timestamped in `neuron_actions.log`. Engine operations and self-describing training headers are written to `neuron.log`. Together with the seed and saved artifacts, these logs make a local GUI or scripted session reproducible and auditable.

Chapter 5

Data preparation and deployment tools

The current support tools are standard-library Python programs in `tools/`. They require Python 3 but no installed packages. The Perl, Palm OS, and early iPhone exporters from neUROn2++ are not part of neuron 3.0.

Program	Purpose
`mkdataset.py`	Convert a delimited table into a numeric neuron dataset plus interpretation files.
`neuron2web.py`	Convert a saved model and its natural-unit interface into a self-contained browser calculator.

The maintained command reference is `tools/README.md`. Both programs are exercised against committed expected outputs by `tests/tools/run_tools.sh`.

5.1 Preparing a dataset

`mkdataset.py` converts an ordinary delimited file into the numeric, outcome-last form used by the engine. It can change delimiters, one-hot encode categorical inputs, map a two-valued text outcome to 0 and 1, create missing-value indicator pairs, and omit one reference category from each categorical group.

```
python3 tools/mkdataset.py --onehot --refcat \
  --key key.txt --inputs inputs.txt \
  -o data.txt raw.csv
```

The key maps encoded columns back to human variables. `inputs.txt` groups nodes belonging to the same conceptual variable for stepwise analysis. The program reports the resulting column count; the number of input nodes is one less than that count for a single outcome.

5.1.1 Current grooming behavior

- The first row is treated as column labels unless `--noheader` is supplied.
- Python's CSV parser preserves quoted fields containing embedded delimiters. `--delimiter` accepts any single character, including `\t` for a tab.
- Rows with a blank outcome are dropped with a warning. `--noelim` retains them when that behavior is intentional.
- A numeric input containing blanks becomes a two-node indicator pair: presence followed by value. A blank is represented by `0,0`; a present value by `1,value`.
- `--onehot` with no column list detects non-numeric columns automatically. A comma-separated list of one-based column numbers limits conversion to selected columns.
- A two-level text outcome becomes one binary output. Text input levels become grouped indicator nodes; a blank input activates none.
- `--refcat` omits the alphabetically first input level and records it as the reference. This is the appropriate form for logistic regression and other intercept-bearing models because it avoids a perfectly collinear full indicator group.
- `--key` records every emitted column and its source label. `--inputs` records conceptual-variable groups in the syntax accepted by stepwise regression.
- `--quiet` suppresses the narrative report without changing the output files.

5.2 Creating a browser calculator

`neuron2web.py` combines three artifacts:

1. a saved network containing architecture and weights;
2. scaling factors mapping natural values to model inputs; and
3. a label specification describing fields, units, categories, and outcome text.

```
python3 tools/neuron2web.py \
  --network network.txt --scales scales.txt \
  --spec spec.txt -o calculator.html
```

The resulting HTML file is self-contained. Its JavaScript, model, scaling rules, and form are embedded, so it can be opened from disk, emailed, or served by any static web host. Inference remains in the visitor's browser.

5.2.1 Supported models and form fields

The exporter currently accepts saved SimpleProp and binary logistic models. It refuses other network types rather than approximating their architecture. The label specification supports:

`N` A numeric field with label and optional units.

`C` A categorical menu whose levels expand to indicator columns. An asterisk marks a reference level omitted by `--refcat`.

`B` A binary choice.

`K` A missingness indicator paired with a numeric field.

`O` Labels for outcomes 0 and 1.

`R` Optional text for the reported odds.

The complete label grammar and validation contract are in `docs/deploy.md`. The tool checks the specification against the saved network's input count and the scaling file before it writes the calculator.

5.2.2 Verification and preview

Before deployment, use `--eval` with a known natural-unit row to verify the complete scaling and forward-pass path. A dataset that arrived already normalized needs the original natural-to-normalized scales; scales saved from a later training run over normalized values would merely reproduce that normalized interface.

`--serve` writes the calculator, serves it on loopback at an operating-system-assigned port, and opens a preview in the browser. `--title` supplies the page title. `--quiet` suppresses informational output. Neither preview nor deployment requires an external JavaScript package or web service.

5.3 Worked examples

The illustrated Civic Choice walkthrough at `docs/datasets/civic-choice/WALKTHROUGH.md` is the current end-to-end guide. The dataset-specific READMEs under `docs/datasets/` provide further preparation and deployment examples.

Part II

Foundational design record

The chapters in this part preserve the mathematical derivations, documentation philosophy, numerical layer, and object design from the neUROn2++ redesign. They explain why the engine is organized as it is, but their method inventories and code examples are not the current public interface specification. For present architecture, build, menus, and HTTP behavior, use Part I and the source tree.

Chapter 6

Methodology

The following chapter outlines algorithms currently implemented in neUROn, as well as many intended to be implemented in the future. As usual, we are indebted to Dr. Golden. The terminology ANN is used to refer to "Artificial Neural Network".

1. **Error Gradient Calculation Algorithm.** The gradient of the objective function is computed by calling a *forward propagation algorithm* followed by a *backward error propagation algorithm*.

2. **Parameter Estimation Algorithm.** The gradient computed by the *Error Gradient Calculation Algorithm* is then used by the parameter estimation algorithm for the purposes of learning.

3. **Data Analysis Algorithm.** If this algorithm decides that reliable statistical inference is possible, this algorithm indicates which input nodes in the network are making statistically significant contributions to the network's performance. Otherwise, this algorithm makes recommendations to the user regarding possible manipulations designed to increase statistical inference reliability. Future work will involve computing confidence intervals on the predictions and relaxing the "Goodness-of-Fit" assumption which is assumed throughout this document.

Note that math calculations for the following algorithms such as matrix inverse, eigenvalues, "line-search", cumulative distribution functions for the chi-square random variable for statistical test result reporting can be done using the routines in *Numerical Recipes in C*. In fact, *Numerical Recipes* has a conjugate gradient routine which could be used to develop a learning algorithm for the ANN system as well.

6.1 Error Gradient Calculation Algorithm

6.1.1 Data Records

Note that the data is assumed to have the following format: $(\mathbf{s}_k, \mathbf{t}_k)$ where $\mathbf{s}_k$ is the d-dimensional input vector for the kth data record and d-dimensional vector $\mathbf{t}_k$ is a list of "targets" for the d output units. If the training data consists of N data records, then we have $k = 1, \ldots, N$.

6.1.2 Objective Function

The *error gradient* is defined as the gradient (i.e., vector first derivative) of a sufficiently smooth *error function* with respect to the "weights" (i.e., parameters) of the ANN. In this document version, only feedforward m-layer ANN systems will be considered where the number of "layers" is defined by "connections". Thus, a 1-layer ANN is essentially a linear feedforward ANN with one set of input units and one set of output units and no hidden units. Let $\mathbf{y}$ denote a vector containing all weights of an m-layer ANN. The activation levels of the units in the output layer of an m-layer ANN generated in response to training stimulus k are denoted by the *activation vector*, $\mathbf{a}_k^{(m)}$. Note that $\mathbf{a}_k^{(m)}$ is a function of both $\mathbf{y}$ and the input vector $\mathbf{s}_k$.

The *error function*, E_k, for the kth training pattern is defined by the formula:

$$E_k(\mathbf{y}) = e(t^k, \mathbf{a}_k^{(m)}) + \frac{1}{2\sigma_w^2} \sum_{i=1}^{m} |\mathbf{y}|^2. \tag{6.1}$$

The second term on the right hand side of (6.1) is required to guarantee reliable statistical inference. It basically "stabilizes" the learning process by preventing the weights in the network from becoming too large or too small. In practice, a connection weight is expected to lie between $-1.96\sigma_w$ and $+1.96\sigma_w$ with probability equal to about 95%. Adjustment of this parameter can "change results" of the statistical analysis so it is best to fix σ_w to be some constant number and try not to change its value. A reasonable choice for σ_w would be $\sigma_w = 100$. This basically constrains the "learning process" to find connection strengths for the ANN such that the connection strength weight ranges between -200 and $+200$. The choice of σ_w should be reported in conjunction with all statistical analysis results. Note that as $\sigma_w \to \infty$, the effect of the summation term becomes negligible.

The first term on the right hand side of (6.1) is called the *prediction error function* term. The prediction error function e must have continuous second partial derivatives with respect to the weights. For example, the classic choice of the prediction error function e is to choose

$$e(\mathbf{t}^k, \mathbf{a}_k^{(m)}) = (1/2)|\mathbf{t}^k - \mathbf{a}_k^{(m)}|^2 \tag{6.2}$$

to be the *least squares prediction error function*. This is a good choice when the target vector $\mathbf{t}^k$ is a continuous (as opposed to a discrete) variable.

When the target vector consists of a set of binary variables taking on the values of zero or one, then a good choice is the *cross-entropy prediction error function* defined by the formula:

$$e(\mathbf{t}^k, \mathbf{a}^{(m)}) = -\mathbf{t}^k .* \mathbf{log}[\mathbf{a}_k^{(m)}] - (\mathbf{1} - \mathbf{t}^k) .* \mathbf{log}[1 - \mathbf{a}_k^{(m)}]. \tag{6.3}$$

Note that the notation $.*$ denotes element by element matrix multiplication, the notation $\mathbf{1}$ denotes a column vector of ones, and the notation **log** denotes a vector-valued function defined such that the $\mathbf{y} = \mathbf{log}(\mathbf{x})$ whenever the ith element of the vector $\mathbf{y}$, y_i is equal to the natural log (log base e) of the ith element of the vector $\mathbf{x}$. When the cross-entropy prediction error function is used, the transfer function for all output units should be the sigmoidal logistic transfer function which is defined formally in the next section of this report.

6.1.3 Forward Propagation of Activations

The response of the artificial neuron units in layer j is computed from the activation levels of the artificial neuron units in layer $j-1$ using the *forward propagation algorithm* formulas. These formulas are defined by an update formula for computing the *netinput*, $\mathbf{u}_k^{(j)}$, to a unit in layer j for the kth training stimulus:

$$\mathbf{u}_k^{(j)} = \mathbf{W}^{(j)}\mathbf{a}_k^{(j)} + \mathbf{b}^{(j)} \tag{6.4}$$

and the *output activation level* for the kth training stimulus in layer $j+1$:

$$\mathbf{a}_k^{(j)} = \mathbf{f}^{(j)}(\mathbf{u}_k^{(j)}) \tag{6.5}$$

using the *layer transfer function*, $\mathbf{f}^{(j)}$. Note that $\mathbf{a}_k^{(0)} = \mathbf{s}_k$ by definition.

The layer transfer function, $\mathbf{f}^{(j)}$, is the chosen vector-valued transfer function for layer j, $\mathbf{W}^{(j)}$ is the connection matrix from the units in layer $j-1$ to the units in layer j, and $\mathbf{b}^{(j)}$ is the bias vector for the units in layer j. The subscript k refers to the kth training stimulus. The function $\mathbf{f}^{(j)}$ must have continuous second partial derivatives.

For example, let $\mathbf{u} = [u_1, u_2, u_3, u_4, u_5]$. A function form for $\mathbf{f}^{(j)}$ might be:

$$\mathbf{f}^{(j)}(\mathbf{u}) = [u_1, u_2, \mathcal{S}(u_3), exp(u_4^2), 2\mathcal{S}(u_5) - 1] \tag{6.6}$$

where $\mathcal{S}(u_3) = 1/(1 + exp(-u_3))$. The layer transfer function in (6.6) illustrates a situation where the first two elements are *linear transfer functions*, the third element is a *logistic sigmoidal* or just *sigmoidal* function, the fourth element is an example of one type of *radial basis* transfer function, and the fifth element is an example of one type of hyperbolic tangent sigmoidal function (i.e., the asymptotes are -1 and +1 unlike the 0 and 1 asymptotes of the logistic sigmoid).

6.1.4 Backward Propagation of Errors

Let the vector $\delta_k^{(j)} = dE_k/d\mathbf{a}_k^{(j)}$ (i.e., the derivative of the error with respect to the activation pattern vector in layer j evaluated at the current set of weights

and the kth training vector). The vector $\delta_k^{(j)}$ is called the *error signal* for the jth layer of units whose activation levels are defined by the vector $\mathbf{u}^{(j)}$ (note this notation may slightly differ from the Rumelhart et al., 1986, paper). Let the matrix $\mathbf{Df}_k^{(j)}$ be the vector first derivative (i.e., the "jacobian") of the vector transfer function $\mathbf{f}_k^{(j)}$ and k refers to the kth training simulus.

Consider a m-layer feedforward network consisting of layers $1, \ldots, m$. The *backward error propagation algorithm* formula for propagating an error signal for the output units of layer j to the output units of layer $j-1$ (i.e., the input units of layer j) using the formula:

$$\delta_k^{(j-1)} = [\mathbf{W}^{(j)}]^T \mathbf{Df}_k^{(j)} \delta_k^{(j)} \tag{6.7}$$

for $j = 1, \ldots, m-1$. The error signal for the mth layer is the vector $\delta_k^{(m)} = de_k/d\mathbf{a}_k^{(m)}$ where e_k is defined as in (6.1).

For example, suppose $\mathbf{f}$ is defined such that $\mathbf{u} = \mathbf{f}(\mathbf{u})$ (i.e., a "linear transfer function"), then $\mathbf{Df}$ is the identity matrix. As another example, suppose that all output units are logistic sigmoidal units so that the activation level of the ith output unit is given by $\mathcal{S}(u_i)$ where u_i is the netinput to the ith output unit and $\mathcal{S}(u_i) = 1/(1 + exp(-u_i))$. Assume there are only three units in a layer. In this case, $\mathbf{f}$ will be a three-dimensional vector whose ith on-diagonal element is defined by $\mathcal{S}(u_i)$ and $\mathbf{Df}$ will be a three-dimensional matrix whose off-diagonal elements are zero and whose ith on-diagonal element is $\mathcal{S}(u_i)(1 - \mathcal{S}(u_i))$.

If the prediction error is a sum-squared error signal then

$$\delta_k^{(m)} = -(\mathbf{t}_k - \mathbf{a}^{(m)}) \tag{6.8}$$

If the prediction error is a cross-entropy error function, then

$$\delta_k^{(m)} = -(\mathbf{t}_k - \mathbf{a}^{(m)})./[\mathbf{a}^{(m)}. * (\mathbf{1} - \mathbf{a}^{(m)}] \tag{6.9}$$

where it is assumed that the m layer consists of logistic sigmoidal functions. Note the notation ./ refers to element by element division.

6.2 Parameter Estimation Formulas

6.2.1 Calculation of the gradient vector (and overall error).

For each data record ($k = 1, \ldots, N$), compute the following:

- *Step 1: Forward Activation Propagation.* Given stimulus $\mathbf{s}_k$, propagate activation levels forward layer by layer through the network using the forward propagation equations (equations (6.4) and (6.5)) in order to generate output activation vector $\mathbf{a}_k^{(m)}$ for the last layer (layer m).
- *Step 2: Compute data record error.* Compute the output error E_k using equation (6.1).

- *Step 3: Backward error Propagation.* Compute the output error signal $\delta_k^{(m)}$ for layer m and propagate this backwards so that an error signal is obtained for layers $m-1$ through layer 1 using equation (6.7).

- *Step 4: Compute and store data record gradient.* Let the column vector $\mathbf{f}_k^{(j-1)}$ be defined such that $\mathbf{f}_k^{(j-1)} = [\mathbf{a}_k^{(j-1)}\ ,\ 1]^T$.

$$\mathbf{G}_k^{(j)} = \delta_k^{(j)} \mathbf{D}\mathbf{f}_k^{(j)} [\mathbf{f}_k^{(j-1)}]^T + (1/\sigma_w^2)[\mathbf{W}\ ,\ \mathbf{b}] \tag{6.10}$$

  for $j = 1, \ldots, m$.

  Let the notation $STACK[\mathbf{G}_k^{(j)}]$ refer to a vector formed by "stacking" the rows of the matrix $\mathbf{G}_k^{(j)}$ one on top of another. Thus, if $\mathbf{G}_k^{(j)}$ has d rows, then the first d elements of $STACK[\mathbf{G}_k^{(j)}]$ will be the first row of $\mathbf{G}_k^{(j)}$, the second d elements of $STACK[\mathbf{G}_k^{(j)}]$ will be the second row of $\mathbf{G}_k^{(j)}$, and so on. It will also be convenient to have the function $UNSTACK$ which is the inverse of the function $STACK$. Now define the column vectors: $\mathbf{g}_k^{(j)} = STACK[\mathbf{G}_k^{(j)}]$ and $\mathbf{g}_k = [\mathbf{g}_k^{(1)}, \ldots, \mathbf{g}_k^{(m)}]$. The vector $\mathbf{g}_k$ is called the *data record gradient* for the kth data record in the training data and should be temporarily stored once it is computed.

6.2.2 Example learning algorithms.

Important indexing issues and definitions. Define $\mathbf{y}^{(j)} = STACK[\mathbf{W}^{(j)} \quad \mathbf{b}^{(j)}]$ and $\mathbf{y} = [\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(m)}]$. We will use the index t to denote the *iteration number* of the learning algorithm. *The index t is not the same as the index k!! The k index ranges over the set of N data records in the training set, while the t index identifies the learning algorithm iteration!!* Using this notation, $E_k(\mathbf{y}_t)$ will be used to refer to the error for the kth data record at iteration number t of the learning algorithm. Similarly, $\mathbf{g}_k(\mathbf{y}_t)$ refers to the gradient of the error for the kth data record at iteration number t of the learning algorithm. Similarly, the variable N **is NEVER** equal to the number of learning trials but is **ALWAYS** defined as the number of data records in the training set.

Initialization issues. In all of the following learning algorithms the choice for the initial value $\mathbf{y}_0$ is absolutely crucial. The elements of this vector should be chosen so that they are independent and identically distributed with mean zero and variance σ_0^2. The parameter σ_0^2 should be "highly visible" to the user. Learning then proceeds until the parameters of the network converge. If σ_0^2 is too small, then the learning process will be very long. If σ_0^2 is too large, then the learning process will rapidly converge to a crummy solution. It is best to err on the side of selecting σ_0^2 to be too small.

Classic on-line backprop. The *classic on-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta \mathbf{g}_k(\mathbf{y}_t) \tag{6.11}$$

where k (the particular training data record index) is chosen at random on iteration t of the learning algorithm. The parameter η is a strictly positive

number called the *learning rate* or *stepsize* for the learning algorithm (e.g., $\eta = 1/N$ is a good choice).

Stochastic approximation on-line backprop. An improved version of the *classic on-line* algorithm called *stochastic approximation on-line* is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \left(\frac{\eta}{t_0 + t}\right) \mathbf{g}_k(\mathbf{y}_t) \tag{6.12}$$

where t_0 is a positive integer whose magnitude is roughly equal to the number of records, N, in the training data set. The stepsize η is a positive number (e.g., $\eta = 1$ is a good choice).

Classic off-line ("batch") backprop. The *classic off-line* or "classic batch" backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta \mathbf{g}(\mathbf{y}_t) \tag{6.13}$$

where the *average gradient* $\mathbf{g}$ is defined by the formula:

$$\mathbf{g} = (1/N) \sum_{k=1}^{N} \mathbf{g}_k. \tag{6.14}$$

A good choice for η for this algorithm would be $\eta = 1$ but as usual η must be a strictly positive real number.

Offline backprop with automatic stepsize selection. Define the *average error* E by the formula:

$$E = (1/N) \sum_{k=1}^{N} E_k. \tag{6.15}$$

An improved version of the *classic off-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{y}_{t+1} = \mathbf{y}_t - \eta_t \mathbf{g}(\mathbf{y}_t) \tag{6.16}$$

where $\mathbf{g}$ is defined as in (6.14) and η_t is chosen such that quantity $E(\mathbf{y}_t - \eta \mathbf{g}(\mathbf{y}_t))$ is minimized with respect to η (treating $\mathbf{y}_t$ as a known constant) subject to the constraint that $0 \leq \eta_t \leq 1$. A "line search" routine from *Numerical Recipes* may be used to solve this one-dimensional optimization problem.

6.3 Data Analysis Algorithms

Conditions for Statistical Inference to be Valid.

The four assumptions that we make here are as follows. All four of these assumptions must be satisfied in order to guarantee reliable statistical inferences.

Log-likelihood Objective Function Assumption. First, it is assumed that the prediction error objective function e_k may be interpreted as the negative natural logarithm of the likelihood of a training data record given the network's

parameters. When the target $\mathbf{t}_k$ takes on a continuous range of values and the least-squares error function defined in (6.2) then this assumption is satisfied by assuming a multivariate conditional Gaussian distribution for the target vector given the input vector and network parameters. When the target consists of elements taking on the values of either zero or one and the output units are sigmoidal, then this assumption is satisfied by interpreting the activation level of an output unit as the probability that the target for that output unit will take on the value of one and using the cross-entropy function defined in (6.3). The best way to satisfy this assumption is to make sure that the user only gets to choose from a set of valid log-likelihood objective functions.

Goodness-of-Fit Assumption. Second, it is assumed that the predictions of the model with respect to the training data would be "perfect" if the model was exposed to a sufficiently large set of training data and was provided with a sufficiently large number of learning iterations. That is, we assume the model provides a good fit to the observed training data. More technically, it is assumed that the probabilistic modeling assumptions introduced in the "log-likelihood objective function assumption" are valid.

We will call the parameters which provide a good fit to the data $\mathbf{y}^*$. Note that in many real-world situations, this assumption will not be satisfied. For example, suppose we teach a linear ANN the "exclusive-or" problem. No matter how long we train the ANN, the ANN will never "fit the data".

To check this assumption, provide the user with plots of training and test set performance. Warn user that statistical inferences may not be reliable if the ANN is not a good data model.

Convergence Assumption. Third, it is assumed that the gradient vector, $\mathbf{g}(\mathbf{y}^*)$, is sufficiently small in magnitude. Typically, we check this assumption by seeing if the element in $\mathbf{g}_t$ with the largest absolute value is less than ϵ. The constant ϵ is typically about 0.0001. If this assumption is not satisfied, the user should be recommended to adjust the parameters of the learning procedure or alter the initial conditions (initialization of the ANN connection weights).

Convergence to a Strict Local Minimum Assumption. Fourth, it is required that this local uniqueness assumption be satisfied. Typically, we check this assumption by computing the matrix:

$$\mathbf{H}^* = (1/N) \sum_{k=1}^{N} \mathbf{g}_k(\mathbf{y}^*)[\mathbf{g}_k(\mathbf{y}^*)]^T. \tag{6.17}$$

All eigenvalues of $\mathbf{H}^*$ can be shown to be always non-negative (if you see any negative eigenvalues then there are numerical problems with the eigenvalue calculation routine). Compute the condition number of $\mathbf{H}^*$ which is defined as the largest eigenvalue of $\mathbf{H}^*$ divided by the smallest eigenvalue of $\mathbf{H}^*$. If the condition number is less than some constant K then we say the local uniqueness assumption is satisfied. A constant K with a value of 1000 is acceptable. Most statisticians use a constant K of value of 100. Usually, $K > 1000000$ is too large. If this assumption is not satisfied, the user should consider collecting more data, changing the network architecture, or decreasing the value of σ_w. Of

course the preferred course of action is collecting more data. Note that altering the value of σ_w is formally equivalent to changing the network architecture.

Wilks Likelihood Ratio Test

[1] Wilk's Likelihood Ratio Test works as follows. Estimate the parameters of the network. Call that set of parameters $\mathbf{y}_{full}$. Now constrain r free parameters of the network equal to the value of zero and re-estimate the network parameters using $\mathbf{y}_{full}$ as the "initialization" for the learning algorithm. The resulting parameter estimates will be called $\mathbf{y}_{reduced}$. It is assumed that $\mathbf{y}_{full}$ satisfies all four assumptions and $\mathbf{y}_{reduced}$ satisfies all assumptions except for the Goodness-of-Fit assumption. It is assumed that $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ are sufficiently close to one another in the sense that under the null hypothesis that any difference between $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ is due to the fact that we only have a finite random sample.

Using equation (6.15), compute

$$G^2 = -2N[E(\mathbf{y}_{full}) - E(\mathbf{y}_{reduced})]. \tag{6.18}$$

If G^2 exceeds the critical value of a chi-square random variable with r degrees of freedom at the α significance level (r=number of parameters in full model minus number of parameters in reduced model), then we reject the null hypothesis at the α significance level that the r parameters could be set equal to zero without hurting the fit of the model to the data.

Another Version of Wilks Test called the Wald Test

[2] A generalization of the Z-test can be used to answer the same questions as Wilks test. This statistical test is called the Wald Test. It has the advantage that you can do the statistical test with only one set of parameter estimates. First, you estimate the parameter vector which we will call $\mathbf{y}^*$. Now suppose you want to test the null hypothesis that r specific elements of $\mathbf{y}^*$ are equal to zero. We will denote this subvector by the r-dimensional vector $\mathbf{q}^*$. Obviously this is just as general as Wilks test because you can select the r elements to the connection strengths leaving a particular input unit (for example).

Now you define a *selection matrix* $\mathbf{S}$ which has r rows and h columns where h is the dimension of $\mathbf{y}^*$. You want to select $\mathbf{S}$ in just such a way so that $\mathbf{q}^* = \mathbf{S}\mathbf{y}^*$. This can always be done and it might make sense to have a set of "canned" selection matrix generation routines available so the user doesn't have to figure this one out.

For example, suppose $\mathbf{y}^* = [1, 2, 11, 4, 6]$ and we want to test the null hypothesis that the third and fourth weights are both equal to zero. Then we would

[1] S. S. Wilks, "The large-sample distribution of the likelihood ratio for testing composite hypotheses," *The Annals of Mathematical Statistics* 9(1), 60–62, 1938, https://doi.org/10.1214/aoms/1177732360.

[2] A. Wald, "Tests of statistical hypotheses concerning several parameters when the number of observations is large," *Transactions of the American Mathematical Society* 54(3), 426–482, 1943, https://doi.org/10.1090/S0002-9947-1943-0012401-3.

construct a selection matrix $\mathbf{S}$ with two rows whose first row is [0, 0, 1, 0, 0] and whose second row is [0, 0, 0, 1, 0]. The only technical restriction on the choice of the selection matrix $\mathbf{S}$ is that the rank of the matrix $\mathbf{S}$ must be exactly equal to the number of rows in $\mathbf{S}$ (i.e., the rows of $\mathbf{S}$ must be linearly independent vectors). clearly in this example the dot product of the two row vectors in $\mathbf{S}$ is equal to zero implying that the two row vectors are orthogonal and thus linearly independent.

Once you've got the selection matrix the remaining calculations are straightforward. Compute the formulas (using equation (6.17)):

$$\mathbf{C}^* = \mathbf{S}^*[\mathbf{H}^*]^{-1}(\mathbf{S}^*)^T \tag{6.19}$$

and

$$\mathcal{W}_N = N(\mathbf{S}\mathbf{y}^*)^T[\mathbf{C}^*]^{-1}(\mathbf{S}\mathbf{y}^*) \tag{6.20}$$

where N is the number of data records.

If the Wald statistic $\mathcal{W}_N$ exceeds the value of a chi-square random variable with r degrees of freedom (where r is the number of rows of $\mathbf{S}$) at the α significance level, then reject the null hypothesis that the r selected weights take on the value of zero at the α significance level.

Chapter 7

Documentation

There is a reason that documentation is described even before the most general features of neUROn design. Students of computer science often think of documentation as ancillary to code. Nothing could be further from the truth. When one realizes that code is secondary to documentation, one becomes a mature programmer.

The reason for this transcendence of documentation is actually quite simple. Code is merely an instance of an abstract idea, the algorithm. Different computer languages yield different instances, but the algorithm is at the core. In order to access the algorithm, a method is coded with inputs and outputs. These inputs and outputs are specified by the programmer, and allow the method to become a "black box" which can be used at will, and, if all goes well, as part of a library of methods that can be used and reused often.

The job of a student programmer is to write code. The job of a master programmer is to write the specifications for the inputs and outputs of methods, which when combined, solve a specific problem. After the specifications are written, then the algorithms within the methods are coded. Documentation ultimately becomes these specifications.

Documentation thus has several purposes. It serves the programmer who wants to reuse code written in the past, but that use is for minor programming tasks. It serves the programmer who wants the code used by other programmers, a legacy for major programming tasks. Ultimately, it serves the programmer who wants the specification to be used by other programmers, and that is the stuff which drives, for example, national economies.

> The neUROn programmer will document code in several ways, but in all, *documentation will be written before the code.* Not after, not even while. Documentation will be written *before* the code.

Each method (and feature corresponding to a set of methods) coded in neUROn is to be documented in 3 specific places:

1. Here, in the Design Manifest. Here a natural language description, algorithmic description if applicable, and specification is to be documented.

2. Immediately prior to the method in the code, as a specification.

3. Throughout the code.

As an example, here is the code for the dot product method in matrix.h:

7.1 Documentation as it appears in the Design Manifest:

Given a matrix object M, and vectors $\vec{v}$ and $\vec{o}$, where $M \cdot \vec{v} = \vec{o}$, e.g.

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{v_1} \\ \vec{v_2} \\ \vdots \\ \vec{v_c} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

the following dot product operators are available in the Matrix class:

- M.dotprod(vector& v) returns a new vector object which is the dot product of the matrix and the passed vector
- M.dotprod(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the matrix and the passed vector v_{in}
- M.dotprod(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the matrix and the passed vector v_{in}

Examples

```
// Matrix A has 4 rows and 3 columns, vector v1 has 3 elements
vector< double > v2;
v2 = A.dotprod( v1 ); // v1 is unchanged
                      // v2 now contains the dot product

// vector v3 has 4 elements
A.dotprod( v1, v3 ); // v1 is unchanged
                     // v3 now contains the dot product

// vector v4 has 5 elements
A.dotprod( v1, v4, 0, 3 ); // v4[0] through v4[3] now
                           // contains the dot product
```

Notes

For M.dotprod(v), the number of columns in M *must equal* the number of elements in v. For M.dotprod(v_{in}, v_{out}), the number of columns in M *must equal* the number of elements in v_{in}, and the number of rows in M *must equal* the number of elements in v_{out}. Because M.dotprod(v) must construct a new vector object, it is less efficient than M.dotprod(v_{in}, v_{out}).

7.1.1 General principles of documentation in the Manifest: emphasis on use of the method

The emphasis of documentation in the Manifest is on how the method is used. The programmer calling the code should not need to know anything about the inner workings of the method, only how it is called, and what it returns.

7.1.2 Emphasis on readability

Documentation in the manifest is written in a conversational tone so that it is easy to read and understand.

7.1.3 Examples

Good, simple examples are provided to demonstrate the use of the method.

7.1.4 Important notes

Any specific behavior of the code is noted so that it may be used in the most effective manner.

7.2 Documentation as it would appear immediately before the procedure in the code:

```
// Dot product
// For vectors o and v and Matrix M, where M . v = o
//
//                          [ v1
// [ M11 M21 M31 M41    .     v2  =  [ o1
//   M12 M22 M32 M42 ]        v3       o2 ]
//                            v4 ]
//
// ( Note that the number of columns in M, 4, is the number of rows in
v )
//    by convention, ALL vectors are assumed to be vertical
// Calling sequence: M.dotprod( v, o );
```

7.2.1 General principles of documentation immediately before the procedure in the code: clear argument specification

Inputs and when applicable, returned values to the method are clearly and succinctly stated so that casual inspection will allow the code to be used correctly.

7.2.2 Critical notes

Critical usage issues (such as the assumption that all vectors are assumed to be vertical in this example) are included in the comments immediately prior to the procedure.

7.3 Documentation in the code itself:

Finally, the code itself is written and commented *while it is being written.* Debugging is the *last* step, and comments are *modified* as the method is debugged.

```
template< class T >
vector< T >& Matrix< T >::dotprod( const vector< T >& iVec, vector< T
>& oVec ) const
{
   // Number of Matrix columns must equal number of input vector
   elements
   // Number of Matrix rows must equal number of output vector
   elements
   assert ( iVec.size() == ncols_ && oVec.size() == nrows_);

   // Set a pointer ( pData ) to Matrix's data array
   T* pData = data_;

   // Set a const iterator ( pi ) to the input vector
   vector< T >::const_iterator pi;

   // Set an iterator ( po ) to the output vector
   vector< T >::iterator po;

   // Clear output vector by setting all elements to zero
   std::fill( oVec.begin(), oVec.end(), 0 );

   // Calculate the dot product using vector iterators
   for ( po = oVec.begin(); po != oVec.end(); ++po )
      // Calculate an output vector element = Matrix row . input
      vector
      for ( pi = iVec.begin(); pi != iVec.end(); ++pi )
         *po += ( ( *pData++ ) * ( *pi ) );
```

```
   return oVec; // enables use in vector formulae
}
```

7.3.1 General principles of documentation within code:

Note how easy the code is to read. These basic principles are to be followed:

- *Comments are meaningful.* Nothing is more useless than a comment which simply restates the code, for example:

  ```
  // Set the end of outputText to NULL
  outputText[ 9 ] = '\0';
  ```

  is truly useless, whereas

  ```
  // Truncate output string
  outputText[ 9 ] = '\0';
  ```

  while less verbose, is actually more meaningful.

- *Comments are capitalized correctly.* Using all capitals is equivalent to email shouting, and equally offensive. Use

  ```
  // Set the end of outputText to NULL
  ```

  instead of:

  ```
  // SET THE END OF OUTPUTTEXT TO NULL
  ```

- *Pretty printing makes the code easier to read.* Consistent tabbing and bracket placement go a long way towards making clear code.

- *Frequent meaningful comments make the code easier to read.* Every single code block is documented with a meaningful comment.

7.3.2 Variable names:

Variables are to be meaningfully named. It is difficult to sort through previously written code when variables assume generic names like 'x' or 'px'. Verbose variable names are rarely a hindrance. Often student programmers dislike verbose variable names because they do not allow easy packing of variables into 1 line of code. However, this very restriction often makes the code much easier to read by requiring the programmer to split the code into several lines. Consider the example:

```
irpp = (EXP(LN((EXP(n*LN(1+(i/n)))-1)+1)/(ap*ppy))-1)*ap*ppy;
```

While it does fit on 1 line, it is thoroughly unreadable. As such, it is difficult for the programmer to debug while programming, and nearly impossible for succeeding programmers to intelligently use in future code. Consider the alternative:

```
// Use times_per_year_compounded = 2 for Canada, 12 for U.S.
incremental_rate = interest_rate / times_per_year_compounded;

// Calculate effective annual rate
effective_rate = EXP( times_per_year_compounded
   * LN( 1 + incremental_rate ) ) - 1;

// Finally, calculate incremental_interest = interest rate per period:
amortization = amortization_period * payments_per_year;
incremental_interest = (EXP( LN( effective_rate + 1 ) / amortization )
- 1 )
   * amortization;
```

Not only is the algorithm that the programmer used to calculate the interest rate per period manifestly clear, the variables are so well named as to make the '//' comments complimentary rather than critical. And when a future programmer sees the variable 'incremental_interest' 2000 lines of code later, he or she will have half a clue about what that variable means, whereas the meaning of 'irpp' would be completely opaque.

7.3.3 Space

One of the simplest ways to make code readable is to insert space around operators in code. Consider the example in section 7.3.2,

```
irpp = (EXP(LN((EXP(n*LN(1+(i/n)))-1)+1)/(ap*ppy))-1)*ap*ppy;
```

One of the difficulties in reading this code is the lack of spaces between operators.

Rules for spaces in coding neUROn2++ are:

- Put spaces after binary operators (like =):

  ```
  a = b + c;
  ```

  rather than:

  ```
  a=b+c;
  ```

- After control keywords like 'for' and 'while', put spaces around the ()s, e.g.:

```
for ( i = 1; i <= 10; i++ ) {
```

rather than:

```
for(i = 1; i <= 10; i++){
```

- Inside of brackets, braces or parentheses, put spaces right after the first bracket, brace, or parenthesis, and right before the second, e.g.:

  ```
  a[ i ] = 10;
  ```

  rather than:

  ```
  a[i] = 10;
  ```

Chapter 8

C++ coding and building

neUROn was written in C, and took advantage of the high- and low-level compilation features available in that language. While neUROn2++ was written in C++, coding began shortly after the first standardized C++ implementation became available. Because C++ is a superset of C, neUROn++ became a hodgepodge of C and C++ coding styles, and ultimately, became too cumbersome to maintain, let alone extend. neUROn2++ is designed to take advantage of the extensibility offered by the object-oriented nature of C++, but it is also designed to take advantage of other significant advances in C++ such as operator and method overloading. The following C++ style rules are expected to be followed closely, but are not intended to be an exhaustive list. For those interested in an excellent source of good C++ coding style rules, see Deitel and Deitel's *C++ How to Program.*[1]

In general, while coding, think of the 3 cardinal rules of good programming: code must be

- *Readable* (see Chapter 7 on rules for documentation.)
- *Efficient*
- *Bulletproof*

8.1 Current build and portability

neuron 3.0 is a C++17 project built with CMake. GSL and a thread library are its compiled dependencies; the browser page is embedded into the executable at configure time. A Release build is important because an unoptimized engine is approximately an order of magnitude slower.

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
```

[1] Deitel and Deitel, C++ How to Program, 3rd edition, Prentice Hall, 2001, ISBN 0-13-089571-7

```
cmake --build build --parallel
./build/neuron --version
```

On macOS, install the prerequisites with `brew install cmake gsl`. On Ubuntu or Debian, use `apt install build-essential cmake libgsl-dev`. There are no hand-maintained IDE project files or compiler-specific RTTI settings.

Portability is expressed in `CMakeLists.txt` and verified by continuous integration, rather than maintained as parallel compiler projects.

8.2 Use friends sparingly

Friend methods are to be used *only* when absolutely necessary. Consider using a friend a failure to find a good solution. Obviously, friends are used in overloading the ostream operator << and istream operator >>, e.g.:

```
friend ostream& operator << ( ostream&, const ... );
```

because there is simply no other good way to overload the ostream operator for a new object. However, friends should *not* be used to overload mathematical operators such as + = and +. *Do not use a friend unless you discuss it with the neUROn2++ project group first.*

8.3 Use const correctness

Const correctness provides one way to write efficient and bulletproof code right from the start. By being const-correct, you think about which objects can be mutated in a method and which cannot. By not allowing certain methods to mutate objects, you prevent particularly intractable runtime errors, and by judiciously allowing certain methods to mutate objects (e.g. passed array references), you know exactly where your code is behaving efficiently, and exactly where runtime errors may be introduced. *The label const is to be applied while code is written, not after. Constness applied as a patch to already written code is uniformly considered to be very poor programming practice.*

8.4 Validate invariants at the appropriate boundary

Assertions remain useful for programmer invariants in development builds, but Release builds define `NDEBUG` and therefore cannot depend on them for user input or memory safety. Public handlers validate request values before calling the engine. Matrix element access performs unconditional bounds checks and throws `BoundsViolation` in every build, and the vector operations of section 9.1.1 likewise reject bounds and dimension violations in every build. Both halves of the

numerical layer are required: until 2026, every precondition in “vector_ops.h” was an assertion, and because Release is the default configuration none of them had ever executed in the test suite.

An assertion can still document an internal precondition such as equal matrix dimensions:

```
// Catch different size matrices
assert ( nrows_ == rhs.nrows_ && ncols_ == rhs.ncols_ );
```

Code reachable from data, GUI, or API input must additionally reject invalid dimensions through ordinary control flow or an unconditional exception.

Chapter 9

vector and Matrix

Establishment of numerical vector and 2-dimensional matrix objects in neURON2++ serves as the basis by which clear, concise, efficient neural computational code may be written. The header file "vector_ops.h" extends the STL <vector> by adding simple unary and binary mathematical operations, and overloaded ostream << and istream >> operators for simplified output and input. The header file "matrix.h" implements a matrix class for simplified 2-dimensional matrix operations.

9.1 vector_ops.h

Inclusion of the header file "vector_ops.h" overloads operators and adds methods to the <vector> STL. Please see <vector> STL documentation for standard <vector> STL methods.

9.1.1 Bounds and dimension violations

Many of the operations below walk two vectors in lockstep, or index into a vector by position. Violating such a precondition is not a data condition to be handled; it is a programming error that reads or writes memory the operation does not own. These preconditions are therefore checked *unconditionally*, and the checks are not assertions: Release builds define `NDEBUG` (see section 8.4), so an assertion cannot protect memory in the shipped binary.

Three exception types are declared in **namespace nvec**:

- `nvec::SizeMismatch`: two parallel containers have incompatible lengths.
- `nvec::RangeViolation`: a position, range or extent lies outside the container it indexes.
- `nvec::EmptyVector`: the operation has no value on an empty set. This applies to maxabs and minabs only; a *sum* has an identity, so squared, dotprod and sumSquaredDifference all return 0 over empty operands.

Each operation below states its own precondition and which of these it throws. The checks are performed once, before the elementwise loop, never per element.

9.1.2 Unary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) vector and left hand side (lhs) vector, returning the result in the left hand side (lhs) vector:

- $+=$: unary addition
- $-=$: unary subtraction
- $*=$: unary multiplication
- $/=$: unary division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
       data2[ 4 ] = { 2, 3, 4, 5 },
       data3[ 3 ] = { 8, 9, 10 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 ), v2( data2, data2 + 4 );

// Unary addition element by element
v1 += v2;

// Different size vectors
v3 += v1; // v1[ 3 ] will be ignored
```

Notes

The vectors may have different numbers of elements, but the right hand side (rhs) vector size *must be* greater than the left hand side (lhs) vector. If the vectors have different sizes, the remaining elements at the end of the rhs vector that are not represented in the lhs vector will be ignored.

This inequality is deliberate and is relied upon: the result carries the lhs size, which is how the bias slot is dropped when a hidden error vector of n elements is multiplied by an output weight vector of $n + 1$. A rhs *shorter* than the lhs is the error, because the operation would then read past its end, and throws `nvec::SizeMismatch` (section 9.1.1).

9.1.3 Unary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a right hand side (rhs) scalar and left hand side (lhs) vector, returning the result in the left hand side (lhs) vector:

- $+=$: unary scalar addition
- $-=$: unary scalar subtraction
- $*=$: unary scalar multiplication
- $/=$: unary scalar division

Example

```
// Standard C-style array
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 );

// Unary scalar addition
v1 += 2.0;
```

Notes

Type resolution may be a problem if the right hand side argument is not specified, e.g. "v1 + = 2;" may give an error. To fix, either cast appropriately, or include decimal points in double values, e.g. "v1 + = 2.0;".

9.1.4 Binary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) vectors, returning the result for allocation in a new left hand side (lhs) vector:

- $+$: binary addition
- $-$: binary subtraction
- $*$: binary multiplication
- $/$: binary division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
       data2[ 4 ] = { 2, 3, 4, 5 },
       data3[ 3 ] = { 8, 9, 10 };

// vector's std array ctor
vector< double > v1( data1, data1 + 4 ),
   v2( data2, data2 + 4 ), v3( data3, data3 + 3 ),
   v4, v5;

// Binary addition element by element
v4 = v1 + v2;

// Binary addition with different size vectors
v5 = v3 + v2; // v5 will have 3 elements
```

Notes

In the passed arguments to the overloaded operator, the left hand side (lhs) is *not* a reference, but a local copy of the lhs vector, and a local copy is returned. There is thus some inefficiency introduced in the use of binary vector operators. The vectors may have different numbers of elements, but like their unary counterparts, the right hand side (rhs) size *must be* greater than the size of the lhs vector. The final size of the returned vector will be the same as the lhs vector size.

9.1.5 Binary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a vector and a scalar, and return the result for allocation in a new vector:

- $+$: binary scalar addition
- $-$: binary scalar subtraction
- $*$: binary scalar multiplication
- $/$: binary scalar division

Example

```
// Standard C-style arrays
double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 };

// vector's std array ctor
```

```
vector< double > v1( data1, data1 + 4 ), v2;

// Binary scalar addition
v2 = v1 + 3.0;
```

Notes

Type resolution may be a problem if the right hand side argument is not specified, e.g. "v1 + = 2;" may give an error. To fix, either cast appropriately, or include decimal points in double values, e.g. "v1 + = 2.0;". In the passed arguments to the overloaded operator, the left hand side (lhs) is *not* a reference, but a local copy of the lhs vector, and a local copy is returned. There is thus some inefficiency introduced in the use of binary vector operators.

9.1.6 Overloaded == and ! =

The boolean operators == and ! = perform element by element comparisons between vectors, == returning *true* if they are equal, and ! = returning *false* if they are not.

Example

```
vector< double > v1, v2;

// Code to load the vectors would follow...

if ( v1 == v2 )
   cout << "They're equal" << endl;
else
   cout << "They're not equal" << endl;
```

9.1.7 Overloaded <<

The ostream operator << is overloaded for vector output.

Example

```
cout << "v1 = " << v1 << endl;
```

Notes

This overloaded operator may also be used for vector of vector (etc.), and will insert double (triple, etc.) spaces between the output vectors.

9.1.8 Overloaded >>

The istream operator >> is overloaded for vector input.

Example

```
vector< double > v1( 5 );
cout << "Enter a 5 element vector: ";
cin >> v1;
cout << "Your 5 element vector is " << v1 << endl;
```

9.1.9 Dot product

The *dotprod* method may be used to compute a scalar dotproduct from 2 vectors. Its signature is:

dotprod(vector& v_1, vector& v_2)

Example

```
x = dotprod( v1, v2 );
```

Notes

v_1 and v_2 *must have* the same number of elements, and `nvec::SizeMismatch` (section 9.1.1) is thrown if they do not. Note that this is *equality*, not the inequality the unary operators accept: a dot product taken over a prefix of the longer vector is a different scalar, and returning it silently is precisely what the check exists to prevent. Two *empty* vectors are accepted and give 0, the identity of a sum.

The method sumSquaredDifference(vector& v_1, vector& v_2) returns $\|v_1 - v_2\|^2$ under the same contract, without constructing the difference vector.

9.1.10 Function passing

The following methods allow functions to be applied to a vector element by element:

- func(vector& v_{in}, function f, vector& v_{out}): applies function f element by element to vector v_{in}, populating vector v_{out} with the result.
- func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*): applies function f element by element to vector v_{in} in the range $v_{in}[begin]$ to $v_{in}[end]$, populating vector v_{out} in the range $v_{out}[begin]$ to $v_{out}[end]$ with the result.
- func(vector& v, function f): applies function f element by element to vector v, and returns a new vector object.
- func(vector& v, function f, unsigned *begin*, unsigned *end*): applies function f element by element to vector v in the range $v[begin]$ to $v[end]$, and returns a new vector object of size $v[begin] - v[end] + 1$.

Example

```
\index{Transfer function!Sigmoidal}

#include <vector>
#include <math.h>

using namespace std;

#include "vector_ops.h"

// Define sigmoidal function as a class which extends
// unary_function.  This is necessary for algorithm::transform.
class sigmoidal : public unary_function< double, double >
{
   public:
      inline double operator()( const double& x )
      {
         return 1.0 / ( 1.0 + exp( -x ) );
      }
};

int main() {
   // Standard C-style arrays
   double data1[ 4 ] = { 1.0, 2.0, 3.0, 4.0 },
          data2[ 4 ] = { 2, 3, 4, 5 };

   // vector's std array ctor
   vector< double > v1( data1, data1 + 4 ),
      v2( data1, data1 + 4 ), v3( 4 ), v4;

   // Apply sigmoidal function to each element of v1
   // Populate v3 with the result
   func( v1, sigmoidal(), v3 );

   // Apply sigmoidal function to v1 with itself as result
   func( v1, sigmoidal(), v1 );

   // Apply sigmoidal function to each element of v2
   // After this function call, v4 will contain the result
   v4 = func( v2, sigmoidal() );

   // Copy vector v1 into a new vector v5
   vector< double > v5( v1 );

   // Apply the sigmoidal function to elements 1 through 2
```

```
   // in vector v1, with the result placed in vector v5
   // elements 1 through 2
   func( v1, sigmoidal(), v5, 1, 2 );

   // The receiving vector may be smaller than the sending
   // vector
   vector< double > v6( 3 );
   func( v1, sigmoidal(), v6, 0, 2 ); // v1 has 4 elements
                                      // v6 has 3 elements

   // Make a new vector
   vector< double > v7;

   // And put the sigmoidal function applied to vector v1
   // elements 1 through 2 into it
   v7 = func( v1, sigmoidal, 1, 2 ); // v7 has 2 elements

   return 0;
}
```

Notes

The passed function *must* be defined as a class extension of unary_function as demonstrated in the example. Common functions useful for neural computation (e.g. sigmoidal) are packaged with neUROn2++.

If a vector to be populated is passed without bounds arguments, it *must have* the same number of elements as the input vector. If a vector to be populated is passed with bounds arguments, the size of the vector *must be* greater than the upper bound. As func(vector& v, function f) and func(vector& v, function f, unsigned *begin*, unsigned *end*) must construct new vectors, they are less efficient than func(vector& v_{in}, function f, vector& v_{out}) and func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*). func(vector& v_{in}, function f, vector& v_{out}) and func(vector& v_{in}, function f, vector& v_{out}, unsigned *begin*, unsigned *end*) are preferred.

To use a class extension of unary_function with a scalar, the following may be written (using sigmoidal as an example):

```
double x, y;
y = sigmoidal()( x );
```

is valid if sigmoidal is defined as described above.

Notes

The two forms taking v_{out} write into a vector the caller supplies; neither ever resizes it, because silently growing a destination would conceal a defect in the

shape of the model that supplied it. Their preconditions, and the exceptions of section 9.1.1:

- func(v_{in}, f, v_{out}): v_{in} and v_{out} *must have* the same number of elements, and `nvec::SizeMismatch` is thrown if they do not. This is equality rather than the inequality the unary operators take, deliberately: the operation writes v_{in}.size() elements into v_{out}, so a longer input runs off the end of the destination. A caller that wants only part of a vector says which part, through the ranged form below.
- func(v_{in}, f, v_{out}, *begin*, *end*): *begin must not exceed end*, and *end must be* within *both* v_{in} and v_{out}. `nvec::RangeViolation` is thrown otherwise.
- func(v, f, *begin*, *end*): *begin must not exceed end*, and *end must be* within v; the returned vector is allocated at the correct size and cannot be overrun. `nvec::RangeViolation` is thrown otherwise.

9.1.11 Calculating the sum of squares of a vector elements

The method squared(vector& v) returns the value of the sum of squares of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double sum;
sum = squared( v );
```

9.1.12 Calculating the maximum absolute value of vector elements

The method maxabs(vector& v) returns the maximum absolute value of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double max;
max = maxabs( v );
```

Notes

v *must not be* empty, and `nvec::EmptyVector` (section 9.1.1) is thrown if it is. The maximum of an empty set has no value, and returning 0 would be acted upon: this method supplies the largest absolute gradient to the training stopping rule, where a value of 0 means *converged*.

9.1.13 Calculating the minimum absolute value of vector elements

The method minabs(vector& v) returns the minimum absolute value of vector v elements.

Example

```
vector< double > v; // v is then populated with double values...
double min;
min = minabs( v );
```

Notes

v *must not be* empty, and `nvec::EmptyVector` (section 9.1.1) is thrown if it is.

9.1.14 Randomization

namespace nvec

The method random(vector& v, double n) populates a double precision vector v with double precision random numbers between $-n$ and $+n$.

Example

```
vector< double > v( 6 ); // Make a 6 element vector
// Populate it with random numbers between -0.5 and +0.5,
//    and print it out
cout << "The random vector is " << nvec::random( v, 0.5 );
```

Notes

This method is only coded for vectors of type double, e.g. vector< double >.

9.1.15 Random positions

namespace nvec

The method random_positions(vector& v) populates an unsigned vector v with a series of unique integers from 0 to $n-1$, where n is the number of elements in the vector.

Example

The example code:

```
vector< unsigned > v( 6 );
cout << nvec::random_positions( v ) << endl;
```

May yield:

```
5 2 1 3 4 0
```

Notes

For obvious reasons, this method is only coded for vectors of type unsigned, e.g. vector< unsigned >.

9.1.16 Converting a vector of vector into a vector

The following methods allow conversion of a vector of vector into a vector, thus "flattening" the vector of vector:

- flatten(vector< vector >& *container*, vector& *v*) converts the data structure vector of vector *container* to passed vector *v*, returns a reference to vector *v*.
- flatten(vector< vector >& *container*) converts the data structure vector of vector *container* to a new vector, returns a reference to the new vector.

Example

See section 10.9 for method *variable_parse*.

```
// Create a vector of vector using util::variable_parse
string parse_test = "0-3,5; 4,8; 6,7";
vector< vector< unsigned > > parse_numbers;
parse_numbers = util::variable_parse( parse_test );
cout << "Here it is: " << parse_numbers << endl;

// Now flatten it
cout << endl << "Flattening:" << endl;
vector< unsigned > v_flat;
flatten( parse_numbers, v_flat );
cout << v_flat << endl;
cout << flatten( parse_numbers ) << endl;
```

9.1.17 Binning a vector into a vector of vector

- bin(const vector& *v*, unsigned *b*, bool *binFlag*, vector< vector >& *vv*): partitions vector *v* into the data structure vector of vector *vv* according to specified bin size, with flag *binFlag* indicating number in the last bin:
  - **false** if the last bin contains <= bin size (last bin = remaining elements,
  - **true** if the last bin contains >= bin size (remaining elements are added to the last bin.
- bin(const vector& *v*, unsigned *b*, bool *binFlag*): partitions vector *v* into a new data structure vector of vector according to specified bin size, with flag *binFlag* indicating number in the last bin:

– **false** if the last bin contains <= bin size (last bin = remaining elements,
– **true** if the last bin contains >= bin size (remaining elements are added to the last bin.

and returns a reference to the new data structure vector of vector.

Example

```
double vpart_[ 10 ] = { 0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
9.0 };
vector< double > vpart( vpart_, vpart_ + 10 );
vector< vector< double > > vbin;

bin( vpart, 3, false, vbin );
cout << vpart << endl << "partitioned with small last bin =" << endl <
< vbin;
cout << endl << "partitioned with large last bin =" << endl <<
   bin( vpart, 3, true ) << endl;
```

Produces the following output:

```
0 1 2 3 4 5 6 7 8 9
partitioned with small last bin =
0 1 2  3 4 5  6 7 8  9
partitioned with large last bin =
0 1 2  3 4 5  6 7 8 9
```

9.2 matrix.h and the Matrix class

Including the header file matrix.h allows implementation of the Matrix class, and allows simplified numerical operations on 2 dimensional arrays.

9.2.1 Constructors

The following constructors are implemented in the Matrix class:

- Matrix(): Constructs a Matrix object of zero size, with zero rows and zero columns.
- Matrix(unsigned r, unsigned c): Constructs a Matrix object with r rows and c columns.
- Matrix(unsigned r, unsigned c, *value*): Constructs a Matrix object with r rows, c columns, and fills it with *value*.
- Matrix(vector& q, vector& p^t): Constructs a Matrix object which is the outer product of vectors q and the transpose of p (see 9.2.17).

- Matrix(string& $filename$, unsigned c): Constructs a Matrix object, and loads it from a file named "$filename$" with c columns.
- Matrix(bool *report*, string& $filename$): Constructs a Matrix object, and loads it from a file named "$filename$". Delimeters are any whitespace character, or any character that is non-numeric (+, -, e and E are considered numeric, as is a decimal point.) Rows are delimited by carriage returns in the file, and the number of columns is the maximum number of elements in any one row. Elements in unpopulated rows will be filled with 0, beginning from the *right* hand side of the row. If the bool flag *report* is true, a report will be printed indicating which rows were found to be unpopulated.

Examples

```
// Make some Matrices
Matrix< double > A( 4, 3 ), B, C( 5, 4, 0.01 );

// Get a file name
string filename;
cout << "The file name is ";
cin >> filename;

// Make a Matrix from the file
Matrix< double > C( filename, 3 ), // It is known to have 3 columns
                 D( true, filename ); // Number of columns unknown,
                                      // report will be printed

// Make a matrix from the outer product of vectors v1 and v2
Matrix< double > E( v1, v2 );
```

Notes

A Matrix constructed by *Matrix(r, c)* is **not** initially populated. Consider it to be filled with garbage.

9.2.2 Accessors

Given a Matrix object M, the following accessors are implemented in the Matrix class:

- M(unsigned r, unsigned c): overloaded () to access the element at row r and column c in Matrix *matrix_object*
- M.rows(): returns number of rows in Matrix
- M.cols(): returns number of columns in Matrix

Examples

```
// Make a Matrix with 4 rows and 3 columns
Matrix< double > A( 4, 3 );

// Fill Matrix A with numbers
for ( i = 0; i < 4; i++ )
   for ( j = 0; j < 3; j++ )
      A( i, j ) = 3 * i + j + 1;

// Print out number of rows and number of columns in Matrix
cout << "Matrix has " << A.rows() << " rows and ";
cout << A.cols() << " columns." << endl;
```

9.2.3 Submatrix accessors

The following accessors return a submatrix from a Matrix:

- M.submatrix(unsigned row_1, unsigned row_N, unsigned col_1, unsigned col_N, Matrix& M_{out}): populates passed matrix M_{out} with Matrix M rows row_1 to row_N, and columns col_1 to col_N.
- M.submatrix(unsigned row_1, unsigned row_N, unsigned col_1, unsigned col_N): returns a new Matrix object with Matrix M rows row_1 to row_N, and columns col_1 to col_N.

Examples

```
// Make some matrices
Matrix< double > P( 5, 6, 2.5 ), R( 2, 4 ), S;

// Populate matrix R with the resulting submatrix
P.submatrix( 3, 4, 2, 5, R );

// Construct a new matrix for result
S = P.submatrix( 3, 4, 2, 5 );
```

Notes

Row and column dimensions must be within the bounds of the originating matrix. If a matrix is passed as an argument, its number of columns *must equal* col_N - col_1 + 1, and its number of rows *must equal* row_N - row_1 + 1. A passed matrix to be populated is more efficient than constructing a new matrix for the result.

9.2.4 Row and column accessors

The following accessors return a vector from a row or column of a Matrix:

- M.row(unsigned r, vector& v): populates vector v with row r from Matrix M.
- M.row(unsigned r): returns a new vector with row r from Matrix M.
- M.col(unsigned c, vector& v): populates vector v with column c from Matrix M.
- M.col(unsigned c): returns a new vector with column c from Matrix M.

Examples

```
// Make a matrix with 5 rows and 6 columns, filled with Pi
Matrix< double > P( 5, 6, 3.14159 );

// Make some vectors
vector< double > v5( 5 ), v6( 6 ), v7, v8;

// Put row 2 of Matrix P in vector v6
P.row( 2, v6 );

// Construct a new vector v7 for row 2 of Matrix P
v7 = P.row( 2 );

// Put column 2 of Matrix P in vector v5
P.col( 2, v5 );

// Construct a new vector v8 for column 2 of Matrix P
v8 = P.col( 2 );
```

Notes

For the methods which take passed vectors as arguments, the size of the vector v *must equal* the number of Matrix columns for M.row(r, v), and the size of the vector v *must equal* the number of Matrix rows for M.col(c, v). The methods which take passed vectors as arguments are more efficient than the methods which construct vectors.

9.2.5 Methods which append vectors to Matrices

The following methods append a vector to a Matrix:

- M.addrow(vector& v, Matrix N): populates Matrix N by appending vector v to the *end* of Matrix M as a *row*.

- M.addrow(vector& v): returns a new Matrix with vector v appended to the *end* of Matrix M.
- M.addcol(vector& v, Matrix N): populates Matrix N by appending vector v to the *right* of Matrix M as a *column*.
- M.addrow(vector& v): returns a new Matrix with vector v appended to the *right* of Matrix M.

Examples

```
// Matrix A has 3 rows and 4 columns, vector v1 has 4 elements
// Matrix B has 4 rows and 4 columns
A.addrow( v1, B );
C = A.addrow( v1 );

// Matrix D has 3 rows and 4 columns, vector v2 has 3 elements
// Matrix E has 3 rows and 5 columns
D.addrow( v2, E );
F = D.addrow( v2 );
```

Notes

The number of columns in M and N *must* be the same as the number of elements of vector v for M.addrow, and the number of rows in M and N *must* be the same as the number of elements of vector v for M.addcol.

Because they construct new Matrices, M.addrow(vector& v) and M.addcol(vector& v) are less efficient than M.addrow(vector& v, Matrix N) and M.addcol(vector& v, Matrix N). M.addrow(vector& v, Matrix N) and M.addcol(vector& v, Matrix N) are to be preferred.

M.addcol is computationally expensive, and its use should be minimized.

9.2.6 Methods which replace a row or column of a Matrix with a vector

The following methods replace a row or column of a Matrix with a vector:

- M.replacerow(unsigned n, vector& v): replaces row n in Matrix M with vector v, returning a reference to Matrix M with the result.
- M.replacecol(unsigned n, vector& v): replaces column n in Matrix M with vector v, returning a reference to Matrix M with the result.

Examples

```
// Replace row 3 of Matrix A with vector v1, and print the result
cout << A.replacerow( 3, v1 );
```

```
// Replace column 2 of Matrix A with vector v2, and print the result
cout << A.replacecol( 2, v2 );
```

Notes

The number of columns in M *must* be the same as the number of elements of vector v for M.replacerow, and the number of rows in M *must* be the same as the number of elements of vector v for M.replacecol.

9.2.7 Column inclusion and exclusion

The following operators include or exclude those columns indicated by the passed vector, which includes the (unsigned) column values:

- M.includecols(Matrix& M_{out}, vector< unsigned >& v): includes in the returned Matrix M_{out} only those columns from Matrix M indicated by v, returning a reference to Matrix M_{out} with the result.
- M.includecols(vector< unsigned >& v): returns a new Matrix *with* only those columns from Matrix M indicated by v
- M.excludecols(Matrix& M_{out}, vector< unsigned >& v): excludes in the returned Matrix M_{out} those columns from Matrix M indicated by v, returning a reference to Matrix M_{out} with the result.
- M.excludecols(vector< unsigned >& v): returns a new Matrix *without* those columns from Matrix M indicated by v

Examples

```
// Make a 5x4 Matrix, fill with some numbers
Matrix< double > M( 5, 4 );
for ( unsigned row = 0; row < 5; row++ )
   for ( unsigned column = 0; column < 4; column++ )
      M( row, column ) = 4 * column + row;
cout << "Here is the original Matrix:" << M;

// Make some smaller Matrices
Matrix< double > M1( 5, 2 ), M2, M3( 5, 2 ), M4;

// Make a vector to indicate columns 1 & 3
vector< unsigned > v;
v.push_back( 1 );
v.push_back( 3 );

// Passing a Matrix to include columns
M.includecols( M1, v );
cout << "From the matrix above, including columns "
```

```
   << v << ": " << M1;

// A new Matrix to include columns
M2 = M.includecols( v );
cout << "Returning a new Matrix: " << M2;

// Passing a Matrix to exclude columns
M.excludecols( M3, v );
cout << "From the matrix above, excluding columns "
   << v << ": " << M3;

// A new Matrix to exclude columns
M4 = M.excludecols( v );
cout << "Returning a new Matrix: " << M4;
```

Notes

If the Matrix M_{out} to be populated is passed as an argument, its rows *must* be the same as the number of rows in M, and its number of columns *must* be the same as the difference between the number of columns in M and the number of elements in the passed vector v. The elements in the passed vector v *must* be *unique* (no duplicated values), and no element can be >= to the number of columns in Matrix M.

These methods are useful to include or exclude variables or nodes represented by columns in a Matrix.

9.2.8 Boolean equality operators

The following operators perform boolean equality and inequality comparisons, element by element:

- ==: equality
- ! =: inequality

Example

```
// Make 2 Matrices with 4 rows and 3 columns
Matrix< double > A( 4, 3 ), B( 4, 3 );

// Fill Matrices A and B with numbers

for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
```

```
      B( i, j ) = 3 * i + j + 1;
   }
}

// Compare Matrix A to B:
if ( A == B )
   cout << "They're equal" << endl;
if ( A != B )
   cout << "They're not equal" << endl;
```

9.2.9 Unary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) Matrix and left hand side (lhs) Matrix, returning the result in the left hand side (lhs) Matrix:

- $+=$: unary addition
- $-=$: unary subtraction
- $*=$: unary multiplication
- $/=$: unary division

Example

```
// Make 2 Matrices with 4 rows and 3 columns
Matrix< double > A( 4, 3 ), B( 4, 3 );

// Fill Matrices A and B with numbers

for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
      B( i, j ) = 3 * i + j + 1;
   }
}

// Add matrix B to A element by element
A += B;
```

9.2.10 Unary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a right hand side (rhs) scalar and left hand side (lhs) Matrix, returning the result in the left hand side (lhs) Matrix:

- $+=$: unary scalar addition
- $-=$: unary scalar subtraction
- $*=$: unary scalar multiplication
- $/=$: unary scalar division

Example

```
// Make a Matrix with 4 rows and 3 columns, filled with 1.0
Matrix< double > A( 4, 3, 1.0 );

// Add 3.0 to each element of A
A += 3.0;
```

9.2.11 Binary mathematical operators

The following operators perform element by element addition, subtraction, multiplication and division on the right hand side (rhs) Matrices, returning a new Matrix object:

- $+$: binary addition
- $-$: binary subtraction
- $*$: binary multiplication
- $/$: binary division

Example

```
// Make 2 Matrices with 4 rows and 3 columns, and a 3rd empty Matrix
Matrix< double > A( 4, 3 ), B( 4, 3 ), C;

// Fill Matrices A and B with numbers
for ( i = 0; i < 4; i++ )
{
   for ( j = 0; j < 3; j++ )
   {
      A( i, j ) = 3 * i + j;
      B( i, j ) = 3 * i + j + 1;
   }
}

// Add matrix A and B element by element, the result is matrix C
C = A + B;
```

Notes

The Matrices *must have* the same number of rows and columns. As a new matrix object must be constructed, unary operations are preferred for efficiency.

9.2.12 Binary mathematical operators with scalars

The following operators perform scalar addition, subtraction, multiplication and division on a Matrix, returning a new Matrix object:

- $+$: binary scalar addition
- $-$: binary scalar subtraction
- $*$: binary scalar multiplication
- $/$: binary scalar division

Example

```
// Make some Matrices
Matrix< double > A( 4, 3, 1.0 ), B;

// Add 3.0 to each element of A, B is the result
B = A + 3.0;
```

Notes

As a new matrix object must be constructed, unary operations are preferred for efficiency. **Known bug: the scalar must be on the right side of the Matrix for all operators.**

9.2.13 Transpose

The following operators transpose a Matrix:

- M.t() transposes matrix M, and returns a matrix object
- M.t(O) populates passed matrix O with the transpose of M

Example

```
// A is a 3 x 4 matrix
Matrix< double > B( 4, 3 ), C;

A.t( B );  // B now contains the transpose of A
C = A.t(); // C now contains the transpose of A
```

Notes

For M.t(O), the number of columns in M *must equal* the number of rows in O and the number of rows in M *must equal* the number of columns in O.

Because M.t() must construct a new matrix object, it is less efficient than M.t(O).

9.2.14 Overloaded <<

The ostream operator << is overloaded for Matrix output.

Example

```
cout << "Matrix A = " << A << endl;
```

setHeader

The setHeader method:

- M.setHeader(bool $flag$)

is provided for ostream output. If $flag$ is true, a header line containing the format:

r rows and c columns:

is output to the stream prior to the Matrix, whereas if $flag$ is false, the header line is not output, only the Matrix. *By default, this flag is set to* **true**.

Examples

```
cout << "Matrix A alone = " << A.setHeader( false ) << endl;
A.setHeader( true );
cout << "Matrix A with header = " << A << endl;
```

9.2.15 Overloaded >>

The istream operator >> is overloaded for Matrix input.

Example

```
Matrix< double > A( 3, 2 );
cout << "Enter Matrix A: ";
cin >> A;
```

9.2.16 Dot product

Given a matrix object M, and vectors $\vec{v}$ and $\vec{o}$, where $M \cdot \vec{v} = \vec{o}$, e.g.

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{v_1} \\ \vec{v_2} \\ \vdots \\ \vec{v_c} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

or other matrix objects N and O, where $M \cdot N = O$, the following dot product operators are available in the Matrix class:

- M.dotprod(vector& v) returns a new vector object which is the dot product of the matrix and the passed vector
- M.dotprod(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the matrix and the passed vector v_{in}
- M.dotprod(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the matrix and the passed vector v_{in}
- M.dotprodt(vector& v) returns a new vector object which is the dot product of the *transpose* of the matrix and the passed vector
- M.dotprodt(vector& v_{in}, vector& v_{out}) which populates the output vector v_{out} with the dot product of the *transpose* of the matrix and the passed vector v_{in}
- M.dotprodt(vector& v_{in}, vector& v_{out}, unsigned *begin*, unsigned *end*) which populates the output vector range $v_{out}[begin]$ to $v_{out}[end]$ with the dot product of the *transpose* of the matrix and the passed vector v_{in}
- M.dotprod(N) returns a new Matrix object which is the dot product of the matrix M and the passed matrix N
- M.dotprod(N, O) which populates the passed matrix O with the dot product of the matrix M and the passed matrix N

Examples

(dotprodt(...) follows the same form as dotprod(...).)

```
// Matrix A has 4 rows and 3 columns, vector v1 has 3 elements
vector< double > v2;
v2 = A.dotprod( v1 ); // v1 is unchanged
                      // v2 now contains the dot product

// vector v3 has 4 elements
```

```
A.dotprod( v1, v3 ); // v1 is unchanged
                     // v3 now contains the dot product

// vector v4 has 5 elements
A.dotprod( v1, v4, 0, 3 ); // v4[0] through v4[3] now
                           // contains the dot product

// Matrix B has 3 rows and 5 columns
Matrix< double >C ( 4, 5 ), D;
A.dotprod( B, C );  // C now contains the dot product
D = A.dotprod( B ); // D now contains the dot product
```

Notes

For M.dotprod(v), the number of columns in M *must equal* the number of elements in v. For M.dotprod(v_{in}, v_{out}), the number of columns in M *must equal* the number of elements in v_{in}, and the number of rows in M *must be equal to or greater than* the number of elements in v_{out}. If the number of rows in M is greater than the number of elements in v_{out}, the resulting vector will be truncated to the size of v_{out}, and the remainder ignored.

For M.dotprodt(v), the number of rows in M *must equal* the number of elements in v. For M.dotprodt(v_{in}, v_{out}), the number of rows in M *must equal* the number of elements in v_{in}, and the number of columns in M *must be equal to or greater than* the number of elements in v_{out}. If the number of columns in M is greater than the number of elements in v_{out}, the resulting vector will be truncated to the size of v_{out}, and the remainder ignored.

For M.dotprod(N), the number of columns in M *must equal* the number of rows in N. For M.dotprod(N, O), the number of rows in M *must equal* the number of rows in O, the number of columns in N *must equal* the number of columns in O, and the number of columns in M *must equal* the number of rows in N.

Because M.dotprod(v) and M.dotprodt(v) must construct new vector objects, and M.dotprod(N) must construct a new matrix object, these methods are less efficient than M. dotprod(v_{in}, v_{out}), M.dotprodt(v_{in}, v_{out}) and M.dotprod(N, O).

dotprod_row

The *dotprod_row* method takes a single row from a Matrix, and transposes it to be used as the right hand side argument for a dot product. It is useful for extracting a single row from a dataset and passing it to the first layer of a neural network. It is functionally equivalent to $M \cdot \vec{d}^t = \vec{o}$, where M is the matrix, $\vec{d}^t$ is the transpose of the vector $\vec{d}$ derived from a single row of the dataset matrix, and $\vec{o}$ is the output vector.

Suppose that d is a single row of the dataset matrix D, e.g.:

$$\begin{bmatrix} \cdots & \cdots & \cdots & \cdots \\ d_{r1} & d_{r2} & \cdots & d_{rc} \\ \vdots & \vdots & \vdots & \vdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

Then M.dotprod_row(D, r, o) is equivalent to:

$$\begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1c} \\ M_{21} & M_{22} & \cdots & M_{2c} \\ \vdots & \vdots & \vdots & \vdots \\ M_{r1} & M_{r2} & \cdots & M_{rc} \end{bmatrix} \cdot \begin{bmatrix} \vec{d_{r1}} \\ \vec{d_{r2}} \\ \vdots \\ \vec{d_{rc}} \end{bmatrix} = \begin{bmatrix} \vec{o_1} \\ \vec{o_2} \\ \vdots \\ \vec{o_r} \end{bmatrix}$$

where r is the row number. The following dotprod_row operators are available in the Matrix class:

- M.dotprod_row(Matrix& D, unsigned r, vector& v) which populates the passed vector v with the dot product of Matrix M and the transposed row r in matrix D
- M.dotprod_row(Matrix& D, unsigned r) which returns a new Matrix object with the dot product of Matrix M and the transposed row r in matrix D
- M.dotprod_row(Matrix& D, unsigned r, vector& v, unsigned *begin*, unsigned *end*) which populates the range in the passed vector v[*begin*] to v[*end*] with the dot product of Matrix M and the transposed row r in matrix D

Examples

```
// Matrix A and D both have 3 columns, vector v1 has 3 elements
A.dotprod_row( D, 3, v1 ); // v1 now contains the dot product
                           // of A and the transpose of D's 3rd row

vector< double > v2;
v2 = A.dotprod_row( D, 3 ); // v2 now contains the dot product
                            // of A and the transpose of D's 3rd row

// Matrix A has 4 rows, vector v3 has 5 elements
A.dotprod_row( D, 3, v3, 0, 3 ); // v3[0] through v3[3] now
                                 // contains the dot product of A
                                 // and the transpose of D's 3rd row
```

Notes

Both matrices *must have* the same number of columns. If a vector is passed, it *must have* the same number of elements as the matrices have columns. Because M.dotprod_row(D, r) must construct a new vector object, it is less efficient than M.dotprod_row(D, r, v).

9.2.17 Outer product

The *outprod* method is available in the Matrix class to populate a Matrix with the outer product of 2 vectors. For example, given 2 vectors $\vec{p}$ and $\vec{q}$, e.g.:

$$\vec{q} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_m \end{bmatrix}, \vec{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}$$

The outer product $\vec{q}\vec{p}^t$ is:

$$\vec{q}\vec{p}^t = \begin{bmatrix} q_1p_1 & q_1p_2 & \cdots & q_1p_n \\ q_2p_1 & q_2p_2 & \cdots & q_2p_n \\ \vdots & \vdots & \vdots & \vdots \\ q_mp_1 & q_mp_2 & \cdots & q_mp_n \end{bmatrix}$$

The outer product method is called as in:
M.outprod(vector& q, vector& p)

Example

```
// Make some vectors
double array1[ 4 ] = { 0, 1, 2, 3 },
   array2[ 5 ] = { 0, 1, 2, 3, 4 };
vector< double > vec1( array1, array1 + 4 ),
   vec2( array2, array2 + 5 );

// Make the right size Matrix
Matrix< double > U( 4, 5 );

// Populate the Matrix with the outer product of the vectors
U.outprod( vec1, vec2 );
```

Notes

The number of elements in the first vector argument *must equal* the number of Matrix rows, and the number of elements in the second vector argument *must equal* the number of Matrix columns.

9.2.18 Function passing

The following methods allow functions to be applied to a Matrix element by element:

- func(Matrix& M_{in}, function f, Matrix& M_{out}): applies function f element by element to Matrix M_{in}, populating Matrix M_{out} with the result.
- func(Matrix& M, function f): applies function f element by element to Matrix M, and returns a new Matrix object.

Example

```
\index{Transfer function!Sigmoidal}

#include <vector>
#include <math.h>

using namespace std;

#include "vector_ops.h"

// Define sigmoidal function as a class which extends
// unary_function.  This is necessary for algorithm::transform.
class sigmoidal : public unary_function< double, double >
{
   public:
      inline double operator()( const double& x )
      {
         return 1.0 / ( 1.0 + exp( -x ) );
      }
};

int main() {
   // Make some Matrices
   Matrix < double > M1( 4, 3, 0.5 ), M2( 4, 3, 0.0 ),
      M3( 4, 3, 2.0 ), M4;

   // Apply sigmoidal function to each element of M1
   // Populate M2 with the result
   func( M1, sigmoidal(), M2 );

   // Apply sigmoidal function to a Matrix, with itself as result
   func( M3, sigmoidal(), M3 );

   // Apply sigmoidal function to each element of M1
   // After this function call, M4 will contain the result
```

```
   M4 = func( M1, sigmoidal() );

   return 0;
}
```

Notes

The passed function *must* be defined as a class extension of unary_function as demonstrated in the example. Common functions useful for neural computation (e.g. sigmoidal) are packaged with neURon2++.

If a Matrix to be populated is passed, it *must have* the same number of rows and columns as the input Matrix. As func(Matrix& M, function f) must construct a new vector, it is less efficient than func(Matrix& M_{in}, function f, Matrix& M_{out}).

9.2.19 Randomization

The method M.random(double n) populates a double precision Matrix M with double precision random numbers between $-n$ and $+n$.

Example

```
Matrix< double > Z( 6, 5 ); // Make a 6 x 5 Matrix
// Populate it with random numbers between -0.5 and +0.5,
//    and print it out
cout << "The random matrix is " << Z.random( 0.5 );
```

Notes

This method is only coded for Matrices of type double, e.g. Matrix< double >.

9.2.20 Loading a Matrix from a file

The following methods will load a matrix from a file:

- M.loadfile(string& $filename$, unsigned c): loads Matrix M from a file named "$filename$" with c columns. The Matrix will be resized if necessary, and if the file does not exist, the user will be queried until a good filename is entered.

- M.loadfile(bool *report*, string& $filename$): loads Matrix M from a file named "$filename$". Delimeters are any whitespace character, or any character that is non-numeric (+, -, e and E are considered numeric, as is a decimal point.) Rows are delimited by carriage returns in the file, and the number of columns is the maximum number of elements in any one row. Elements in unpopulated rows will be filled with 0, beginning from the *right* hand side of the row. If the bool flag *report* is true, a report will be printed indicating which rows were found to be unpopulated. The

Matrix will be resized if necessary, and if the file does not exist, the user will be queried until a good filename is entered.

Examples

```
Matrix< double > A, B;
string filename;
unsigned cols;

cout << "What is the name of the data file? ";
cin >> filename;
cout << "How many columns does it have? ";
cin >> cols;

A.loadfile( filename, cols );

cout << "What is the name of another data file? ";
cin >> filename;

B.loadfile( true, filename ); // a report will be printed
```

9.2.21 Saving a Matrix to a file

The following method will save a matrix to a file:

- bool M.savefile(string& $filename$): saves Matrix M to a file named "$filename$", and returns a Boolean flag indicating whether the save operation was successful.

Example

```
Matrix< double > A( 5, 4, 3.14159 ); // fill a 5 x 4 Matrix with pi
A.savefile( "pi.txt" ); // and save it to a file named pi.txt
```

9.2.22 Resizing a Matrix

The method M.resize(unsigned r, unsigned c) resizes Matrix M to r rows and c columns.

Example

```
A.resize( 5, 4 ); // resize Matrix A to 5 rows and 4 columns
```

Notes

The following Matrix so resized is filled with **garbage**.

9.2.23 Clearing a Matrix

The method M.clear() clears Matrix M, setting its dimensions and data to zero.

Example

```
A.clear(); // clear Matrix A
```

9.2.24 Populating a Matrix with a value

The method M.fill(v) populates Matrix M with value v of its type.

Example

```
Matrix< double > A( 3, 2 ); // Make a Matrix of type double with
                            // 3 rows and 2 columns

A.fill( 3.14159 ); // and fill it with pi
```

9.2.25 Calculating the sums of columns of a Matrix

The method M.colsums() returns a new vector with each element the sum of the respective column in M. The method M.colsums(vector& v) returns the result in the passed vector v.

Example

```
#include <iostream>
#include <string>
#include "matrix.h"
#include "vector_ops.h"

string filename;
cout << "The file name is ";
cin >> filename;
Matrix< double > M( true, filename ); // load Matrix from file
vector< double > v;
v = M.colsums(); // v will contain the sum of the columns in M
cout << "Sum of columns = " << v << endl;
```

Note

If the vector to be returned is passed as an argument, its size *must* be the same as the number of columns in the Matrix.

9.2.26 Calculating the sum of squares of a Matrix elements

The method M.squared() returns a the value of the sum of squares of Matrix elements.

Example

```
Matrix< double > M; // M is then populated with double values...
double sum;
sum = M.squared();
```

9.2.27 Calculating the maximum absolute value of Matrix elements

The method M.maxabs() returns the maximum absolute value of Matrix elements.

Example

```
Matrix< double > M; // M is then populated with double values...
double max;
max = M.maxabs();
```

9.2.28 Finding the row elements with value 1 in a Matrix

The method (bool) M.rowindex(vector< unsigned >& v) populates a passed (unsigned) vector v with each vector element representing in which Matrix column each of the Matrix row elements was 1, takes passed vector as argument, returns true if successful.

Example

```
vector< unsigned > v( M.rows() );
if ( M.rowindex( v ) )
   cout << v << endl;
else
   cout << "can't do it: goofy Matrix." << endl;
```

Note

The passed Matrix must consist of rows that have a single row element with a value of 1, and the remainder with values of 0. For example, the Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

would return true and populate the passed vector with

$$\begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}$$

whereas Matrices

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

would all return false.
The size of the vector *must* be the same as the number of rows in the Matrix.

9.2.29 Converting a Matrix to a vector, and a vector to a Matrix

A Matrix may be coverted row by row to a vector, e.g. Matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

would be converted to vector

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \end{bmatrix}$$

A vector may also be converted to a Matrix. Methods to perform the conversion include:

- M.toVector(vector& v): converts Matrix M to passed vector v which is populated with the result, and returns a reference to vector v
- M.toVector(): converts Matrix M to a new vector, and returns a reference to the new vector
- toMatrix(Matrix& M, vector& v): converts vector v to passed Matrix M which is populated with the result, and returns a reference to Matrix M
- toMatrix(vector& v, unsigned r, unsigned c): converts vector v to a new Matrix M with r rows and c columns, and returns a reference to the new Matrix

Example

```
// Make a Matrix as an example, and print it out
Matrix< double > A( 4, 3 );
for ( unsigned i = 0; i < 4; i++ )
    for ( unsigned j = 0; j < 3; j++ )
        A( i, j ) = 3 * i + j + 1;
cout << "Matrix A: " << A << endl;

// Convert the Matrix to vectors
vector< double > v1( 12 ), v2;
A.toVector( v1 );
cout << "toVector passed existing vector: " << v1 << endl;
v2 = A.toVector();
cout << "toVector passed new vector: " << v2 << endl;

// Convert the vectors back to Matrices
Matrix< double > B( 4, 3 ), C;
toMatrix( B, v2 );
cout << "back to an existing Matrix: " << B << endl;
C = toMatrix( v2, 4, 3 );
cout << "back to a new Matrix: " << C << endl;
```

Notes

If a vector is passed as an argument to toVector, its size *must be* the same as the product of the rows and the columns of the Matrix. If a Matrix is passed as an argument to toMatrix, the product of its rows and columns *must be* the same as the size of the vector which is also passed. If a Matrix is not passed as an argument to toMatrix, the product of the 2^{nd} and 3^{rd} arguments *must be* the same as the size of the passed vector.

9.2.30 First order covariance of a Matrix

The method M.covariance() returns a new Matrix with the first-order covariance Matrix of M. The method M.covariance(Matrix< double >& N) populates the passed Matrix N with the result, and returns a reference to Matrix N.

Example

```
#include <iostream>
#include <string>
#include "matrix.h"

string filename;
cout << "The file name is ";
cin >> filename;
```

```
Matrix< double > Data( true, filename ), // load data Matrix from file
   C; // for the covariance Matrix

C = Data.covariance();
cout << "Covariance Matrix = " << C << endl;
```

Notes

The covariance method is only coded for Matrices of type < double >. If the Matrix to be populated with the result is passed as an argument, it's dimensions must be $Col \times Col$, where Col is the number of columns in the data Matrix.

9.2.31 Matrix inverse and determinant

The following methods calculate the inverse and determinant of a Matrix:[1]

- M.inverse(Matrix< double >& I): calculates the inverse of Matrix M by LU decomposition, populating Matrix I with the result, and returns a reference to Matrix I
- M.inverse(): calculates the inverse of Matrix M by LU decomposition, and returns a new Matrix with the result
- M.inverse(Matrix< double >& I, double& d): calculates the inverse of Matrix M by LU decomposition, populating Matrix I with the result, returns the determinant in the passed argument d, and returns a reference to Matrix I
- M.inverse(double& d): calculates the inverse of Matrix M by LU decomposition, returns the determinant in the passed argument d, and returns a new Matrix with the result
- M.determinant(): returns the determinant of Matrix M by LU decomposition
- M.inverse(Matrix< double >& I, unsigned *choice*): allows the user to force type of inverse calculation by setting *choice* to "GaussJordan," (default is LU decomposition) populating Matrix I with the result, and returns a reference to Matrix I
- M.inverse(unsigned *choice*): allows the user to force type of inverse calculation by setting *choice* to "GaussJordan" (default is LU decomposition), and returns a reference to the new Matrix

In the case of a singular Matrix, these methods throw exception Matrix< double >::Singular& e, where e.what() is "singular Matrix".

[1] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pages 36-48

Example

```
#include <iostream>
#include <string>
#include "matrix.h"

string inverse_filename;
cout << "The file name for Matrix inverse is ";
cin >> inverse_filename;
Matrix< double > I( true, inverse_filename ), J, K;
double determinant;

try
{
   // Test populating a passed Matrix
   J.resize( I.rows(), I.cols() ); // resize receiving Matrix to input
   Matrix
   cout << "The inverse of " << I << "is " << I.inverse( J ) << endl;

   // Test returning a new Matrix
   K = I.inverse();
   cout << "The inverse of " << I << "is " << K << endl;

   // Test populating a passed Matrix, and returning a determinant
   cout << "The inverse of " << I << "is " << I.inverse( J,
   determinant )
      << endl;
   // Store the inverse and determinant separately for clarity:
   cout << "The determinant is " << determinant << endl;

   // Test returning a new Matrix with a determinant
   K.clear();
   K = I.inverse( determinant );
   cout << "The inverse of " << I << "is " << K
      << endl << "The determinant is " << determinant << endl;

   // Test returning just the determinant
   cout << "That determinant again is " << I.determinant() << endl;

   // Test populating a passed Matrix, and forcing type of inverse
   procedure
   cout << "The Gauss Jordan inverse of " << I << "is "
      << I.inverse( J, GaussJordan ) << endl;

   // Test returning a new Matrix and forcing type of inverse
   procedure
```

```
   K.clear();
   K = I.inverse( GaussJordan );
   cout << "The Gauss Jordan inverse of " << I << "is " << K << endl;

   // Examine the difference between LU decomposition and Gauss Jordan
   cout << "The difference between LU decomposition and Gauss Jordan
   inverse of "
      << I << "is " << I.inverse() - I.inverse( GaussJordan ) << endl;
}
catch ( Matrix< double >::Singular& e )
{
   cout << "Can't do it: " << e.what() << endl;
}
```

Notes

Use a try block with these methods to catch the Singular exception. If you don't, your program may bomb out if a singular Matrix is encountered.

These methods are only coded for Matrices of type < double >. Matrices to invert must be square, e.g. dimensions must be $Col \times Col$, where Col is the number of columns in the Matrix. If the Matrix to be populated with the result is passed as an argument, it's dimensions must be the same as the Matrix object.

9.2.32 Eigenvalues

The calculation of the eigenvalues of a symmetric matrix is the only current instance of using pre-existing code and libraries, as these calculations can be quite intensive, and are currently beyond the scope of the current program. Gaurav Bansal ported the eigenvalue routines for a symmetric matrix from the Gnu Scientific Library (GSL) in 2003, recoding those portions of GSL to integrate with the matrix specification in neUROn2++. **As a result, do not expect GSL functionality beyond eigenvalue routines in the neUROn2++ environment.**

The best way to understand calling eigenvalue routines in neUROn2++ is to study the given example. A GSL matrix is first created from a neUROn2++ matrix using *gsl_matrix_view*, as is a GSL vector using *gsl_vector*. A GSL workspace is then created using *gsl_eigen_symm_workspace* for symmetric matrix eigenvalue calculations, and the eigenvalues are calculated using *gsl_eigen_symm*. Finally, the GSL vector is returned to an STL vector containing the eigenvalues using *gsl_vector_ptr*, and memory is freed using *gsl_eigen_symm_free*.

For those interested in GSL, it may be found at http://sources.redhat.com/gsl/ as of the time of this writing (August 11, 2003).

Example

```
#include "gsl_eigen.h"

// Create a symmetric matrix using our matrix class, and fill with
random values
Matrix< double > m1( 4, 4 );
m1.random( 5 );

// Check to see that we perform eigenvalue routines to symmetric
routines only
assert ( m1.rows() == m1.cols() );

// Create a gsl matrix from our matrix--begin function is cautioned to
use but
//    here we need the value as double*
gsl_matrix_view m = gsl_matrix_view_array( m1.begin(), 4, 4 );

// Create a gsl vector to store all the eigenvalues
gsl_vector *eval = gsl_vector_alloc( 4 );

// Create gsl workspace for eigenvalue calculations
gsl_eigen_symm_workspace *w = gsl_eigen_symm_alloc( 4 );

// Calculate eigenvalues
gsl_eigen_symm( &m.matrix, eval, w );

// Create vector< double > from gsl vector
vector< double > eigenv( gsl_vector_ptr( eval, 0 ), gsl_vector_ptr(
eval, 4 ) );

// Free workspace
gsl_eigen_symm_free( w );

// Output results
cout << "Matrix: " << m1 << endl << "Eigenvalues are: " << eigenv <<
endl;
cout << "Maximum value = " << maxabs( eigenv ) << endl;
cout << "Minimum value = " << minabs( eigenv ) << endl;
```

Notes

GSL is discovered and linked by CMake through `find_package(GSL REQUIRED)` and the imported targets `GSL::gsl` and `GSL::gslcblas`. Do not add local GSL copies, compiler search paths, or IDE-specific library settings to the source tree.

9.3 An example of the Matrix and vector classes

The following example constructs a simple neural network with 3 hidden nodes on one layer to train the exclusive-or problem. The file "xor.set" contains the exclusive-or dataset:

```
-0.9 -0.9 0.1
-0.9 0.9 0.9
0.9 -0.9 0.9
0.9 0.9 0.1
```

The following is the program:[2]

```
#include <iostream>
#include <iomanip>
#include <string>
#include <math.h>

#include "matrix.h"
#include "vector_ops.h"
#include "function_defs.h"

using namespace std;

int main()
{
   // Assumes a file named xor.set
   string filename = "xor.set";

   // Load the XOR dataset from the file xor.set
   Matrix< double > dataset( filename, 3 ); // it has 3 columns

   // Get the input Matrix In from the dataset as a submatrix
   Matrix< double > In = dataset.submatrix( 0, 3, 0, 1 );

   // Get the output vector y from the dataset's last column
   vector< double > y = dataset.col( 2 );

   // Make the 3 x 2 hidden weight Matrix hW and the hW update Matrix
   hWup
   Matrix< double > hW( 3, 2 ), hWup( 3, 2 );
   hW.random( 0.5 ); // randomize hW

   // Make the 2 element vector for the inputs I
```

[2]References are to James A. Freeman and David M. Skapura, Neural Networks, Algorithms, Applications and Programming Techniques, Addison-Wesley, 1991, ISBN 0-201-51376-5

```
// and the 3 element vector hidden output hO
// and the 3 element vector output weights oW
vector< double > I( 2 ), hO( 3 ), oW( 3 );
nvec::random( oW, 0.5 ); // randomize oW

// o is the neural net's output, o_err the output error term,
// eta is the learning rate
double o, o_err, eta = 0.05;

// h_err is the vector with hidden error terms
vector< double > h_err( 3 );

for ( unsigned iteration = 0; iteration < 1000000; iteration++ )
{
   for ( unsigned example = 0; example < 4; example++ )
   {
      // Get a single row from the input Matrix
      // (Freeman & Skapura p. 102, #1)
      In.row( example, I );

      // Take the dot product of the hidden weights Matrix hW and
      the
      // transpose of a row of the input Matrix I, and apply the
      // sigmoidal function to the resulting vector
      // (Freeman & Skapura p. 102, #2 & #3)
      func( hW.dotprod( I, hO ), sigmoidal(), hO );

      // Take the dotproduct of the hidden output vector and the
      output
      // weights vector, and apply the sigmoidal function to the
      result
      // (Freeman & Skapura p. 102, #4 & #5)
      o = sigmoidal()( dotprod( hO, oW ) );

      // Calculate the error term for the output unit
      // (Freeman & Skapura p. 102, #6)
      o_err = ( y[ example ] - o ) * d_sigmoidal()( o );

      // Calculate the error terms for the hidden units
      // (Freeman & Skapura p. 102, #7)
      // Note that using func which takes the output vector as the
      // last argument, and *= instead of *, is the most efficient
      way
      ( func( hO, d_sigmoidal(), h_err ) *= o_err ) *= oW;

      // Update the output weights, note again the use of *= for
```

```
         efficiency
         // (Freeman & Skapura p. 102, #8)
         oW += ( hO *= ( eta * o_err ) );

         // Update the hidden weights
         // (Freeman & Skapura p. 102, #9)
         hW += ( hWup.outprod( h_err, I ) *= eta );
      }

      // Every 1000 iterations, print out the error
      if ( iteration % 1000 == 0 )
         cout << setw( 8 ) << iteration << ": LMS error = " <<
         0.5 * ( y[ 3 ] - o ) * ( y[ 3 ] - o ) << endl;
   }

   return 0;
}
```

Chapter 10

Utility methods

The following are contained in the header file "utility.h", and contain global utility methods. These methods throw exception util::utilErr& e, where e.what() specifies the module from which the error originated, and the nature of the error.

10.1 Random number generator

namespace util

The following utilities seed or generate random numbers of double precision:

- d_random(): seeds the random number generator with an internal timestamp
- d_random(double n): returns a double precision random number between $-n$ and $+n$
- d_random(double *lower*, double *upper*): returns a double precision random number between *lower* and *upper*

10.1.1 Examples

```
util::d_random(); // seed the generator

double x, y;

x = util::d_random( 0.5 ); // Generates a random number between -0.5
and +0.5

y = util::d_random( 1.0, 10.0 ); // Between 1.0 and 10.0
```

10.2 Timestamp

The timestamp(unsigned x) utility inserts a timestamp of the form HHHH:MM:SS (where H is hour, M is minute, S is second) into the output stream, given x seconds.

10.2.1 Example

```
unsigned start = time( 0 ); // gets the current time in seconds

// Do something for a while
for ( i = 0; i < 100000; i++ )
   cout << ".";

// Calculate the elapsed time
unsigned increment = time( 0 ) - start;

// Print out how long it took
cout << endl << "That took " << timestamp( increment ) << endl;
```

10.3 Round

namespace util

The following method will round a number to a specified number of significant digits:

- round(double x, unsigned d): rounds the passed value x to d significant digits, returns the rounded value

10.3.1 Example

```
double x;
unsigned d;

cout << "x = ";
cin >> x;
cout << "d = ";
cin >> d;
cout << "round( x, d ) = " << util::round( x, d ) << endl;
```

10.3.2 Notes

Zero significant digits is not allowed. Rounds positive numbers up, and negative numbers down, e.g. util::round(30.9999, 2) = 31, and util::round(-30.9999, 2) = -31.

10.4 Queries with bounds checking

namespace util

The *askI and askD* methods allow user queries with bounds checking. Specifications include:

- askI(string& *query*, int *lower*) which queries the user with the string *query* repeatedly if necessary until the user enters an integer value >= *lower*, and returns the value
- askI(string& *query*, int *lower*, int *upper*) which queries the user with the string *query* repeatedly if necessary until the user enters an integer value >= *lower* and <= *upper*, and returns the value
- askD(string& *query*, double *lower*) which queries the user with the string *query* repeatedly if necessary until the user enters a value >= *lower*, and returns the value
- askD(string& *query*, double *lower*, double *upper*) which queries the user with the string *query* repeatedly if necessary until the user enters a value >= *lower* and <= *upper*, and returns the value

10.4.1 Example

```
int i, h;
double d;

i = util::askI( "How many input nodes? ", 1 );
h = util::askI( "How many hidden nodes? ", 1, 10 );
d = util::askD( "What is the learning rate? ", 0, 1 );
```

10.5 Query for yes/no answer

namespace util

The *askYesNo* method allows user queries with yes/no answers. Specifications include:

- askYesNo(string& *query*) queries the user with the string *query* repeatedly if necessary until the user enters "yes", "y", "no" or "n", and returns boolean true for yes, false for no

10.5.1 Example

```
bool yesNoAnswer;
yesNoAnswer = util::askYesNo( "Shall we clean the closet? " );
```

10.6 Query for existing file name

namespace util

The *getGoodFile* method tests for a file name to see if it actually exists, then queries for an existing file name repeatedly until a file that actually exists is entered, and returns that file name as a *string*:

- getGoodFile(string& $filename$) tests $filename$ to see if it actually exists, then if it doesn't, queries the user for a file name repeatedly if necessary until the user enters an existing file name, and returns it as a string.

10.6.1 Example

```
#include <iostream>
#include <fstream>
#include <string>
#include "utility.h"

string filename; // for the name of a loaded file

cout << "What is the name of your data file (it must exist)? ";
cin >> filename;

// Open the file as read-only, and associate it with the 'label'
inputFile
ifstream inputFile( ( util::getGoodFile( filename ) ).c_str(), ios::in
);
```

10.7 Remove a carriage return

namespace util

The *chopEndl* method removes ASCII 13, generally a carriage return, from the end of a string if it exists, and returns a reference to the string.

- chopEndl(string& *stringObj*) removes the last character from *stringObj* if it is ASCII 13, and returns a reference to *stringObj*.

10.7.1 Example

```
#include <iostream>
#include <fstream>
#include <string>
#include "utility.h"

string lineString; // string to hold the 1st line of file
```

```
// Open the file "test.txt" for input
ifstream loadfile( "test.txt", ios::in );

getline( loadfile, lineString ); // get the 1st line of file
util::chopEndl( lineString ); // remove <cr> from end of string if
exists

loadfile.close(); // close the input file
```

10.8 Parse a variable definition string

namespace util

- variable_parse(string& *stringObj*) parses *stringObj* for variable representations from nodes (see below), and returns a vector of vector of unsigned representing the nodes.
- variable_parse(ifstream& $fileObj$) parses the file $fileObj$ for variable representations from nodes (see below), and returns a vector of vector of unsigned representing the nodes.

The *variable_parse* method parses a string passed as the argument to determine variable representation from nodes, and returns a *new* vector of vector of unsigned. In the string,

- **;** delimits variables
- **,** delimits nodes
- **-** specifies a range of nodes
- spaces are ignored

For example, passed string "0-3,5; 4,8; 6,7" returns vector of vector of unsigned (each line is a vector):

```
0 1 2 3 5
4 8
6 7
```

util::utilErr exceptions thrown by this method include e.what()

- "util variable_parse error: bad character found at string position *position* (*character*)"
- "util variable_parse error: too many hyphens"
- "util variable_parse error: hyphen without a number on both ends "
- "util variable_parse error: 1st number in range greater than 2nd "

- "util variable_parse error: numbers in range are the same"
- "util variable_parse error: missing zero"
- "util variable_parse error: duplicated positions"
- "util variable_parse error: missing positions"

10.8.1 Example

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

#include "vector_ops.h"
#include "utility.h"

using namespace std;

int main()
{
   // The returned structures
   vector< vector< unsigned > > parse_numbers, parse2;

   string parse_test = "0-3,5; 4,8; 6,7"; // a test string

   // Parse the string
   try
   {
      parse_numbers = util::variable_parse( parse_test );

      // Then print out the returned structure, a vector at a time
      cout << "Here it is: " << endl;
      unsigned pCount;
      for ( pCount = 0; pCount < parse_numbers.size(); pCount++ )
         cout << parse_numbers[ pCount ] << endl;
   }
   catch ( util::utilErr& e )
   {
      cout << e.what() << endl;
   }

   // Do the same from a file
   string reply;
   cout << "What is the name of the file? ";
   cin >> reply;
```

```
   ifstream inputFile( ( util::getGoodFile( reply ) ).c_str(), ios::in
   );

   try
   {
      parse2 = util::variable_parse( inputFile );
      cout << "Here it is: " << endl;
      for ( pCount = 0; pCount < parse2.size(); pCount++ )
         cout << parse2[ pCount ] << endl;
   }
   catch ( util::utilErr& e )
   {
      cout << e.what() << endl;
   }

   return 0;
}
```

variable_parse(ifstream& *fileObj*) will close and clear the file for you. Parsing a file will support multiple line definitions of variables by concatenating the lines prior to parsing the resulting string.

10.9 Remove a variable from a vector of vector of unsigned data structure

namespace util

- remove_variable(vector< vector< unsigned > >& *variables*, unsigned *n*, vector< vector< unsigned > >& *result*) removes the subvector representing variable *n* from the vector of vector of unsigned data structure *variables*, populates *result* as a vector of vector of unsigned data structure representing the result, returns a reference to the resulting data structure.
- remove_variable(vector< vector< unsigned > >& *variables*, unsigned *n*) as above, except returns a new vector of vector of unsigned data structure with the result.

The *remove_variable* method removes a single subvector, representing a single variable, from a vector of vector of unsigned data structure which represents a set of variables. As neUROn2++ variables may contain more than 1 input node (e.g. a variable may contain 2 nodes, the value of the variable itself, and whether or not that variable was present in the dataset), this method is used to remove a single variable from a set of variables.

10.9.1 Example

```
// Create a vector of vector of unsigned with some variables
string parse_test = "0-3,5; 4,8; 6,7";
vector< vector< unsigned > > parse_numbers, parse_new, parse_new2;
parse_numbers = util::variable_parse( parse_test );

// Test remove_variable
parse_new = parse_numbers;
unsigned position;
position = util::askI( "What is the position to remove? ", 0 );
util::remove_variable( parse_numbers, position, parse_new );
cout << "After I've removed variable " << position << ": " << endl;
cout << "Here is the original: " << parse_numbers << endl;
cout << "Here is the result: " << parse_new << endl;

parse_new2 = util::remove_variable( parse_numbers, position );
cout << "Making a new data structure : " << endl;
cout << "Here is the original: " << parse_numbers << endl;
cout << "Here is the result: " << parse_new2 << endl;
```

Chapter 11

Statistical methods

The following are contained in the header file "stats.h", and implemented in "stats.cpp", and contain global statistical methods. Those that are not classes are contained within namespace "stats", and throw exception stats::statsErr& e, where e.what() specifies the module from which the error originated, and the nature of the error.

11.1 Log gamma

namespace stats

The following method returns the natural logarithm of the gamma function $\log \Gamma(a)$[1] where

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt \tag{11.1}$$

- gammln(double a): returns the double value $\log \Gamma(a)$

11.1.1 Example

```
double x;
cout << "For log gamma, x = ";
cin >> x;
cout << "Log gamma " << x << " = " << stats::gammln( x ) << endl;
```

11.2 Incomplete gamma by fraction

namespace stats

[1] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 214

The following method returns the incomplete gamma function

$$P(a,x) \equiv \frac{\gamma(a,x)}{\Gamma(a)} \equiv \frac{1}{\Gamma(a)} \int_0^x e^{-t} t^{a-1} dt \quad (a > 0) \tag{11.2}$$

by continued fraction development:[2]

$$\Gamma(a,x) = e^{-x} x^a \left(\frac{1}{x + \frac{1-a}{1 + \frac{1}{x + \frac{2-a}{1 + \frac{2}{x + \cdots}}}}} \right) \quad (x > 0) \tag{11.3}$$

- gcf(double a, double x): returns the double value $\Gamma(a,x)$

11.2.1 Example

```
double a, x;
cout << "For gcf, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gcf( " << a << " , " << x << " ) = " << stats::gcf( a, x
     );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.3 Incomplete gamma by series

namespace stats

The following method returns the incomplete gamma function (equation 11.2) by series: [3]

$$\gamma(a,x) = e^{-x} x^a \sum_{n=0}^{\infty} \frac{\Gamma(a)}{\Gamma(a+1+n)} x^n \tag{11.4}$$

- gser(double a, double x): returns the double value $\gamma(a,x)$

[2] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 219

[3] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 218–219

11.3.1 Example

```
double a, x;
cout << "For gser, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gser( " << a << " , " << x << " ) = " << stats::gser( a,
     x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.4 Incomplete gamma composite

namespace stats

The following method returns the incomplete gamma function (equation 11.2) by either continued fraction or series, based on speed of convergence:[4]

- gammp(double a, double x): returns the double value $P(a, x)$

11.4.1 Example

```
double a, x;
cout << "For gammp, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gammp( " << a << " , " << x << " ) = " << stats::gammp(
     a, x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.5 Incomplete gamma complement

namespace stats

[4] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 218

The following method returns the complement of the incomplete gamma function

$$Q(a,x) \equiv 1 - P(a,x) \tag{11.5}$$

(see equation 11.2 for the definition of $P(a,x)$): [5]

- gammq(double a, double x): returns the double value $Q(a,x)$

11.5.1 Example

```
double a, x;
cout << "For gammq, a = ";
cin >> a;
cout << "x = ";
cin >> x;
try {
     cout << "gammq( " << a << " , " << x << " ) = " << stats::gammq(
     a, x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.6 Beta function

namespace stats

The following method returns the beta function $B(z,w)$ [6] where

$$B(z,w) = B(w,z) = \int_0^1 t^{z-1}(1-t)^{w-1}dt = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \tag{11.6}$$

- beta(double z, double w): returns the double value $B(z,w)$

11.6.1 Example

```
double z, w;
cout << "For beta function, z = ";
cin >> z;
cout << "w = ";
cin >> w;
cout << "Beta = " << stats::beta( z, w ) << endl;
```

[5] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, p. 218

[6] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 215–216

11.7 Incomplete Beta function

namespace stats

The following method returns the incomplete beta function $I_x(a,b)$ [7] where

$$I_x(a,b) \equiv \frac{B_x(a,b)}{B(a,b)} \equiv \frac{1}{B(a,b)} \int_0^x t^{a-1}(1-t)^{b-1} dt \tag{11.7}$$

Note that the continued fraction representation is also available, but as a utility function should not be necessary to call directly:

$$I_x(a,b) = \frac{x^a(1-x)^b}{aB(a,b)} = \left(\frac{1}{1 + \frac{d_1}{1+\frac{d_2}{1+\cdots}}} \right) \tag{11.8}$$

where

$$d_{2m+1} = -\frac{(a+m)(a+b+m)x}{(a+2m)(a+2m+1)} \tag{11.9}$$

and

$$d_{2m} = \frac{m(b-m)x}{(a+2m-1)(a+2m)} \tag{11.10}$$

- betai(double a, double b, double x): returns the double value $I_x(a,b)$
- betacf(double a, double b, double x): returns the double value $I_x(a,b)$ by continued fraction evaluation, used by betai

11.7.1 Example

```
double a, b, x;
cout << "For incomplete beta function, a = ";
cin >> a;
cout << "b = ";
cin >> b;
cout << "x = ";
cin >> x;
cout << "Incomplete beta = " << stats::betai( a, b, x ) << endl;
```

[7] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 226–228

11.8 Error functions

namespace stats

The following methods returns the error function (commonly referred to as *erf*) [8] and its complement

$$erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt = P(\frac{1}{2}, x^2) \tag{11.11}$$

$$erfc(x) \equiv 1 - erf(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt = Q(\frac{1}{2}, x^2) \tag{11.12}$$

- erff(double x): returns the double value $erf(x)$
- erfc(double x): returns the double value $erfc(x)$
- erffc(double x): returns the double value $erfc(x)$ by Chebyshev fitting

11.8.1 Example

```
double x;
cout << "x = ";
cin >> x;
try {
     cout << "erf( " << x << " ) = " << stats::erff( x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
try {
     cout << "erfc( " << x << " ) = " << stats::erfc( x );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.8.2 Notes

Note the 2 f s in erff, so that this is not confused with the innate compiler function erf.

11.9 Error functions inverses

namespace stats

[8] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 220–221

The following methods return the inverse of the error function and the inverse of the complement of the error function:[9]

- invErff(double y): returns the double value $erf^{-1}(y)$
- invErfc(double y): returns the double value $erfc^{-1}(y)$

11.9.1 Example

```
double y;
cout << "y = ";
cin >> y;
try {
     cout << "inverse erf( " << y << " ) = " << stats::invErff( y );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
try {
     cout << "inverse erfc( " << y << " ) = " << stats::invErfc( y );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.10 Gaussian distribution

namespace stats

The following method returns the unit Gaussian function

$$\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2} \tag{11.13}$$

where given mean μ and standard deviation σ the following transformation $x \to z$ is made:

$$z \equiv \frac{x - \mu}{\sigma}$$

x, μ and σ may also be passed, where the Gaussian function is then

$$\varphi(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)} \tag{11.14}$$

- gaussD(double z): returns the double value $\varphi(z)$
- gaussD(double x, double μ, double σ): returns the double value $\varphi(x, \mu, \sigma)$

[9] Algorithm for $erfc^{-1}(y)$ by Takuya Ooura © 1996, $erf^{-1}(y) = erfc^{-1}(1 - y)$

11.10.1 Example

```
double z, x, u, s;
cout << "for gaussD, z = ";
cin >> z;
cout << "gaussD = " << stats::gaussD( z ) << endl;
cout << "for gaussD, x = ";
cin >> x;
cout << "u = ";
cin >> u;
cout << "s = ";
cin >> s;
cout << "gaussD = " << stats::gaussD( x, u, s ) << endl;
```

11.11 z Area

namespace stats

The following method returns the area under a unit Gaussian distribution

$$\Phi(z) \equiv \frac{1}{\sqrt{2\pi}} \int_0^z e^{-z^2/2} dz = \frac{1}{2} erf(\frac{z}{\sqrt{2}}) \tag{11.15}$$

- Zarea(double z): returns the double value $\Phi(z)$

11.11.1 Example

```
double z;
cout << "for z Area, z = ";
cin >> z;
try {
     cout << "z Area = " << stats::Zarea( z );
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
cout << endl;
```

11.12 z Area inverse

namespace stats

The following method returns the inverse of the area under a unit Gaussian distribution

$$z = Z(A) = \Phi^{-1}(A) = \sqrt{2} erf^{-1}(2A - 1) \tag{11.16}$$

- invZarea(double A): returns the double value $Z(A) = \Phi^{-1}(A)$

11.12.1 Example

```
double A;
cout << "for invZarea, A = ";
cin >> A;
try {
     cout << "invZarea ( " << A << " ) = " << stats::invZarea( A ) <<
     endl;
} catch ( stats::statsErr& e ) {
     cout << e.what();
}
```

11.13 Population metrics

The *Population* class provides statistical metrics such as mean, variance, standard deviation, skew and kurtosis. It's constructor takes vector< double > as its argument, representing the population from which to generate metrics. Errors are caught by *PopulationErr* (see below for examples).

- Population(vector< double > x) defines a population from vector< double > x from which to generate statistical metrics

Accessors include:

- P.mean(): returns the mean
- P.var(): returns the variance
- P.std(): returns the standard deviation
- P.skew(): returns the skew
- P.kurtosis(): returns the kurtosis

11.13.1 Example

```
double numbers[ 4 ] = { 1.0, 2.0, 3.0, 4.0 }; // define the dataset
vector< double > dset( numbers, numbers + 4 );
Population P( dset ); // define the population from the dataset
cout << "average = " << P.mean() << endl;
cout << "variance = " << P.var() << endl;
cout << "standard deviation = " << P.std() << endl;
try {
   cout << "skew = " << P.skew() << endl;
   cout << "kurtosis = " << P.kurtosis() << endl;
} catch ( Population::PopulationErr& e ) {
   cout << e.what() << endl;
}
```

11.13.2 Notes

The only allowable constructor takes vector< double > as an argument, and there is no copy constructor or overloaded = operator. The vector< double > argument *must* contain 2 or more elements.

11.14 F and Student's t

namespace stats

The following methods return probabilities for the F-test that 2 sample variances are different (p=1.0 if they are equal), and Student's t- tests that 2 samples are different. All take 2 vectors of double as the samples S_1 and S_2.

- ftest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for an F-test.
- ttest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a Student's t- test with equal sample variance.
- tutest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a Student's t- test with unequal sample variance.
- tptest(vector< double > S_1, vector< double > S_2): returns a double value with the probability for a paired Student's t-test.

Note that[10]

- For the F-test,
$$\alpha = I_{\frac{\nu_2}{\nu_2+\nu_1 F}}(\frac{\nu_2}{2}, \frac{\nu_1}{2}) \tag{11.17}$$
(see equation 11.7), where $F = \frac{\sigma_1^2}{\sigma_2^2}$ ($\sigma_1^2 > \sigma_2^2$), and $\nu_1 = N_1 - 1$ and $\nu_2 = N_2 - 1$
- For Student's distribution, $\alpha = 1 - A(t|\nu)$, where
$$A(t|\nu) = \frac{1}{\sqrt{\nu} B(\frac{1}{2}, \frac{\nu}{2})} \int_{-t}^{t} (1 + \frac{x^2}{\nu})^{\frac{\nu+1}{2}} dx \tag{11.18}$$
(see equation 11.6) and is related to the incomplete beta function by
$$A(t|\nu) = 1 - I_{\frac{\nu}{\nu+t^2}}(\frac{\nu}{2}, \frac{1}{2}) \tag{11.19}$$
(see equation 11.7)

[10] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 228–229, 615–619

– For Student's t with equal sample variance,

$$t = \frac{\mu_1 - \mu_2}{s_D} \tag{11.20}$$

where

$$s_D = \sqrt{\frac{\sum_i^{N_1}(x_i - \mu_1)^2 + \sum_i^{N_2}(x_i - \mu_2)^2}{N_1 + N_2 - 2}\left(\frac{1}{N_1} + \frac{1}{N_2}\right)} \tag{11.21}$$

and $\nu = N_1 + N_2 - 2$

– For Student's t with unequal sample variance,

$$t = \frac{\mu_1 - \mu_2}{\sqrt{\frac{\sigma_1^2}{N_1} + \frac{\sigma_2^2}{N_2}}} \tag{11.22}$$

and

$$\nu = \frac{\left(\frac{\sigma_1^2}{N_1} + \frac{\sigma_2^2}{N_2}\right)^2}{\frac{\left(\frac{\sigma_1^2}{N_1}\right)^2}{N_1 - 1} + \frac{\left(\frac{\sigma_2^2}{N_2}\right)^2}{N_2 - 1}} \tag{11.23}$$

– For paired Student's t,

$$t = \frac{\mu_1 - \mu_2}{s_D} \tag{11.24}$$

where

$$s_D = \sqrt{\frac{\sigma_1^2 + \sigma_2^2 - 2Cov(x1, x2)}{N}} \tag{11.25}$$

where

$$Cov(x1, x2) \equiv \frac{1}{N-1}\sum_{i=1}^{N}(x_{i1} - \mu_1)(x_{i2} - \mu_2) \tag{11.26}$$

and $\nu = N - 1$, where N is the *number of pairs.*

11.14.1 Example

```
double d1[ 9 ] = { 3, 4, 5, 8, 9, 1, 2, 4, 5 };
double d2[ 9 ] = { 6, 19, 3, 2, 14, 4, 5, 17, 1 };
vector< double > v1( d1, d1 + 9 ), v2( d2, d2 + 9 );

cout << "F-test = " << stats::ftest( v1, v2 ) << endl;
cout << "Student's t for same variance = " << stats::ttest( v1, v2 ) <
< endl;
cout << "Student's t for unequal variance = " << stats::tutest( v1, v2
) << endl;
cout << "Paired Student's t = " << stats::tptest( v1, v2 ) << endl;
```

11.14.2 Notes

Note that in all cases, sample (passed vector) sizes *must be* > 1, and for the paired Student's t-test, they *must be* equal.

11.15 Chi-square

namespace stats

The following method returns the probability p for df degrees of freedom and Chi-square value χ^2:

- pX2(unsigned df, double χ^2): returns a double value with the probability.

Note that the probability p for the Chi-square value χ^2 with df degrees of freedom is $Q(\frac{df}{2}, \frac{\chi^2}{2})$ (see equation 11.5).

11.15.1 Example

```
double x; // chi-square value
unsigned df; // degrees of freedom

// Query the user for the value and degrees of freedom
cout << "For chi-square, df = ";
cin >> df;
cout << "X2 = ";
cin >> x;

// Then calculate the probability p
try {
     cout << "p = " << stats::pX2( df, x );
} catch ( stats::statsErr& e ) { // catch a stats error
     cout << e.what();
}
cout << endl;
```

11.16 Kolmogorov-Smirnov statistic

namespace stats

The following method returns the Kolmogorov-Smirnov probability Q_{KS}[11]

$$Q_{KS}(\lambda) = 2\sum_{j=1}^{\infty}(-)^{j-1}e^{-2j^2\lambda^2} \tag{11.27}$$

- double probks(double λ): returns a double value with $Q_{KS}(\lambda)$.

[11] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pages 623-626

11.16.1 Example

```
double alam;
cout << "For Kolmogorov-Smirnov, lambda = ";
cin >> alam;
cout << "Q_KS " << alam << " = " << stats::probks( alam ) << endl;
```

11.17 Functions

The *Funct* class provides functional manipulation such as root finding, etc. The *Funct* constructor takes as its arguments an instance of the function (generally **this*) followed by a reference to the *member function of a class*, for example:

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   Funct< TestIt > squarer(){ return Funct< TestIt >( *this, &
   TestIt::xsqrd ); }

private:
   double xsqrd( double x ){ return x * x - 4; }
};
```

Note that *Funct* is a template class, and takes as its template argument the name of the class containing the member function. Errors are caught by *functErr* (see below for examples).

11.17.1 Bracketing

The following methods bracket a function:[12]

- F.mnbrak(double $x1$, double $x2$) takes boundary points $x1$ and $x2$ to bracket function F. Accessors below return the bracketed result, where ax, bx and cx are 3 sequential x-axis points, with $fb = f(bx)$ less than $fa = f(ax)$ and $fc = f(cx)$. All returned values are of type double.
- F.mnbrak_ax(): accessor returns ax
- F.mnbrak_bx(): accessor returns bx
- F.mnbrak_cx(): accessor returns cx
- F.mnbrak_fa(): accessor returns fa
- F.mnbrak_fb(): accessor returns fb
- F.mnbrak_fc(): accessor returns fc

[12] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 400–401

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void mnbrak( double x1, double x2 )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         f.mnbrak( x1, x2 );
         cout << "ax = " << f.mnbrak_ax() << endl;
         cout << "bx = " << f.mnbrak_bx() << endl;
         cout << "cx = " << f.mnbrak_cx() << endl;
         cout << "fa = " << f.mnbrak_fa() << endl;
         cout << "fb = " << f.mnbrak_fb() << endl;
         cout << "fc = " << f.mnbrak_fc() << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double x1, x2;
   cout << "For testing mnbrak, x1 = ";
   cin >> x1;
   cout << "x2 = ";
   cin >> x2;
   tester.mnbrak( x1, x2 );

   return 0;
}
```

11.17.2 Find Minimum

The following methods isolate the minimum of function F using Brent's method:[13]

[13] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 402–405

- F.brent(double ax, double bx, double cx, double tol): takes initial bracketing triplet of abscissas ax, bx and cx and tolerance tol to isolate minimum of function F.
- F.brent_xmin(): returns the isolated minimum (type double) of function F.

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void brent( double a1, double b1, double c1, double tol )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         cout << "brent = " << f.brent( a1, b1, c1, tol ) << endl;
         cout << "xmin = " << f.brent_xmin() << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double a, b, c, tol;
   cout << "For testing brent, a = ";
   cin >> a;
   cout << "b = ";
   cin >> b;
   cout << "c = ";
   cin >> c;
   cout << "tolerance = ";
   cin >> tol;
   tester.brent( a, b, c, tol );

   return 0;
}
```

11.17.3 Find Roots

The following method finds roots of function F using Brent's method:[14]

- F.zbrent(double $x1$, double $x2$, double *tol*): takes 2 x-axis bounds $x1$ and $x2$, and tolerance *tol*, and returns the root of function F between those bounds.

Example

```
class TestIt {
public:
   TestIt(){}
   ~TestIt(){}
   void zbrent( double x1, double x2, double tol )
   {
      try
      {
         Funct< TestIt > f( *this, &TestIt::xsqrd );
         cout << "zbrent = " << f.zbrent( x1, x2, tol ) << endl;
      }
      catch ( Funct< TestIt >::functErr& e ) { cout << e.what() <<
      endl; }
   }

private:
   double xsqrd( double x ){ return x * x - 4; }
};

int main()
{
   TestIt tester;
   double x1, x2, tol;
   cout << "For testing zbrent, x1 = ";
   cin >> x1;
   cout << "x2 = ";
   cin >> x2;
   cout << "tolerance = ";
   cin >> tol;
   tester.zbrent( x1, x2, tol );

   return 0;
}
```

[14] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 359–362

11.18 Linear Regression

The *XY* class performs least squares linear regression. The *XY* constructors takes as their arguments vector< double >s containing the x- and y-coordinates, and if present, the x- and y-standard deviations:[15]

- XY(vector< double >& x, vector< double >& y): loads an XY dataset with x-coordinates x and y-coordinates y without standard deviations
- XY(vector< double >& x, vector< double >& y, vector < double >& σ_x): loads an XY dataset with x-coordinates x, y-coordinates y, and x standard deviations σ_x
- XY(vector< double >& x, vector< double >& y, vector < double >& σ_x, vector < double >& σ_y): loads an XY dataset with x-coordinates x, y-coordinates y, x standard deviations σ_x, and y standard deviations σ_y

The following accessors return results of least squares linear regression on *XY* dataset D (all of type double):

- D.a(): returns coefficient a (y-intercept) in least squares linear fit $y = a + bx$
- D.b(): returns coefficient b (slope) in least squares linear fit $y = a + bx$
- D.x0(): returns x-intercept x_0 in least squares linear fit $y = a + bx$
- D.siga(): returns standard errors on a
- D.sigb(): returns standard errors on b
- D.chi2(): returns minimized chi-squared value of least squares fit
- D.q(): returns chi-squared probability of least squares fit (small is a poor fit)
- D.r(): returns correlation coefficient r

11.18.1 Notes

Errors are caught by *XYErr* (see below for examples).

[15] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 661–670

11.18.2 Examples

```
double xs[ 5 ] = { 1.0, 2.0, 3.0, 4.0, 5.0 };
double ys[ 5 ] = { 1.0, 2.2, 3.0, 4.0, 6.0 };
double stdxs[ 5 ] = { 0.2, 0.4, 0.2, 0.2, 1 };
double stdys[ 5 ] = { 0.2, 0.3, 0.2, 0.2, 0.8 };

vector< double > xv( xs, xs + 5 ), yv( ys, ys + 5 ), stdx( stdxs,
stdxs + 5 ),
   stdy( stdys, stdys + 5 );

try {
   XY xyset( xv, yv );
   cout << "For dataset without errors, a = " << xyset.a() << endl;
   cout << "b = " << xyset.b() << endl;
   cout << "x0 = " << xyset.x0() << endl;
   cout << "siga = " << xyset.siga() << endl;
   cout << "sigb = " << xyset.sigb() << endl;
   cout << "chi2 = " << xyset.chi2() << endl;
   cout << "q = " << xyset.q() << endl;
   cout << "r = " << xyset.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl;

try {
   XY xyset2( xv, yv, stdx );
   cout << "For dataset with x errors only, a = " << xyset2.a() <<
   endl;
   cout << "b = " << xyset2.b() << endl;
   cout << "x0 = " << xyset2.x0() << endl;
   cout << "siga = " << xyset2.siga() << endl;
   cout << "sigb = " << xyset2.sigb() << endl;
   cout << "chi2 = " << xyset2.chi2() << endl;
   cout << "q = " << xyset2.q() << endl;
   cout << "r = " << xyset2.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl;

try {
   XY xyset3( xv, yv, stdx, stdy );
   cout << "For dataset with x & y errors, a = " << xyset3.a() <<
   endl;
   cout << "b = " << xyset3.b() << endl;
   cout << "x0 = " << xyset3.x0() << endl;
   cout << "siga = " << xyset3.siga() << endl;
   cout << "sigb = " << xyset3.sigb() << endl;
```

```
   cout << "chi2 = " << xyset3.chi2() << endl;
   cout << "q = " << xyset3.q() << endl;
   cout << "r = " << xyset3.r() << endl << endl;
}
catch ( XY::XYErr& e ) { cout << e.what() << endl; }
```

Chapter 12

Object Design

The design of the network object pursues 2 objectives. The first is a well organized, object oriented representation of a generic network object that is flexible and easily extended to specific network types. In fact, the current design allows for all models, including statistical ones.

The second objective of the network object design is that the code be readable. This was the logic of constructing efficient yet readable matrix and vector operations, as described in Chapter 9.

Figure 12.1 demonstrates the current inheritance hierarchy and the services which compose experiments around it. It is split into two panels so inheritance is never confused with a call dependency. In panel A an open arrow runs from a base class to a derived class. In panel B every arrow means that the source calls or uses the target; boxes ending in "API" are the class-layer interfaces used by those services. All boxes use the same plain shape; no decorative mark carries a second meaning.

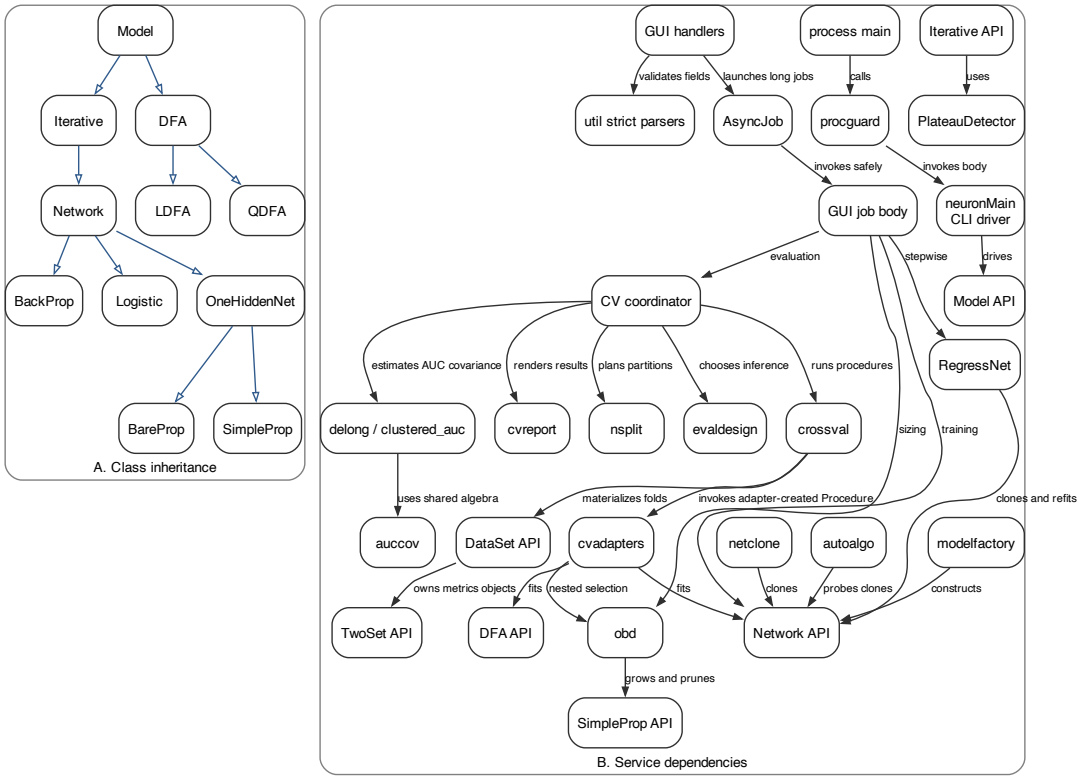

Figure 12.1: Current class inheritance and service dependencies

- **DataSet** contains and manipulates the data to be modeled.
- **TwoSet** is a specialized object designed to calculate metrics for *one* output *Models*, including sensitivity, specificity, predictive value positive, predictive value negative, and ROC area.
- **Model** is the base object. *Model* provides access to the dataset, and contains the virtual methods common to all modeling techniques.
- **Iterative** extends *Model*. Neural computational algorithms are iterative, and thus use *Iterative* as a base. *Iterative* manages methods and parameters related to iterating through a dataset.
- **Network** extends *Iterative*. Algorithms with hidden layers, weights, learning rates, transfer functions and other general properties of neural networks are encapsulated in *Network*.
- **RegressNet** is a compound object including a *Network* object which implements stepwise regression. See Chapter 13.
- **BackProp** extends *Network*. *BackProp* is a multilayer multiple output back-propagation neural network.
- **SimpleProp** also extends *Network*. *SimpleProp* is a backpropagation neural network which has 1 output node, 1 hidden node layer, and biases.
- **BareProp** also extends *Network*. *BareProp* is an example *Network* object for illustrative purposes which has 1 output node, 1 hidden node layer, and no biases.
- **Logistic** also extends *Network*. *Logistic* implements binary logistic regression, which is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function.
- **DFA** extends *Model*, and encodes discriminant function analysis. **QDFA** extends *DFA*, and encodes quadratic discriminant function analysis. **LDFA** extends *DFA*, and encodes linear discriminant function analysis.

12.1 Current object and service map

Figure 12.1 includes both the inheritance hierarchy and the modern services that compose experiments around it. Panel A is only inheritance, with open arrows from base to derived classes. Panel B is only dependencies: an arrow means that the source calls or uses the target, and an "API" suffix says explicitly that the target is a class-layer interface rather than another orchestration service. The panels deliberately use one uncomplicated box shape.

The important division is between mechanisms and coordination. `DataSet` owns data and materializes row partitions; `nsplit` chooses indices; `Network`

objects fit models; `obd` selects an architecture; `crossval` repeats a supplied procedure; `evaldesign` records what design was achieved and which inference it permits; the AUC covariance modules estimate uncertainty; and `cvreport` renders and writes results. No outer coordinator reimplements a lower layer's rule.

12.1.1 Current additions to DataSet

The foundational `DataSet` design later in this chapter explains its ownership and normalization model. The source and the current contract here govern where an older signature or field description differs. The following public methods extend it for validation and planned partitions:

- `bool valLoaded()`: reports whether a validation matrix is present; it does not inspect or create one.
- `Matrix<double>& getValMatrix()`: returns the owned validation matrix. Call it only when `valLoaded()` is true; mutating it changes the data used for validation.
- `unsigned getNumVal()`: returns the number of validation exemplars, or zero when no validation set is loaded.
- `MonitorSet monitorSet()`: returns `MONITOR_NONE`, `MONITOR_VALIDATION`, or `MONITOR_TEST`; validation takes precedence over test, and multi-output data returns `MONITOR_NONE` because this monitor is defined only for one output.
- `const char* monitorSetName()`: returns the report label for the same choice: "validation", "test", or "none".
- `bool randomize3(unsigned nTest,unsigned nVal)`: creates an outcome-stratified train/validation/test split with exact requested test and validation counts; it returns false and reports an impossible request.
- `bool randomize3D(double testRatio,double valRatio)`: supplies the same operation in fractions, converts them to counts, and returns the status from `randomize3`.
- `void makeFold(const vector<unsigned>& trainRows,const vector<unsigned>& testRows,const vector<unsigned>& valRows={})`: materializes explicit raw-row indices and computes every scale from the training rows only. Matrix bounds are enforced here; callers that require a complete, non-overlapping partition validate it with `nsplit` before materialization.
- `void setStrataColumns(const vector<unsigned>& columns)`: stores the zero-based input columns used for covariate strata.

- `const vector<unsigned>& getStrataColumns() const`: returns that stored configuration without rebuilding keys.
- `void setStrataBins(unsigned bins)`: stores the quantile-bin count for continuous stratum columns; the caller validates its domain.
- `unsigned getStrataBins() const`: returns the configured count.
- `void setGroupColumns(const vector<unsigned>& columns)`: stores the zero-based columns whose joint values define indivisible groups.
- `const vector<unsigned>& getGroupColumns() const`: returns that configuration without calculating group identities.
- `vector<unsigned> strataKey(columns,bins) const`: returns dense outcome-by-covariate cell identifiers; `groupKey(columns)` returns dense exact-match group identifiers. Both read the raw matrix without changing configuration; columns outside the input range are ignored because the public REST caller validates them before reaching this mechanism.
- `vector<unsigned> groupKey(columns) const`: returns one dense identifier per raw row for exact joint values in the named columns. If no usable column remains, every row receives its own group rather than being collapsed into one accidental cluster.

Example

```
vector<unsigned> group = data.groupKey({2, 3, 4});
nsplit::GroupFoldPlan p =
    nsplit::stratifiedGroupKFold(label, group, 5);
data.makeFold(trainRows, testRows, validationRows);
cout << data.monitorSetName();       // "validation"
```

Notes

Column positions passed by C++ callers are zero-based input positions. Public REST fields are one-based and are validated and converted by the handler. Grouping takes precedence over covariate stratification in the ordinary raw split. A three-way split is outcome-stratified and does not currently compose with either policy.

12.2 Current contracts added to established objects

The detailed specifications later in this chapter preserve the foundational design and equations. The source and this section govern the current public API where the older record differs; the contracts below must not be read as optional addenda to a superseded signature or field.

12.2.1 Model

The following method exposes the objective needed by likelihood-based callers:

- `bool getXEerror() const`: returns `true` when cross-entropy is active and `false` for least-mean-square error.

Callers whose mathematics depends on likelihood, notably stepwise Wilks comparisons, query this state and refuse an incompatible source fit; they do not mutate the model's objective to make it appear compatible.

Example and notes

```
if (!model.getXEerror())
    throw runtime_error("Wilks comparison requires cross-entropy");
```

The query is observational and does not promise that every cross-entropy model is otherwise eligible for a particular likelihood analysis.

12.2.2 TwoSet

`TwoSet` now exposes explicit cache invalidation and structured ROC results. Its empirical AUC is a ranking probability: the probability that a random outcome-1 row receives a higher score than a random outcome-0 row, with half credit for a tie. This is the normalized Mann–Whitney statistic, not an area obtained by choosing an arbitrary number of thresholds.[1]

The bootstrap interval repeatedly resamples outcome-0 and outcome-1 rows within their own classes, recomputes the fitted binormal area, and reports its 2.5th and 97.5th empirical percentiles. Resampling is stratified so a replicate cannot lose an outcome class merely by chance. This is the ordinary nonparametric bootstrap applied to the statistic, with a repository-specific stratification rule.[2] The following methods are available:

- `void invalidate()`: clears every cached statistic after a caller changes predictions through a mutable accessor.
- `unsigned getTP()`: returns true positives at the current threshold.
- `unsigned getTN()`: returns true negatives at the current threshold.
- `unsigned getFP()`: returns false positives at the current threshold.
- `unsigned getFN()`: returns false negatives at the current threshold.

[1] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology* 143(1), 29–36, 1982, https://doi.org/10.1148/radiology.143.1.7063747.

[2] B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics* 7(1), 1–26, 1979, https://doi.org/10.1214/aos/1176344552.

- `double getTrapROCarea()`: returns the exact empirical Mann–Whitney AUC over every distinct operating point.
- `const vector<double>& getROCx()`: returns the empirical ROC false-alarm coordinates, 1−specificity.
- `const vector<double>& getROCy()`: returns the aligned empirical hit-rate coordinates, sensitivity.
- `ROCfit getROCfit()`: returns the current binormal ROC fit as a structured value; an unsupported fit has `valid=false`.
- `void bootstrapROC()`: performs the configured number of outcome-stratified bootstrap resamples and caches a percentile interval for the binormal area.
- `CI getStatCi()`: returns the cached bootstrap interval.
- `void setBootstrapResamples(unsigned n)`: sets the requested resample count; zero disables the bootstrap.
- `unsigned getBootstrapResamples()`: returns that count.
- `unsigned getStatPoints()`: returns the number of distinct interior operating points fitted by the zROC line.
- `double getStatChi2()`: returns the zROC fit chi-square.
- `double getPearsonX2()`: returns the individual-level Pearson statistic; it is not assigned a chi-square p-value on continuous data.

`ROCfit` contains the following public variables:

- `double az`: fitted binormal ROC area.
- `double chi2`: zROC fit chi-square, or not-a-number when it cannot be computed.
- `double p`: fit p-value, or not-a-number when unavailable.
- `unsigned points`: operating points included in the fit.
- `bool valid`: whether a fit was computed.

`CI` contains:

- `double lo, hi`: 2.5th and 97.5th percentiles.
- `double se`: standard deviation of successful resampled areas.
- `unsigned resamples`: successful resamples.
- `unsigned failures`: resamples for which no area was computable.
- `bool valid`: whether enough resamples succeeded.

Example and notes

```
TwoSet& test = data.getTestTwoSet();
double empirical = test.getTrapROCarea();
test.bootstrapROC();
TwoSet::CI interval = test.getStatCi();
```

Mutable prediction access requires `invalidate()` before another cached statistic is read. The bootstrap uses an independent resampling stream and cannot perturb training. A caller checks `valid` and the failure count before presenting an interval.

12.2.3 Iterative

`StopReason` identifies why the last run ended:

- `STOP_NONE`: training has not run.
- `STOP_MIN_ERROR`, `STOP_CHANGE`, `STOP_WINDOW`, `STOP_GRADMAX`, `STOP_PLATEAU`, and `STOP_EARLY_STOP`: a configured stopping rule fired, so the run converged under its declared rule.
- `STOP_MAX_ITERATIONS`: the safety ceiling was exhausted; this is not convergence.
- `STOP_CANCELLED`: an external request stopped the fit.
- `STOP_PROBE_BUDGET`: an optimizer probe used its wall-clock allowance; the probe is an experiment, not a completed fit.

The following methods expose and control this state:

- `static const char* stopReasonToken(StopReason r)`: returns the single machine-readable name used in reports and JSON.
- `static bool converged(StopReason r)`: returns `true` only for the six stopping-rule outcomes listed above.
- `void setObserver(Observer* obs)`: installs a non-owning iteration observer; null removes it, and clones do not copy the pointer.
- `Observer* getObserver()`: returns the installed pointer or null.
- `StopReason getStopReason()`: returns the last run's reason.
- `unsigned getIterations()`: returns iterations completed by the last run.
- `void setAutoStop(bool flag,double tol,unsigned win)`: enables or disables plateau stopping and sets its tolerance and window.

- `bool getAutoStop()`: reports whether plateau stopping is enabled.
- `double getAutoStopTol()`: returns its relative-improvement tolerance.
- `unsigned getAutoStopWindow()`: returns its window width.
- `void setQuiet(bool flag)`: enables or disables training narration and the reporting epilogue; it never suppresses fitting or stopping work.
- `bool getQuiet()`: returns the current narration setting.

An `Iterative::Observer` supplies:

- `bool onIteration(unsigned iteration,double setError)`: receives every completed iteration and returns `false` to stop.
- `StopReason whyStopped() const`: identifies why the observer returned `false`; the conservative default is cancellation.

Example and notes

```
network.setAutoStop(true, 1e-4, 100);
double error = network.train();
if (!Iterative::converged(network.getStopReason())) rejectFit(error);
```

Reaching `STOP_MAX_ITERATIONS` means the operation returned weights, not that the fit converged. The observer is non-owning and must outlive the run. Quiet mode changes output only.

12.2.4 Network and concrete models

The following current methods supplement the detailed `Network` specification later in this chapter:

- `double sampleTestError(unsigned stride)`: returns mean error over every `stride`-th exemplar in the authoritative validation/test monitoring set without changing cached statistics; returns -1 when no one-output monitoring set exists.
- `double getCondNum() const`: returns the last computed condition number, or -1 before calculation.
- `double getCondMaxEig() const`: returns the largest absolute eigenvalue used in the condition number, or -1 before calculation.
- `double getCondMinEig() const`: returns the corresponding smallest absolute eigenvalue, or -1 before calculation.

The packed parameter boundary

These four `protected` contracts exist because a genuine Wolfe line search must evaluate the objective at *trial* points, and `innerTrainSet()` cannot serve as that evaluator: it updates the weights. They are not a public capability and have no menu, GUI or API surface; the sole consumer is `Network::Evaluator`, which serves `LBFGS`.

- `virtual unsigned packedSize() const`: the number of doubles a packed parameter vector holds. *Zero means the boundary is unavailable* — it is the eligibility question, asked before a run.
- `virtual void packWeights(vector<double>& destination) const`: writes the current weights in the one layout the gradient also uses, so a step computed in one applies to the other. `OneHiddenNet` writes `hW` row by row then `oW`, for both bias architectures, taking every width from `hW`'s own dimensions rather than from a flag; `BackProp` writes `Weights[0]` through the output layer in container order.
- `virtual void unpackWeights(const vector<double>& source)`: installs packed parameters, validating the length once at entry and throwing `nvec::SizeMismatch` otherwise. A refused call installs nothing.
- `virtual double batchObjectiveGradient(vector<double>& packedRawGradient)`: returns the mean batch objective at the currently installed weights, including the current weight-decay policy, and writes the *raw* mean gradient of that same objective — the gradient itself, never a search direction. It updates no weight, no optimizer history, no iteration state, no report and no cached statistic; it writes only the model's per-pass scratch, as `forward()` writes `o`. One virtual call per full evaluation, and none inside the exemplar loop it runs.

The default implementations refuse: `packedSize()` returns zero and the other three throw `Network::NoPackedBoundary`. Refusal is the point — an empty parameter vector is a value a caller would act on, and a packed optimizer handed one would report that it had converged. `Logistic` deliberately does not implement the boundary.

The batch separate-gradient path of `OneHiddenNet::innerTrainSet()` and `BackProp::innerTrainSet()` consumes the same authoritative evaluation, so the model equations exist once and a trial point and a training iteration cannot disagree.

`OneHiddenNet` sits between `Network` and the two one-hidden-layer models. `SimpleProp` and `BareProp` describe the same architecture — one hidden layer and one output node — differing only in whether bias nodes are present. They therefore carry identical state, and several of their methods differed by nothing but the class name. `OneHiddenNet` owns that state (the hidden weight, update, and gradient matrices; the input, hidden-output, output-weight, hidden-error,

and output-gradient vectors; the output error term; and the hidden-unit count) and implements what does not depend on the bias architecture:

- `virtual void setHidden(unsigned n)`: pure virtual. Sizing is the bias architecture, so each concrete model sizes its own structures; `load` calls this method rather than deciding widths itself.
- `virtual void randomize()`: assigns uniform random weights within the configured limit and marks the weights set.
- `virtual bool save(string& filename)`: writes the model file — the concrete class's `outputHeader`, the hidden layer, then the output layer.
- `virtual bool load(string& filename)`: reads that file back, refusing a file whose input count disagrees with the loaded dataset.
- `virtual void pack()` (protected): flattens the hidden and output gradients into the packed gradient vector the conjugate-direction optimizers use.
- `virtual double innerTrainSet()`: performs one training pass over the training set and returns the set error. This is the training loop itself — online and batch, canonical descent and the separate-gradient path, weight decay, and the optimizer call — for both models.
- `void propagate()`: applies the forward equations to the exemplar already in the input vector, with the hidden bias slot already established if the model has one. Non-virtual, and called directly by each concrete `forward()`: this is a per-exemplar path and may not acquire indirect calls in order to remove source duplication.

The training loop and the forward equations are shared with no parameterization at all, rather than with a bias flag. Their one differing statement, the hidden error term, is the same formula once its domain is written out: elements 0 through $n_{\text{hidden}} - 1$ are the hidden units, which is every element of an unbiased hidden-output vector and all but the pinned bias slot of a biased one. The ranged operations of `Matrix` and the vector operators express that domain directly.

Three things deliberately remain in the concrete classes. `df()` is two published parameter counts, $((n_{\text{input}} + 2)n_{\text{hidden}}) + 1$ with biases and $(n_{\text{input}} + 1)n_{\text{hidden}}$ without. `outputHeader()` writes a model-file format line that `load` reads back. Every width that follows from the bias column and the pinned bias slot — `setDataSet`, `setHidden`, `unpack`, `removeInputs`, and the statements of `forward()` that read the exemplar and, in `SimpleProp` alone, pin the bias slot — stays where the biased and unbiased architectures can be read side by side.

Two notes matter to callers. `SimpleProp` and `BareProp` remain distinct concrete types: type identity drives cloning, the model factory, and the first line of every saved model file, and neither is assignable to the other. Nothing

in the shared implementation reads `Network::biasFlag`; that flag is publicly settable, and a shared method that consulted it could reshape a live model or change what it writes to disk. The bias architecture is a property of the concrete type.

`SimpleProp` adds the following OBD mechanisms:

- `void growHidden(unsigned extra)`: adds hidden units as a warm start, preserving existing weights and assigning each new unit zero outgoing contribution.
- `void removeHidden(const vector<unsigned>& indices)`: removes the named non-bias hidden units while preserving the order and weights of survivors; at least one unit must remain.
- `vector<double> hiddenSaliency()`: returns one saliency per non-bias hidden unit, $|w_j|$ times the training-set standard deviation of that unit's output.

`Logistic` exposes its last coefficient audit:

- `const vector<double>& getBetas() const`: returns fitted coefficients; the last element is the intercept.
- `const vector<double>& getBetaSE() const`: returns coefficient standard errors; -1 marks an unavailable value.
- `const vector<double>& getWaldP() const`: returns coefficient Wald-test p-values.

Example and notes

```
double monitor = network.sampleTestError(20);
if (monitor >= 0) plot(monitor);
const vector<double>& beta = logistic.getBetas();
```

The sampling call is diagnostic and does not invalidate the full test report. Condition-number values are populated only after their calculation. Hidden-unit growth/removal applies only to `SimpleProp`; callers retain type identity and never use the shared base to infer a bias architecture.

12.2.5 DFA, LDFA and QDFA

`DFA` is the discriminant-analysis base. It owns the class matrices, mean vectors, a priori probabilities and constants that both concrete analyses build, and since 2026 it also owns the run itself:

- `double train() override final`: performs one analysis. It writes the banner and header, calls the fit and then the report, catches a singular covariance and reports the refusal, writes the history and last-operation

files, and returns -1 because a discriminant analysis has no set error. It is `final` because no subclass may replace the scaffold, and it remains a virtual override of `Model::train()` — cross-validation reaches it through a `unique_ptr<Model>`.

- `virtual void fitDiscriminant()`: pure virtual, protected. The fit, and only the fit. Runs once per analysis, inside the scaffold's `try`.
- `void reportAccuracy(ostream&) override = 0`: pure virtual. Each model writes this out in full.

`LDFA` fits one *pooled* covariance across all classes, inverts it, and forms constants $K_c = \frac{1}{2}\mathbf{u}_c^{\mathsf{T}} S \mathbf{u}_c - \log P_c$. Its discriminant is $d_c = \mathbf{u}_c^{\mathsf{T}} S \mathbf{x} - K_c$ and the *larger* discriminant wins; the binary graded score is $\sigma(d_1 - d_0)$.

`QDFA` fits a *separate* covariance per class, inverts each, and forms constants $K_c = \log |C_c| - \log P_c$. Its discriminant is $d_c = (\mathbf{x} - \mathbf{u}_c)^{\mathsf{T}} S_c (\mathbf{x} - \mathbf{u}_c) + K_c$ and the *smaller* discriminant wins, because it is a distance where the linear one is a similarity; the binary graded score is $\sigma(d_0 - d_1)$.

Those opposite senses are why only the scaffold is shared. Sharing the per-exemplar scoring loops would require either a virtual call per exemplar or a comparator parameter, and the base class deliberately contains no sign, winner flag, formula descriptor or type switch: the published mathematics stays in the class that owns it. Both models write a *graded* score through the logistic function rather than a hard decision, so the ROC sweep sees a curve; the classification boundary is unchanged.

Example and notes

```
unique_ptr<Model> analysis(new LDFA);
analysis->setDataSet(data);
analysis->train();              // dispatches to DFA's final scaffold
```

A singular covariance is reported by the scaffold as an analysis refusal. The opposite linear/quadratic winner senses are mathematical contracts and may not be normalized through a sign or comparator flag.

12.2.6 RegressNet

The following methods support non-interactive, cancellable stepwise analysis:

- `void setThreshold(double t)`: supplies the p-value threshold without an interactive prompt.
- `void setProgress(ProgressFn f)`: installs the progress callback.
- `void setObserver(Iterative::Observer* obs)`: installs the non-owning cancellation observer on every candidate clone.

- `const vector<Candidate>& getCandidates() const`: returns every candidate in evaluation order.
- `const vector<pair<double,unsigned> >& getSelectionPath() const`: returns selected p-value/variable pairs in selection order.
- `const vector<unsigned>& getFinalVariables() const`: returns the variables settled by completed passes.
- `unsigned getFitsCompleted() const`: returns candidate fits made.
- `bool getComplete() const`: distinguishes a completed selection from a partial audit after failure or cancellation.

`Progress` is one callback snapshot:

- `string direction`: "forward" or "reverse".
- `unsigned step`: zero-based outer selection pass.
- `unsigned candidate` and `candidatesThisStep`: one-based position and the fixed candidate count for this pass.
- `unsigned fitsCompleted`: candidates finished across all passes.
- `unsigned variable` and `vector<unsigned> inputs`: conceptual variable and its complete, never-split input-node group.
- `string phase`: human-readable work currently underway.
- `bool finished`, `bool converged`, and `string stopReason`: status of the most recently completed candidate; convergence and reason are meaningful when finished.

`Candidate` is the permanent audit row written before eligibility is judged:

- `step`, `candidate`, and `variable`: selection identity.
- `inputs`: complete input-node group tested together.
- `priorError` and `error`: likelihood objectives compared.
- `df`: difference in free parameters.
- `G2` and `p`: Wilks statistic and p-value; not-a-number when a failed or cancelled fit was recorded but never compared.
- `converged`, `stopReason`, and `iterations`: fit validity and work used.
- `selected`: whether this candidate won its pass.

Example

```
RegressNet stepwise(network);
stepwise.setThreshold(0.05);
stepwise.setProgress([](const RegressNet::Progress& p) {
    cout << p.direction << " pass " << p.step << endl;
});
stepwise.reverse_regress();
if (stepwise.getComplete())
    use(stepwise.getFinalVariables());
```

Notes

Stepwise candidate fits must converge under cross-entropy before entering a Wilks comparison, and the original model is never mutated. A grouped variable is always added or removed as a unit. A partial audit after cancellation or a failed candidate remains inspectable, but `getComplete()` must be true before `getFinalVariables()` is reported as the procedure's conclusion.

12.3 Partition planning: nsplit

The `nsplit` namespace is an index-level service. It never scales data, fits a model, or interprets an input column. Its allocation and validation rules are project-defined planning mechanisms, not a statistical test or a claim that one universally optimal split exists. They make three policies explicit: preserve requested strata as closely as indivisible rows permit, keep a declared group wholly on one side, and audit the completed assignment rather than trusting the construction procedure. "Largest remainder" below means that each stratum first receives its integer quota and the remaining test positions go to the largest fractional quotas until the requested total is reached.

`Holdout` contains:

- `vector<unsigned> test, train`: row positions assigned to each side.
- `unsigned n0, n1`: whole-sample outcome counts.
- `unsigned n0Test, n1Test`: achieved test-set outcome counts.

`StratHoldout` contains:

- `vector<unsigned> test, train`: row positions assigned to each side.
- `vector<unsigned> cellTotal`: whole-sample count in each stratum.
- `vector<unsigned> cellTest`: achieved test count in each stratum.

`GroupHoldout` contains:

- `vector<unsigned> test, train`: row positions assigned to each side, with every group wholly on one side.
- `unsigned nGroups`: number of distinct groups.
- `vector<unsigned> groupsInTest`: group identifiers assigned to the test side.

`FoldPlan` contains:

- `bool ok` and `string reason`: planning status and refusal.
- `vector<unsigned> foldId`: fold number for every input row.
- `unsigned k, nStrata`: requested folds and observed strata.
- `vector<unsigned> foldSize`: achieved rows per fold.
- `vector<vector<unsigned> > stratumByFold`: achieved stratum counts by fold.
- `vector<string> warnings`: non-fatal planning facts.

`GroupFoldPlan` extends `FoldPlan` with:

- `unsigned nGroups`: distinct groups in the planned rows.
- `vector<unsigned> groupsPerFold`: whole groups assigned per fold.
- `vector<array<unsigned,2> > classByFold`: negative and positive rows achieved per fold.
- `unsigned leakageCount`: groups found in more than one fold; a valid plan has zero.
- `double imbalanceScore`: worst relative row or class imbalance.
- `unsigned largestGroup`: size of the largest indivisible group.

The following functions are available:

- `string partitionError(unsigned nRows,train,test,bool coverage)`: returns an empty string for a valid two-way row partition, otherwise a reason describing an out-of-range, duplicate, overlapping, or missing row.
- `Holdout stratifiedHoldout(label,nTest)`: returns an outcome-stratified train/test partition of binary labels.
- `StratHoldout holdoutByStrata(stratum,nTest)`: apportions test positions across arbitrary stratum identifiers by largest remainder.

- `GroupHoldout groupHoldout(label,group,nTest)`: assigns each complete group to training or test while approaching the requested size and outcome balance.
- `FoldPlan stratifiedKFold(stratum,k)`: returns a balanced k-fold assignment over arbitrary stratum identifiers.
- `GroupFoldPlan stratifiedGroupKFold(label,group,k)`: returns an outcome-balanced whole-group fold assignment with measured leakage and imbalance.
- `vector<unsigned> kFold(label,k)`: returns the outcome-only case of `stratifiedKFold`.

Example

```
util::set_seed(42);
nsplit::GroupFoldPlan plan =
    nsplit::stratifiedGroupKFold(label, group, 5);
if (!plan.ok || plan.leakageCount != 0)
    throw runtime_error(plan.reason);
```

Notes

Every plan is reproducible under the engine seed. Group identifiers are merely equivalence labels; renumbering them must not change the partition. Achieved fold counts and leakage are recomputed from the final assignment. Indivisible groups may make exact sizes or exact class balance impossible, so imbalance is reported rather than hidden. Fewer than `k` groups is a refusal.

12.4 Typed evaluation design: evaldesign

`evaldesign` is a project-defined vocabulary and policy layer; it computes no statistic. It records what was actually partitioned and what the independent sampling unit is, then permits only the covariance method whose assumptions match those declarations. The purpose is to prevent a GUI, report writer or caller from inferring statistical independence from a loose string such as "grouped," or silently choosing an estimator merely because one is available. Its enumerations are:

- `SamplingUnit`: `Unspecified`, `Row`, or `Cluster`, declaring the independent sampling unit.
- `AucInference`: `None`, `DeLongIndependent`, or `ObuchowskiClustered`, naming the permitted estimator.

- `PartitionMethod`: `OutcomeStratified`, `CovariateStratified`, or `StratifiedGroup`, naming how rows were assigned.

`Partition` contains the following public variables:

- `PartitionMethod method`: assignment method.
- `unsigned seed`: reproducibility seed.
- `unsigned k`: number of folds; zero for a holdout.
- `vector<unsigned> strataColumns`: caller's one-based covariate columns; `strataBins` and `nStrata` give bin count and achieved cell count.
- `vector<unsigned> groupColumns`: caller's one-based group columns; `nGroups` gives distinct groups in the partitioned rows.
- `bool developmentOnly`: whether the folds exclude a locked test.
- `unsigned nRequested, nAchieved`: requested and achieved holdout sizes; indivisible groups may make them differ.
- `unsigned leakage`: groups or rows found on both sides; a valid group plan requires zero.
- `vector<string> warnings`: non-fatal design facts.
- `string refusal`: reason no partition could be built.
- `foldRows, foldEvents, foldGroups`: parallel achieved counts for each fold.
- `double imbalanceScore`: worst relative deviation from a fold's fair row or class share.
- `unsigned largestGroup`: largest indivisible cluster size.

`InferenceChoice` contains:

- `AucInference method`: estimator permitted by the declared design.
- `string reason`: required explanation when `method` is `None`; empty when an estimator is permitted.

The following functions are available:

- `bool parseSamplingUnit(string text,SamplingUnit& out)`: parses the REST value `rows`, `cluster`, or an empty declaration; returns `false` for an unrecognized token.

- `samplingUnitName`, `samplingUnitPhrase`, and `independenceStatus`: return the authoritative display wording for a sampling-unit declaration.
- `inferenceName`, `inferenceText`, and `inferenceShortName`: return long, reason-bearing, and compact names for an inference choice.
- `partitionMethodName`: returns the display name of a partition method.
- `string describeFolds(const Partition& p)`: derives a fold-plan sentence from structured fields.
- `string describeHoldout(const Partition& p)`: derives the corresponding holdout sentence, including requested and achieved sizes.
- `InferenceChoice chooseInference(unit,split,estimable,reason)`: returns the only covariance estimator permitted by the declared unit and achieved split, or `None` with a reason.

Display strings must come from these functions rather than being composed by a GUI or report.

Example

```
evaldesign::InferenceChoice c = evaldesign::chooseInference(
    evaldesign::SamplingUnit::Cluster,
    evaldesign::PartitionMethod::StratifiedGroup,
    clustered.ok, clustered.reason);
```

Notes

A group-disjoint split prevents leakage; it does not establish row independence. An unspecified unit intentionally withholds inferential claims. Independent-row inference over a grouped design is refused, and neither covariance estimator may silently substitute for the other.

12.5 Shared AUC covariance algebra: auccov

`auccov` owns the operations that are invariant to the sampling unit. For a positive score X and negative score Y, it computes

$$A = P(X > Y) + \frac{1}{2}P(X = Y),$$

the probability that the model ranks a randomly chosen positive row above a randomly chosen negative row, with half credit for ties. In the sample this is the Mann–Whitney pair count divided by $n_1 n_0$.[3] The per-positive and per-negative pair averages are the structural components called `v10` and `v01`. They

[3] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics* 18(1), 50–60, 1947, https://doi.org/10.1214/aoms/1177730491.

are retained because both covariance estimators below use them; only the rule for combining their variation changes.

`Placements` contains:

- `bool ok` and `string reason`: computation status.
- `unsigned n0, n1`: negative and positive row counts.
- `vector<unsigned> pos, neg`: original row indices by outcome.
- `vector<vector<double> > v10, v01`: procedure-by-observation structural components.
- `vector<double> auc`: exact point AUC for each procedure.

`Interval` contains:

- `double auc`: point area.
- `double se`: covariance-derived standard error.
- `double lo` and `hi`: normal-interval limits.
- `bool valid`: whether the covariance supported an interval.

`Contrast` contains:

- `aucI`, `aucJ`, `seI`, and `seJ`: the two point areas and their marginal standard errors.
- `delta` and `seDelta`: $AUC_i - AUC_j$ and its paired error.
- `ciLoI`/`ciHiI`, `ciLoJ`/`ciHiJ`, and `ciLoDelta`/`ciHiDelta`: interval limits for both areas and their difference.
- `z` and `p`: normal statistic and two-sided p-value.
- `degenerate`: equal areas with zero difference variance.
- `separated`: nonzero deterministic difference with zero variance.
- `string note`: qualification for either zero-variance case or refusal.
- `bool valid`: whether the requested paired contrast was computable.

The following functions are available:

- `vector<double> midranks(const vector<double>& x)`: returns one-based ranks; tied observations receive the mean rank they span.
- `Placements compute(label,pred)`: returns positive/negative row indices, structural components, and exact point AUCs for paired procedures.

- `Interval interval(auc,cov,i,bool clamp)`: returns the normal interval for area `i`; `clamp` controls truncation to $[0, 1]$.
- `Contrast contrast(auc,cov,i,j,bool clamp)`: returns $AUC_i - AUC_j$, its standard error, interval, z statistic, and two-sided p-value.

The estimator above this layer supplies the covariance matrix; it does not reimplement these operations.

Example

```
auccov::Placements p = auccov::compute(label, predictions);
auccov::Contrast c =
    auccov::contrast(p.auc, covariance, 0, 1, false);
```

Notes

Ties receive half credit through common mid-ranks, so the ordinary and clustered paths have identical point AUCs. A zero-variance, zero-difference contrast has `p=1`; a zero-variance nonzero difference is deterministic separation with `p=0`. A materially negative difference variance is refused.

12.6 AUC covariance estimators

12.6.1 delong

DeLong, DeLong and Clarke-Pearson answer a specific question: when two or more procedures score the *same independent subjects*, how uncertain is each empirical AUC, and how uncertain is the difference between two correlated AUCs? Treating the areas as independent would discard the pairing and generally give the wrong standard error. Their nonparametric method views each AUC as a two-sample U-statistic and estimates the full procedure-by-procedure covariance from the positive and negative structural components.[4]

If S_{10} is the sample covariance of the per-positive components and S_{01} the sample covariance of the per-negative components, the implemented estimator is

$$\widehat{\mathrm{Cov}}(A) = \frac{S_{10}}{n_1} + \frac{S_{01}}{n_0}.$$

The diagonal entries are variances of individual AUCs; off-diagonal entries preserve the correlation between procedures evaluated on the same rows. A contrast uses $\mathrm{Var}(A_i - A_j) = C_{ii} + C_{jj} - 2C_{ij}$, then reports the normal z statistic and two-sided p-value. "DeLong" therefore names the covariance estimator, not a different point-AUC calculation.

`delong::Result` contains:

[4] E. R. DeLong, D. M. DeLong and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach," *Biometrics* 44(3), 837–845, 1988, https://doi.org/10.2307/2531595.

- `unsigned n0, n1`: negative and positive row counts.
- `vector<double> auc`: one exact point area per procedure.
- `Matrix<double> cov`: paired DeLong covariance matrix.
- `bool ok` and `string reason`: estimator status and refusal.

The following functions are available:

- `Result analyze(label,pred)`: estimates paired DeLong covariance for procedures scored on the same independent rows.
- `Interval interval(const Result& r,unsigned i)`: returns the normal DeLong interval for procedure `i`, clamped to $[0, 1]$.
- `Contrast contrast(const Result& r,unsigned i,unsigned j)`: returns the paired area contrast $AUC_i - AUC_j$.

12.6.2 clustered_auc

Ordinary DeLong assumes rows are independent. That is false when several rows belong to one patient, site, family or other sampling unit: those rows may move together, so counting them as independent can make an interval or comparison too confident. Obuchowski extends DeLong's structural-component construction to clustered ROC data by centering and summing the positive and negative components within each cluster and adding the within-cluster cross-covariance term that the independent-row formula lacks.[5]

The point AUC remains the same row-level ranking probability. What changes is its uncertainty: the independent replication count is the number of informative clusters, not the number of rows. A cluster containing many rows supplies more pairwise information but does not become many independent experimental units. This is why the implementation requires at least two clusters carrying each outcome class and why a group-disjoint split and clustered inference solve different problems.

`clustered_auc::Result` contains:

- `bool ok` and `string reason`: estimator status and refusal.
- `unsigned nRows, nClusters`: row and independent-cluster counts.
- `unsigned clusters10, clusters01`: clusters carrying at least one positive or negative row; these are the covariance divisors' populations.
- `unsigned n0, n1`: negative and positive row counts.
- `vector<double> auc`: exact patient-row point areas.

[5] N. A. Obuchowski, "Nonparametric analysis of clustered ROC curve data," *Biometrics* 53(2), 567–578, 1997, https://doi.org/10.2307/2533958.

- `Matrix<double> cov`: Obuchowski clustered covariance matrix.
- `string method`: authoritative estimator name for reports.

The following functions are available:

- `Result analyze(label,cluster,pred)`: estimates Obuchowski's nonparametric clustered covariance for paired procedures.
- `Interval interval(const Result& r,unsigned i)`: returns the unclamped published normal interval for procedure `i`.
- `Contrast contrast(const Result& r,unsigned i,unsigned j)`: returns the clustered paired area contrast.

Example

```
delong::Result iid = delong::analyze(label, predictions);
clustered_auc::Result grouped =
    clustered_auc::analyze(label, cluster, predictions);
// The design policy decides which result may be reported.
```

Notes

DeLong requires at least two rows of each outcome class and assumes independent rows. Obuchowski requires at least two clusters carrying each outcome class. Clustering can increase or decrease the estimated standard error; its defining property is that the cluster is the independent unit. Out-of-fold predictions are not an independent test sample for either locked-test contrast.

12.7 Training-control services

12.7.1 PlateauDetector

`PlateauDetector` is neuron's project-defined early-stopping heuristic, not a statistical convergence test. It maintains two adjacent moving-average windows of the training error and compares their relative change. A flat or rising newer window records a strike; the configured number of consecutive strikes requests a stop. Windowing prevents one noisy iteration from deciding the run, while patience requires the lack of improvement to persist. The following methods are available:

- `PlateauDetector(unsigned window,double tol,unsigned patience)`: constructs a detector; window is at least two and patience at least one.
- `void reset()`: clears both windows and the strike count.

- `bool update(double error)`: adds one iteration error and returns `true` after the configured number of consecutive comparisons fail to achieve the required relative improvement.
- `unsigned strikes() const`: returns the current consecutive-strike count.

12.7.2 autoalgo

`autoalgo` is a project-defined bounded tournament among the three existing optimizers. Each optimizer starts from a clone of the same weights, runs for the same wall-clock allowance, and is ranked by the finite training error it reached; the winning clone, including the progress it already made, continues the real fit. It is an engineering choice under a time budget, not a claim that the selected optimizer is globally best.

`autoalgo::Probe` contains:

- `int algorithm` and `string name`: training-menu/API number (1 through 3) and display name.
- `double error`: final probe error.
- `unsigned iterations`: iterations completed within the probe.
- `bool usable`: whether the result may compete for selection.
- `Iterative::StopReason stop`: reason the probe ended.

`autoalgo::Result` contains:

- `int selected` and `string selectedName`: winning algorithm and display name.
- `vector<Probe> probes`: complete optimizer audit.
- `unique_ptr<Network> winner`: selected clone, including probe progress.
- `bool cancelled`: whether external cancellation ended selection.
- `Result pick(const Network& start,unsigned budgetMs, const atomic<bool>* cancel)`: probes canonical, CGD, and Shanno on clones of identical starting weights and returns the lowest-error usable clone.

12.7.3 LBFGSObjective and LBFGS

`LBFGS` is a *research-only* limited-memory BFGS optimizer. It is not selectable from the training menu, the GUI, the HTTP API or automatic selection, it is never written to a saved network, and it is not a public capability of the engine. It is reachable only

through `Network::setTrainingType(Network::TRAIN_LBFGS)`, a training type no menu produces, so that the optimizer benchmark and `tests/network/check_lbfgs.cpp` can measure it. This section documents what exists in the source; it does not announce a user-visible feature.

Published derivation. The limited-memory update is Nocedal (1980), *Mathematics of Computation* **35**(151):773–782, and Liu and Nocedal (1989), *Mathematical Programming* **45**:503–528. The two-loop recursion is Nocedal and Wright, *Numerical Optimization*, 2nd ed. (Springer, 2006), Algorithm 7.4, p. 178; the initial inverse-Hessian scaling $\gamma_k = (\mathbf{s}_{k-1}^T \mathbf{y}_{k-1})/(\mathbf{y}_{k-1}^T \mathbf{y}_{k-1})$ is equation (7.20) on the same page; the outer loop is Algorithm 7.5, p. 179. The line search is a *complete strong-Wolfe search*: Algorithm 3.5 (bracketing, p. 60) with Algorithm 3.6 (`zoom`, p. 61), enforcing sufficient decrease (3.6a) and the two-sided curvature bound (3.7b) of Wolfe (1969), *SIAM Review* **11**(2):226–235. Armijo backtracking is deliberately not implemented: it never tests curvature and therefore cannot guarantee $\mathbf{s}^T\mathbf{y} > 0$.

neuron-specific policy, distinguished from the mathematics. The published constants are $c_1 = 10^{-4}$ and $c_2 = 0.9$ (Nocedal and Wright pp. 33–34), a unit initial step for a quasi-Newton direction (p. 142) and $\min(1, 1/\|\mathbf{g}\|_1)$ for a steepest-descent one (p. 59). Declared by this project rather than by the sources: a bracketing expansion factor of 2, a bracketing ceiling $\alpha_{\max} = 10^{10}$, at most twenty evaluations per line search, bisection as `zoom`'s interpolation, refusal of a bracket narrower than $10^{-16}\max(1, |\alpha_{lo}|)$, and acceptance of a curvature pair only when $\mathbf{s}^T\mathbf{y} > 10^{-10}\,\mathbf{y}^T\mathbf{y}$ — a relative floor on top of the published $\mathbf{s}^T\mathbf{y} > 0$ so that $\rho = 1/(\mathbf{s}^T\mathbf{y})$ stays representable. A rejected pair is skipped, never damped. All are recorded in `docs/learning_research/lbfgs_source_decision.md`, which was written before the implementation and may not be revised after a measurement.

Ownership. `LBFGS` owns the memory length m, the $\mathbf{s}$, $\mathbf{y}$ and ρ ring buffers, the last accepted parameters, objective and raw gradient, the two-loop scratch, the line search and its counters, curvature acceptance and history reset, the direction, and trial installation, acceptance, failure and exact restoration. It does *not* own stopping, cancellation, the iteration counter or reporting: `Iterative` keeps all of those, and this class only asks whether cancellation has been requested. It does not touch `eta`, and it does not share `lastG`/`lastF` with conjugate gradient descent and Shanno. It does not pass through `Network::engine()`, which dispatches a gradient transformation that the caller then applies as $\mathbf{w} \leftarrow \mathbf{w} - \eta\mathbf{g}$; a search making several objective evaluations per iteration cannot honestly fit that contract.

`LBFGSObjective` is the boundary `LBFGS` consumes, with one implementation — `Network::Evaluator`, a local of one outer iteration that forwards to the protected packed boundary of section 12.2. It is not a general callback framework:

- `virtual void currentPoint(vector<double>& w) const`: the currently installed parameters, packed.
- `virtual void install(const vector<double>& w)`: installs packed parameters, for a trial point and for exact restoration.
- `virtual double evaluate(vector<double>& g)`: returns the objective at the installed parameters and writes the raw gradient.
- `virtual bool cancelled() const`: whether an outside request has asked the run to stop. Consulted before every trial evaluation, because one evaluation is a whole pass over the training set.

`LBFGS` provides:

- `void setMemory(unsigned m)`: sets the history length and resets the working state. Throws `LBFGS::Ineligible` when m is zero.
- `unsigned getMemory() const`: the configured history length.
- `void reset()`: clears history, accepted point, scratch and counters. Called from `Network::prepareRun()`, so working state is per `train()` run and a quiet run resets exactly as an audible one does.
- `bool started() const`: whether this run has evaluated its starting point.
- `double iterate(LBFGSObjective& obj)`: performs one outer iteration and returns the objective *at the point the step departed from*, matching the pre-update reporting every other optimizer here uses, so no stopping rule in `Iterative` sees a shifted timeline. On line-search failure or cancellation it reinstalls the last accepted parameters exactly, drops the history and reports the unchanged accepted objective; it invents no stop reason.
- `double gradMax() const`: $\max_i |g_i|$ of the *raw* gradient at that same point, taken before the direction is computed.
- `objectiveEvaluations()`, `gradientEvaluations()`, `lineSearchFailures()`, `historyResets()`, `curvatureRejections()`, `cancellations()`: counts, all `unsigned long long`, kept separate from outer iterations because a single iteration may cost many full passes.
- `unsigned pairs() const`: stored curvature pairs, 0 to m.
- `void applyInverseHessian(const vector<double>& g, vector<double>& result) const`: Algorithm 7.4, writing $H_k\mathbf{g}$.
- `double scaling() const`: γ_k of equation (7.20); 1 with an empty history.
- `bool pushPair(const vector<double>& s,const vector<double>& y)`: stores a curvature pair, returning `false` when it is skipped.

- `void resetHistory()`: drops the history, the fallback shared by a non-descent direction and a failed line search.
- `void copyConfigurationFrom(const LBFGS& rhs)`: copies the memory length only. A copied model does not promise to continue a mid-run trajectory, because `train()` begins a new run and resets the working history.

Failure contract. `LBFGS::Ineligible` is raised, rather than the method silently becoming a different one, for on-line training (a per-exemplar update has no deterministic objective to line search), the automatic step-size search (two step rules cannot own one iteration), a model that does not implement the packed boundary, and $m = 0$. A measured property worth recording: at the objective's floating-point floor no step can demonstrate sufficient decrease, so the search spends its evaluation ceiling and fails; the weights then do not move, the history is dropped, and a real run stops earlier on a rule `Iterative` owns.

Performance evidence. On Civic Choice at the committed practical endpoint, from identical starting weights, L-BFGS reached the same endpoint between 8.8× and 13.0× faster than Shanno — the incumbent — at 6,000, 25,000 and 100,000 rows and across four weight seeds, taking 14 to 20 full training-set traversals against Shanno's 123 to 210. The measurements, their spread and their limitations are in `docs/learning_research/lbfgs_screen_results.md`.

12.7.4 modelfactory and netclone

`modelfactory` and `netclone` are project-defined construction mechanisms. They keep knowledge of concrete model types in one cold-path boundary: callers describe or clone a model without duplicating a type switch, while fitting remains virtual behavior owned by the constructed model.

`modelfactory::Spec` is the interface-independent construction request:

- `bool logistic`: request binary logistic regression; when true, neural-layer fields do not select the concrete type.
- `bool bias`: include neural-network bias nodes.
- `vector<unsigned> hidden`: hidden-layer widths in forward order.
- `bool xentropy`: use cross-entropy rather than least-mean-square output error for a neural network.
- `bool logLastop`: write the most recent operation record.
- `bool logHistory`: append the session history.
- `unique_ptr<Model> build(const Spec& spec,DataSet& data, string& err)`: constructs and binds a fresh untrained model in the required order; returns null and fills `err` for an invalid request.

- `unique_ptr<Model> createByTypeName(string typeName,bool bias)`: creates the appropriate empty concrete type for a saved-network load; returns null for an unknown type name.
- `unique_ptr<Network> cloneNetwork(const Network& source)`: returns a deep copy of a known concrete network, including its dataset and training state; returns null for an unknown concrete type.

Example

```
PlateauDetector stop(100, 1e-4, 3);
if (stop.update(trainingError)) requestStop();

modelfactory::Spec spec;
spec.logistic = true;
unique_ptr<Model> model = modelfactory::build(spec, data, error);
```

Notes

A plateau is an eligible stopping rule; an iteration ceiling is not. Automatic optimizer probes are bounded experiments, not completed fits. Factory construction and cloning centralize type knowledge; training remains virtual behavior of the concrete model.

12.8 OBD architecture selection

Optimal Brain Damage originally uses a second-order approximation to predict the increase in training error caused by deleting a parameter, then removes parameters with low saliency to reduce an over-large network.[6] neuron applies that pruning idea at hidden-unit granularity and surrounds it with a project-defined validation search: it grows candidate hidden layers while held-out error improves, restores the best eligible size, then prunes by the engine's saliency calculation. Thus `obd::run` is not a verbatim implementation of the paper's entire procedure; the cited method supplies the saliency-based pruning principle, while the grow/validation/eligibility policy is neuron-specific and is specified below.

`obd::Eligibility` gives one authoritative admission decision:

- `ELIGIBLE`: a declared stopping rule ended a finite trial.
- `INCOMPLETE_CEILING`: the iteration safety limit ended an unfinished descent.
- `INCOMPLETE_CANCELLED`: external cancellation ended the trial.

[6] Y. LeCun, J. S. Denker and S. A. Solla, "Optimal Brain Damage," *Advances in Neural Information Processing Systems 2*, 598–605, 1990, https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html.

- `NUMERICAL_FAILURE`: the score or training error is non-finite.

`SizeTrial` is the audit row for one evaluated architecture:

- `unsigned hidden`: hidden-unit count.
- `double trainErr` and `testErr`: training error and minimum held-out monitor error at the saved validation minimum.
- `double trainCA` and `testCA`: corresponding classification accuracies in $[0, 1]$.
- `Iterative::StopReason stop`: engine reason the fit ended.
- `bool phaseGrow`: true for growth, false for pruning.
- `Eligibility eligibility`: whether this trial may be compared.
- `unsigned iterations`: work used by the trial.

`Config` contains:

- `hStart, hMax`: first and largest hidden-unit counts.
- `iterBudget, sampleEvery`: per-size ceiling and validation sampling cadence.
- `earlyStopTol, earlyStopPatience`: tolerated held-out reversal and consecutive samples required to stop a size.
- `growPatience, pruneTol`: growth and pruning decisions.
- `algorithm`: $0/1/2$ for a fixed optimizer or -1 for automatic selection.
- `plateauTol, plateauWindow`: training-error plateau backstop.

`Result` is the owning outcome of the complete search:

- `bool ok` and `string message`: success or precise refusal.
- `unsigned selectedHidden`: chosen hidden-unit count on success.
- `vector<SizeTrial> history`: every trial in execution order.
- `unique_ptr<Network> winner`: trained selected model; null on refusal.
- `bool cancelled`: external cancellation ended the search.
- `bool ceilingExhausted`: a required trial reached its safety limit, distinguished so the caller can recommend a larger budget.
- `int algorithm`: resolved optimizer, 0 through 2, or -1 if none ran.

- `bool autoSelected`: whether the optimizer tournament chose it.

The following functions are available:

- `Eligibility classify(StopReason stop,double score,double error)`: returns the authoritative trial-admission classification.
- `const char* stopToken(StopReason stop,Eligibility e)`: returns the machine-readable trial outcome.
- `Result run(DataSet& data,const Config& cfg,ProgressFn progress, const atomic<bool>* cancel)`: performs the grow-then-prune search and returns the owning selected network only after a valid search.

Example

```
obd::Config cfg;
cfg.hMax = 8;
cfg.algorithm = -1;                    // automatic optimizer probe
obd::Result r = obd::run(data, cfg, nullptr, nullptr);
if (r.ok) current = std::move(r.winner);
```

Notes

OBD requires one discrete output and a held-out monitoring set. Validation is preferred; the test set is only the two-way fallback. A needed trial that hits its ceiling is refused and never compared. In nested cross-validation the whole search, including optimizer choice, is repeated inside each fold.

12.9 Cross-validation objects

Cross-validation estimates out-of-sample behavior by partitioning the available rows into folds, fitting on all but one fold, predicting the held-out fold, and repeating until every row has exactly one out-of-fold prediction. Those pooled predictions describe the performance of the *procedure* that produced them, not a single model fitted once to all rows. When the procedure includes model selection, that selection must be repeated using only each outer fold's training rows; otherwise the reported error is optimistically biased. Nested cross-validation supplies the inner selection loop and leaves the outer fold for assessment.[7]

The objects below separate those roles: a caller supplies one complete fitting procedure, `crossval` owns repetition and out-of-fold assembly, and `cvadapters` defines procedures whose tuning steps are contained within the fold. This ownership rule is the executable form of the statistical requirement above.

[7] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics* 7, 91, 2006, https://doi.org/10.1186/1471-2105-7-91.

`crossval` owns repetition over an immutable fold assignment. `Metrics` contains:

- `unsigned n`: rows contributing to the calculation.
- `double az`: fitted binormal AUC.
- `double trap`: exact empirical AUC.
- `double sens, spec, ca`: sensitivity, specificity, and classification accuracy at the selected threshold.

`FoldResult` contains:

- `unsigned fold, n`: fold number and held-out row count.
- `bool ok` and `string reason`: fit status and refusal.
- `Metrics metrics`: the fold's held-out statistics.

`RunResult` contains:

- `bool ok, cancelled` and `string message`: run status.
- `vector<FoldResult> folds`: one audit record per fold.
- `vector<double> prediction, outcome`: row-aligned out-of-fold predictions and outcomes.
- `unsigned validFolds, pooledN`: successful folds and pooled rows.
- `double pooledAz, pooledTrap`: pooled OOF binormal and exact AUCs.

`ProcResult` is the adapter's answer for one fold:

- `bool ok`: true only when the adapter produced one prediction for every held-out row.
- `bool cancelled`: distinguishes cooperative cancellation from a fitting refusal.
- `vector<double> pred`: held-out predictions in test-row order; empty on failure, never padded with fabricated values.
- `string reason`: explanation when `ok` is false.

`Procedure` is the callback type that turns one already materialized fold into a `ProcResult`. Its arguments are the fold's `DataSet`, the corresponding raw-row training and test identities, and an optional cancellation latch. The indices let a nested procedure make an inner split without seeing the outer held-out rows; a plain model may ignore them.

`Progress` is a snapshot of the repetition coordinator:

- `Stage stage`: `CROSS_VALIDATION` while folds repeat or `LOCKED_EVALUATION` during the final once-only refit.
- `string procedure`: name of the procedure currently fitting.
- `procIndex` and `procCount`: one-based current position and total.
- `completedProcedures`: procedures finished in this stage.
- `fold` and `k`: one-based current outer fold and total; both are zero during locked evaluation because no folding occurs there.
- `completedFolds`: folds finished within the current procedure.

`ProgressFn` is an optional callback receiving these snapshots on the thread running the comparison. A callback that writes shared GUI state supplies its own synchronization; a null callback costs only the branch at fold boundaries.

`FoldSelection` records what a procedure selected inside one fold:

- `unsigned hidden`: selected hidden-unit count.
- `int algorithm`: resolved optimizer number, 0 through 2.
- `bool autoSelected`: whether an automatic probe chose it.
- equality operators compare the complete record for reproducibility tests.

`ProcedureSpec` names and wires a procedure:

- `string name`: stable display and deterministic-substream identity.
- `Procedure proc`: complete fit-and-score callback.
- `vector<FoldSelection>* selections`: optional non-owning audit sink shared with a nested adapter; null for procedures that select nothing.

`Comparison` is the paired result over one immutable fold plan:

- `bool ok`, `bool cancelled`, and `string message`: overall status, cancellation distinction, and refusal explanation.
- `outcome` and `foldId`: row-aligned truth and shared fold identity.
- `unsigned k`: number of folds.
- each `Entry` contains a procedure `name`, its `RunResult`, elapsed `seconds`, and per-fold `selections`; `entries` preserves request order.

`LockedEntry` is one procedure's once-only locked-test result:

- `name`, `ok`, and `reason`: identity and fit status.

- `pred`: predictions in the common locked-test row order; empty on refusal.
- `seconds`: elapsed fit-and-score time.
- `selections`: architecture/optimizer choices made by that frozen refit.

`LockedResult` keeps all procedures paired on the untouched test:

- `ok`, `cancelled`, and `message`: overall status.
- `testRows`: original raw-row identities of the locked sample.
- `outcome`: aligned binary outcomes.
- `entries`: one `LockedEntry` per procedure in request order.

The following functions are available:

- `Metrics metricsFor(const vector<unsigned>& outcome,const vector<double>& pred,const vector<unsigned>& rows)`: calculates held-out classification and ROC metrics for the named rows; unavailable metrics are -1.
- `RunResult run(DataSet& data,const vector<unsigned>& foldId, Procedure proc,const atomic<bool>* cancel=nullptr,bool substreams=false, unsigned seed=0,ProgressFn progress=nullptr)`: runs one procedure over every fold and scatters successful predictions back to their original row positions. It refuses a malformed plan and records a failed fold rather than manufacturing its missing predictions.
- `Comparison compare(DataSet& data,const vector<unsigned>& foldId, const vector<ProcedureSpec>& procs,const atomic<bool>* cancel=nullptr, bool substreams=false,unsigned seed=0,ProgressFn progress=nullptr)`: runs several named procedures over one immutable fold plan and returns paired OOF results.
- `LockedResult evaluateOnce(DataSet& data,const vector<unsigned>& trainRows,const vector<unsigned>& testRows,const vector<ProcedureSpec>& procs,const atomic<bool>* cancel=nullptr,bool substreams=false,unsigned seed=0,ProgressFn progress=nullptr)`: refits each procedure once on the development rows and scores the common untouched locked test. No cross-validation metric is reused as a locked-test result.

12.9.1 cvadapters

`crossval` deliberately knows no model family: its `Procedure` callback says only "fit on these rows and return predictions for those rows." Concrete model APIs do not naturally have that signature. `cvadapters` is the narrow translation layer between the two. It captures the construction and fitting rule for a Network, DFA, or nested OBD search and returns a callback the generic runner can repeat. The adapters are used when a CV or locked-test comparison is assembled; they are not used by ordinary one-model training, and they neither choose folds nor compute metrics or reports. Keeping the translation here prevents the runner from acquiring a model-type switch and, for nested OBD, keeps architecture selection inside the outer training rows where it cannot leak.

`InnerSplit` is the explicit result of that nested adapter's inner train/validation decision:

- `bool ok`: true only when both inner sets satisfy the requested design.
- `string reason`: refusal explanation when no usable split exists.
- `vector<unsigned> train`: raw-row identities used to fit candidates.
- `vector<unsigned> validation`: disjoint raw-row identities used to select and early-stop them.

The following functions create procedure callbacks or the inner split they need:

- `Procedure trainProcedure(const Network& templateNet, unsigned maxIter)`: clones and trains a configured network from fresh weights in each fold. The template is captured by value through a deep clone; `maxIter` is the per-fold safety ceiling. A nonconverged or cancelled fit returns a failed `ProcResult`.
- `Procedure dfaProcedure(bool quadratic)`: returns an LDFA adapter when false and QDFA adapter when true. Each invocation fits only the fold training matrix and returns the concrete DFA's graded held-out score.
- `Procedure nestedObdProcedure(const obd::Config& cfg, double innerValFraction,vector<FoldSelection>* selections=nullptr, obd::ProgressFn progress=nullptr,const vector<unsigned>& rowGroup={})`: performs the complete OBD architecture search within each outer training fold and returns only outer held-out predictions. The optional sinks expose selection and inner-search progress without teaching `crossval` about OBD. When group identities are supplied, the inner split is group-disjoint or the fold fails.
- `InnerSplit innerValidationSplit(const vector<unsigned>& trainRows, const vector<unsigned>& rowLabel,const vector<unsigned>& rowGroup, double fraction)`: returns a stratified inner split, group-disjoint when `rowGroup` is supplied; returns `ok=false` rather than falling back when the grouped split is infeasible.

Example

```
crossval::ProcedureSpec logistic{
    "Logistic",
    cvadapters::trainProcedure(templateNet, 20000), nullptr};
crossval::Comparison c = crossval::compare(
    data, foldPlan.foldId, {logistic}, nullptr, true, 42);
```

Notes

A failed fold contributes no fabricated prediction and remains visibly missing. All procedures share the same fold plan. Deterministic substreams key fits by seed, procedure name, and fold, so comparison membership and order cannot change a procedure's fit. Nested OBD's inner validation is group-disjoint under a grouped design; infeasibility is a failure, never a row-wise fallback.

12.10 Cross-validation reporting: cvreport

`cvreport` is neuron's project-defined presentation and artifact layer. It performs no fitting, fold assignment or covariance estimation. It receives already computed results and renders three levels of detail: a compact summary, an audit of every repetition and fold, and machine-readable files retaining raw-row identity. Keeping those jobs here ensures that the screen report, CSV/JSON artifacts and REST response describe the same run rather than recomputing or reinterpreting it independently.

`PlanInfo` supplies descriptive context not owned by a `Comparison`:

- `string dataset`: optional label printed in the report header.
- `unsigned n` and `events`: whole-dataset row and outcome-1 counts; these are not development-fold counts.
- `evaldesign::Partition folds`: achieved typed fold design from which every displayed plan sentence is derived.
- `string primary` and `reference`: prespecified names for the headline contrast.
- `vector<unsigned> cluster`: optional cluster identity aligned to CV rows; empty means ungrouped.
- `vector<unsigned> rawRow`: optional original-row identity aligned to CV rows; empty means the CV index already is the raw identity.
- `unsigned rawRowCount`: source row count used to range-check a supplied identity map; zero means the bound is unknown.

Its methods are:

- `string foldPlanText() const`: derives the fold-plan heading from the typed partition.
- `string rawRowError(unsigned n) const`: returns an empty string for a valid absent/identity map or for exactly `n` distinct in-range raw-row identifiers; otherwise returns the defect.

`LockedColumn` contains:

- `string name`: procedure display name.
- `bool hasAuc` and `double auc`: point-area availability and value.
- `bool hasInterval` and `double lo, hi`: interval availability and limits.
- `string architecture`: selected architecture description.
- `string note`: refusal or qualification shown with the result.
- `vector<double> prediction`: predictions paired to locked rows.

`LockedContrast` contains:

- `bool hasPoint, hasInference`: availability of the AUC difference and of its inferential comparison.
- `string primary, reference`: contrasted procedure names.
- `double delta, p`: AUC difference and two-sided p-value.
- `bool significant`: whether the declared comparison crosses its significance rule.
- `bool degenerate, separated`: zero-variance case indicators.
- `string note`: refusal or qualification.

`LockedInfo` contains:

- `evaldesign::Partition split`: achieved locked-holdout design.
- `SamplingUnit samplingUnit`: declared independent unit.
- `AucInference inference`: covariance estimator permitted by the design.
- `string inferenceReason`: explanation when inference is withheld.
- `vector<unsigned> rawRow`: original locked-row identities.
- `vector<double> outcome`: locked outcomes.
- `vector<unsigned> cluster`: cluster identity for each locked row.

- `unsigned nClusters`: distinct clusters present in the locked sample.
- `vector<LockedColumn> columns`: procedure results.
- `LockedContrast contrast`: prespecified paired comparison.

`LockedInfo` derives every displayed design string from typed state. Its `clusterError()` method validates cluster identity before clustered inference or artifact writing.

`ArtifactResult` reports one attempted Tier-3 write:

- `string name`: artifact filename.
- `string path`: full destination attempted.
- `bool ok`: true only after open, write, flush, and close succeed.
- `string error`: I/O failure explanation when `ok` is false.

The following functions are available:

- `string tier1(cmp,info,locked)`: returns the one-screen headline table and verdict.
- `string tier2(cmp,info,locked)`: returns per-fold, selection, timing, design, and locked-test detail.
- `vector<ArtifactResult> writeArtifacts(cmp,info,dir,locked)`: writes `cv_predictions.csv`, `cv_metrics.csv`, `cv_run.json`, and, when applicable, `cv_locked_predictions.csv`; returns one status per attempted file.
- `void render(ostream& out,cmp,info,dir,locked)`: writes Tier 2, then Tier 1, then the machine-readable artifacts.

Example

```
cvreport::PlanInfo info;
info.folds = achievedFoldDesign;
cvreport::LockedInfo locked;
locked.split = achievedLockedDesign;
cvreport::render(cout, comparison, info, runDirectory, locked);
```

Notes

The reporter chooses neither partitions nor covariance estimators. Every design sentence is derived from typed state. Original raw-row and cluster identifiers are validated before artifacts are written so development and locked prediction files remain joinable. Tier 1 is the conclusion, Tier 2 is descriptive detail, and Tier 3 is the machine-readable audit substrate.

12.10.1 AsyncJob

`AsyncJob` is neuron's project-defined owner of the lifecycle every long GUI operation shares: training, hidden-layer sizing, cross-validation and stepwise regression. It answers one question — how a request hands work to a background thread and how the result comes back — and it answers it once. Before it existed the same launch sequence appeared at four call sites, character for character, differing only in the payload each thread carried.

Construction, destruction and lifecycle

`FailureRenderer` is the public callable type `function<string(const string&)>`. The explicit constructor takes one such renderer, which dresses a boundary failure in the caller's wire format; the job therefore owns the failure fact and wording without learning JSON.

The destructor calls `joinForShutdown()`. A finished `std::thread` remains joinable until it is joined, and destroying a joinable thread invokes `std::terminate`; this is required by the C++ thread contract.[8] Thus destruction requests cancellation and waits rather than making the containing object's lifetime a process-ending event.

- `bool start(function<string()> body)`: reaps the previous worker, clears every per-run progress field, clears the stop latch, marks the job running, and starts `body` on a thread. Returns false only when no worker could be started, having first published a failure and released the job, so a failed launch cannot leave the engine owned forever.
- `bool requestStop()`: sets the cancellation latch, or returns false when nothing is running. Cancellation is cooperative: the engine polls the latch and finishes the current iteration.
- `void clearStopRequest()`: clears the latch without starting anything, for a blocking operation that shares its implementation with the asynchronous form and would otherwise inherit an earlier run's Stop.
- `void joinForShutdown()`: cancels and joins a worker that outlived the server loop.
- `bool isRunning() const`: reports whether the job currently owns the engine. The flag is atomic so the busy gate can read it without the progress mutex.
- `bool isStopRequested() const`: reports the cooperative cancellation latch's current value.

[8] ISO C++ working draft, "Destructor [thread.thread.destr]," https://eel.is/c++draft/thread.thread.destr.

- `atomic<bool>* cancelLatch()`: returns the address of the one authoritative latch for an engine routine that polls it directly. The pointer is valid only while its owning `AsyncJob` is alive.
- `void clearCvProgress()`: clears every cross-validation and nested search progress field. Its caller must hold `progressMutex`.
- `void resetForNewRun()`: clears the error series, result and all operation-specific progress before a new launch. Its caller must hold `progressMutex`; `start()` supplies that lock.
- `void pushSample(unsigned iteration,double trainError,double testError)`: appends one training-progress point and performs bounded decimation. It takes `progressMutex` itself, so a caller must not already hold that mutex.

Public progress state

All fields below are public because the GUI status renderer reads or fills the typed state directly; `AsyncJob` deliberately does not render an HTTP response. Except for the two atomic latches accessed through methods, readers and writers hold `progressMutex` across every group of fields that must describe one instant.

- `static const unsigned MAX_POINTS = 2000`: maximum retained chart points before decimation halves the stored series and doubles the future sampling stride.
- `iters`, `trainErr`, `testErr`, `keepEvery`, and `sampleCounter`: the aligned, bounded error series and its decimation state. A negative test error means the held-out error was not sampled at that iteration.
- `string result`: the completed operation's response body; empty while the current operation is running. It is published before the running flag is cleared.
- `obdPhase` and `obdHidden`: hidden-layer search phase and current hidden-unit count; an empty phase means the job is not OBD.
- `stepwise`: the current stepwise-progress object body; empty means the job is not stepwise. It is separate from OBD state so a client never has to reinterpret a hidden-unit field as a candidate variable.
- `cvStage`, `cvProcedure`, procedure and fold counters, plus `cvInnerPhase` and `cvInnerHidden`: outer cross-validation/locked-test progress and the nested architecture search within the current fold. The inner fields are cleared when the outer fold changes so one fold's last trial cannot appear under the next.

The ordering contract

The worker stores its result under the progress mutex and clears the running flag only after releasing it. A reader must therefore hold that mutex across *both* reads. Its critical section then totally orders against the worker's publication: either entirely before it, and the job still reads as running, or entirely after it, and the result is visible. Reading one of the two outside the lock allows a finished run to be reported as an idle one with nothing to show. The constraint binds the caller, which is why it is stated on the class rather than in the status handler that happens to satisfy it.

The lock order is the other half. The mutex is never held across a join, a thread construction, or an engine call. The join in particular precedes taking it: a departing worker's last act is to take that same mutex to publish, so joining while holding it would deadlock.

The worker boundary

An exception escaping a thread's function calls `std::terminate`, which would end the server while the page was still polling for status. Every escape becomes a published failure instead — a `std::exception` carrying its own message, anything else a generic one — and the publish-then-clear order holds on the throwing paths too. How a failure message becomes a response body is supplied by the caller, so the class speaks no wire format. The language rule is general: when no matching exception handler is found, `std::terminate` is invoked.[9]

Example

```
AsyncJob job([](const string& message) {
    return jsonMsg(false, message);
});
if (!job.start([] { return runObdJob(config); }))
    return jsonMsg(false, "could not start a worker thread");
```

Notes

Bodies capture request values, not request objects. A raw pointer may be captured only when the GUI busy gate guarantees its pointee cannot be replaced for the run's duration. No engine operation, join or thread construction occurs while `progressMutex` is held.

12.10.2 procguard

`procguard` is neuron's project-defined process boundary. `int procguard::run(const function<int()>& body,ostream& err,const char* program)` is the command-line program's top-level exception boundary:

[9] ISO C++ working draft, "Handling an exception [except.handle]," https://eel.is/c++draft/except.handle.

it runs the body and returns its exit status, and converts anything that reaches the top into a named diagnostic and status 1. Before it existed a contract failure ended the program with no message at all.

It is a separate unit from `AsyncJob` deliberately. Both are boundaries around an uncaught exception, but the command-line entry point has no worker, no cancellation and no result to publish — it has an exit status and a diagnostic, and filing that under asynchronous-job behaviour would misdescribe it. Taking the body and the stream as arguments is what makes it testable: a test supplies a throwing body and an in-memory stream, and the shipped program gains no way to inject a fault.

Example

```
return procguard::run(
    [argc, argv] { return neuronMain(argc, argv); }, cerr, "neuron");
```

Notes

A normally returning body keeps its own status. A `std::exception` produces `<program>: fatal: <what>` and status 1; an exception of any other type produces `<program>: fatal: unrecognized error` and status 1.

12.11 Strict field parsing: util

12.11.1 The problem these solve

A value that arrives as text — from an HTTP request, a configuration line, a file — has to become a number before anything can be computed with it, and the conversion can fail in ways the caller must be told about. The C library functions this project used for twenty years, `atol` and `atof`, cannot tell anybody: each consumes as much of its argument as it understands, stops, and returns a value. There is no error channel at all, so `"5junk"` is indistinguishable from `"5"`, `"abc"` is indistinguishable from `"0"`, and a count wider than the destination type is converted to whatever the truncation produces.

That is a documentation problem as much as a correctness one: a caller cannot be told which of twenty parameters was wrong if the program never learned that any of them was. These six project-defined functions give the conversion an error channel and give the caller enough structure to name the offending field. They are in `util` because they are the lowest layer that both the HTTP boundary and any future caller already depend on, and they know nothing about HTTP, JSON, or which endpoint asked.

12.11.2 The status

`util::ParseStatus` is a scoped enumeration with seven values, ordered from "nothing was there" to "the text cannot be represented": `Ok`; `Empty`, nothing

but surrounding whitespace; `Syntax`, not a value of that kind at all; `Trailing`, a valid value followed by something else; `Negative`, a negative value where the contract has none; `Range`, representable as text but not as the destination type; and `NotFinite`, a floating value of `nan` or $\pm\infty$.

`Negative` is separated from `Syntax`, and `NotFinite` from `Range`, for one reason: the resulting message can say what is actually wrong. "`test_n` cannot be negative" and "`gradmax` is not a finite number" are both more useful than a single "invalid value", and the caller does not have to re-examine the text to produce them.

12.11.3 The parsers

- `ParseStatus parseUnsigned(const string& text,unsigned& out)`: parses a base-10 nonnegative whole number, requiring full consumption and a magnitude no greater than `UINT_MAX`. It returns a specific status and leaves `out` untouched on every refusal.
- `ParseStatus parseDouble(const string& text,double& out)`: parses a finite decimal or scientific-notation value, requiring full consumption. Overflow and non-finite spellings are refused; representable underflow is accepted. `out` changes only on `Ok`.
- `ParseStatus parseBool(const string& text,bool& out)`: accepts exactly `"1"` or `"0"`, after trimming boundary whitespace, and writes the corresponding value only on `Ok`.

Each writes `out` *only* when it returns `Ok`, so a caller that forgets to check the status still cannot be handed a half-converted value, and a caller that does check can leave its destination holding a default.

Three rules govern all three, and each was settled against measured behaviour rather than chosen:

Whitespace. Leading and trailing spaces and tabs are removed and the remainder must be consumed entirely. This is exactly what the C conversions already tolerated — they skip leading whitespace and stop at trailing whitespace — so no input that worked before is refused, while `"5junk"` and `"5 5"` both become `Trailing`. Interior whitespace is never permitted.

Sign and width. `parseUnsigned` decides the sign before any conversion, because `strtoull` would otherwise fold a leading minus into a wrap-around of its own — the behaviour being replaced. A magnitude above `UINT_MAX` is `Range`.

Finiteness. `parseDouble` refuses every spelling of `nan` and infinity that `strtod` would otherwise accept. Overflow to `HUGE_VAL` is `Range`; *underflow* is accepted and yields the closest representable value, which may be zero, because

a legitimately tiny tolerance must not be refused as a syntax fault and every domain that excludes zero rejects it one line later in the caller.

`parseBool` accepts exactly `"1"` and `"0"`, case-sensitively. That token set is not a preference: it is what the graphical client sends and what the operational documentation specifies, and widening it would silently change the meaning of requests that are accepted today.

12.11.4 The messages

- `string unsignedError(const string& field,const string& text, ParseStatus status)`: turns an unsigned-parser refusal into a field-specific message such as "is not a whole number," "cannot be negative," or "is out of range." Call it only with a non-`Ok` status.
- `string numberError(const string& field,const string& text, ParseStatus status)`: renders floating syntax, trailing text, range, and finiteness failures while preserving the original field and token.
- `string boolError(const string& field,const string& text, ParseStatus status)`: states the accepted `1/0` vocabulary and quotes the refused token.

Three functions rather than one because the expectation differs by destination: the same `Trailing` status means "is not a whole number" for a count and "is not a number" for a quantity. Each names the field first and quotes the text it was given, so a response never has to be assembled by its caller and a value that is empty, or is nothing but spaces, is still visible in the message.

Example

```
unsigned folds = 5;                          // the endpoint's own default
string text = param(request, "folds");
util::ParseStatus st = util::parseUnsigned(text, folds);
if (st != util::ParseStatus::Ok)
    return refuse(util::unsignedError("folds", text, st));
if (folds < 2)                               // the DOMAIN stays at the call site
    return refuse("folds must be at least 2");
```

produces, for `folds=5junk`, `folds:  '5junk' is not a whole number`, and leaves `folds` holding 5.

Notes

The division of labour is deliberate and is the reason these are not a validation framework. `util` owns *syntax*: what a whole number, a number and a flag look like, and what is wrong with the text when they do not. The caller owns the *domain*: which minimum, which interval, which combination of fields is a conflict. A syntax fault and a domain fault therefore read differently, which

is the distinction the previous code lost when an unreadable bin count was reported as a bin count that was too small.

These functions never call `setlocale`, and neither does any other part of the program, so `strtod`'s decimal separator is a period on every platform. That is recorded rather than assumed: introducing a locale change elsewhere would alter how every field is read.

The HTTP boundary's use of them, the omission and present-but-empty rules it adds, and the accepted tokens for every endpoint are in Section 4.1.

12.12 Function definitions

For clarity and ease of use, function definitions are placed in the file "function_defs.h". The implementation of function definitions in this file may take a number of forms, depending on how the function is to be used in the code. The reader is strongly encouraged to study "function_defs.h" to see the various forms functions may take.

12.12.1 In-line functions

The simplest form is the in-line function. An example is that which calculates the least mean squared error for a single output node:

```
// LMS error for a single output, takes known value y and model's
//    guess o as arguments, returns LMS error
inline double singleLMSerror( double y, double o )
{
     return 0.5 * ( y - o ) * ( y - o );
}
```

12.12.2 Function classes

For functions that are to be passed to *vector* or *Matrix* objects using the **func** method, classes must be constructed. An example is that which implements the sigmoidal transfer function:

```
// The sigmoidal function
class sigmoidal : public unary_function< double, double >
{
     public:
          inline double operator()( const double& x )
          {
               return 1.0 / ( 1.0 + exp( -x ) );
          }
};
```

12.12.3 The errorFunction object

A very important object defined in **function_defs.h** is *errorFunction*, which handles the calculation of model error. As neural network sigmoidal output is defined as

$$\vec{\mathbf{o}} = \frac{1}{1 + e^{-\vec{\mathbf{x}}}}$$

calculation of the error function may depend on $\vec{\mathbf{x}}$ as well as $\vec{\mathbf{o}}$ (see the **special note about numerical precision and the cross-entropy error** below), and thus *errorFunction* methods may be passed $\vec{\mathbf{x}}$ as well as $\vec{\mathbf{o}}$. *errorFunction* consists of

- *Public Methods*
- errorFunction(double y, double o, double x, bool $errorType$): Constructor for a *single* output, which takes known value y, model output o, argument x in sigmoidal transfer function $1/(1 + e^{-x})$ and bool flag $errorType$, where
  - $errorType = 0$ denotes the least mean squared error

    $$E = \frac{1}{2}(y_i - o_i)^2$$

  - $errorType = 1$ denotes the cross-entropy error

    $$E = -y \log(o) - (1 - y) \log(1 - o)$$

- errorFunction(vector< double > $\vec{\mathbf{y}}$, vector< double > $\vec{\mathbf{o}}$, vector< double > $\vec{\mathbf{x}}$, bool $errorType$): Constructor for *multiple* outputs, which takes known values $\vec{\mathbf{y}}$, model outputs $\vec{\mathbf{o}}$, arguments $\vec{\mathbf{x}}$ in sigmoidal transfer function $1/(1 + e^{-\vec{\mathbf{x}}})$ and bool flag $errorType$, where
  - $errorType = 0$ denotes the least mean squared error for m output nodes

    $$E = \frac{1}{2} \sum_{i=1}^{m} (y_i - o_i)^2$$

  - $errorType = 1$ denotes the cross-entropy error for m output nodes

    $$E = \sum_{i=1}^{m} -y \log(o) - (1 - y) \log(1 - o)$$

- double value(): Accessor returns numerical error result
- bool boundsErr() Accessor returns true if numerical bounds error (e.g. log(0)) (see the **special note about numerical precision and the cross-entropy error** below)

Example

```
// Calculate x-entropy error for single output and add to set error
//    which is previously defined as double setError (double y, double
o
//    and double x have also previously been calculated)
errorFunction E( y, o, x, 1 );
setError += E.value();

if ( E.boundsErr() ) // check for out of bounds error in log(o)
    cout << "ERROR: Numerical bounds error" << endl;
```

A special note about numerical precision and the cross-entropy error

As the cross-entropy error function is defined for network output o and target value y:

$$E = -y \log(o) - (1 - y) \log(1 - o)$$

And for the sigmoidal transfer function,

$$o = \frac{1}{1 + e^{-x}}$$

Then for large positive ($>> 0$) and large negative ($<< 0$) x, $o \to 1$, and $o \to 0$ respectively, and thus $log(1 - o) \to \infty$ and $log(o) \to \infty$. In these cases, the following transformations are made (which require those methods to be passed the sigmoidal transfer function argument x for the cross-entropy error.)

Large negative x

If $x << 0$,

$$o = \frac{1}{1 + e^{-x}} \approx \frac{1}{e^{-x}} = e^{x}$$

and thus

$$\log(o) = \log(e^{x}) = x$$

since $o \to 0$,

$$log(1 - o) \to log(1) = 0$$

thus

$$E = -y \log(o) - (1 - y) \log(1 - o) \approx -yx - 0 = -yx$$

Large positive x

$$1 - o = 1 - \frac{1}{1 + e^{-x}} = \frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}}$$

If $x >> 0$, since $1 + e^{-x} \rightarrow 1$

$$1 - o = \frac{e^{-x}}{1 + e^{-x}} \approx e^{-x}$$

thus

$$\log(1 - o) \approx log(e^{-x}) = -x$$

if $x >> 0$, $o \rightarrow 1$ and $-y \log(o) \rightarrow 0$, thus

$$E = -y \log(o) - (1 - y) \log(1 - o) \approx 0 - (1 - y)(-x) = (1 - y)x$$

12.13 What you really need to know about DataSet and TwoSet

All models use training and test sets. The model builds itself on the training set, and is tested on the test set. *DataSet* is designed for generating and manipulating training and test sets, and *TwoSet* is a special object that is designed to calculate accuracies for one-output models *only*.

All training and test sets are expressed by Matrices in the following format, where i is an input to the model, and o is an output:

$$\begin{array}{cccccccc} i_{11} & i_{12} & \dots & i_{1I} & o_{11} & o_{12} & \dots & o_{1O} \\ i_{21} & i_{22} & \dots & i_{2I} & o_{21} & o_{22} & \dots & o_{2O} \\ i_{31} & i_{32} & \dots & i_{3I} & o_{31} & o_{32} & \dots & o_{3O} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ i_{N1} & i_{N2} & \dots & i_{NI} & o_{N1} & o_{N2} & \dots & o_{NO} \end{array}$$

for N examples ("exemplars") in the set, I inputs and O outputs. Although there are many highly useful features in *DataSet*, a programmer building a *Model* need only know how to access the training and test Matrices, whether they've been loaded into the *DataSet* object, how many inputs and outputs there are, whether the output is discrete (held to 0 or 1), and the number of examples in each set. These methods are shown in red in the *DataSet* specification.

In addition, the programmer of a *Model* needs only to understand that the methods setTrainTwoSet and setTestTwoSet must be called in order for the method metricsReport, which reports accuracy metrics of a one-output *Model*, to be called. These methods are also shown in red in the *DataSet* specification.

12.14 DataSet

DataSet is a standalone object that contains and manipulates the dataset that *Model* will access. All *DataSets* have raw, training and test datasets, and know the number of inputs and outputs. *DataSet* consists of:

- *Public* methods
  - Raw dataset methods:
    - void loadRaw(string& $filename$): Takes $filename$ as argument (string), and loads a file into the *Raw* dataset Matrix, returns nothing
    - bool rawLoaded(): Returns a flag to indicate if *Raw* dataset is loaded
    - Matrix< double >& getRawMatrix(): returns the *Raw* dataset
    - void setRawMatrix(Matrix< double >& $inMatrix$): loads Matrix $inMatrix$ into the *Raw* dataset Matrix
  - Training set methods:
    - void loadTrain(string& $filename$): Takes $filename$ as argument (string), and loads a file into the $TrainSetData$ Matrix, returns nothing
    - bool trainLoaded(): Returns a flag to indicate if $TrainSetData$ Matrix is loaded
    - bool saveTrain(string& $filename$): Takes $filename$ as argument (string), and saves the $TrainSetData$ Matrix into a file, returns flag indicating if save operation was successful
    - bool saveScales(string& $filename$): Takes $filename$ as argument (string), and saves the scaling factors for the training set into the file, returns flag indicating if save operation was successful
      Note: see equation 12.2 below for an explanation of scaling. Saved to the file will be *lbound* (1 value), and for each x, S and min.
      $$x_{norm} = lbound + S(x - min) \tag{12.1}$$
      where x_{norm} is the normalized result, *lbound* is the lower limit of normalized values, S is the scaling factor, and min is the minimum value of each x in the dataset.
    - Matrix< double >& getTrainMatrix(): returns the $TrainSetData$ Matrix
    - void setTrainMatrix(Matrix< double >& $inMatrix$): loads Matrix $inMatrix$ into the $TrainSetData$ Matrix
  - *TwoSet* accessors for the training set:

    - bool setTrainTwoSet(): Constructs an internal *TwoSet* object from the training set, populating the 'real' column of the *TwoSet* object with the output column from the training set *Matrix*, returns true if successful
    - TwoSet& getTrainTwoSet(): Returns the constructed *TwoSet* object from the training set
    - bool TrainTwoSetLoaded(): Returns true if the internal training *TwoSet* object was successfully constructed
    - bool saveTrainTwoSet(string& $filename$): Takes $filename$ as argument (string), and saves the *TwoSet* object from the training set into a file, returns flag indicating if save operation was successful
- Test set methods:
    - void loadTest(string& $filename$): Takes $filename$ as argument (string), and loads a file into the $TestSetData$ Matrix, returns nothing
    - bool testLoaded(): Returns a flag to indicate if $TestSetData$ Matrix is loaded
    - bool saveTest(string& $filename$): Takes $filename$ as argument (string), and saves the $TestSetData$ Matrix into a file, returns flag indicating if save operation was successful
    - Matrix< double >& getTestMatrix(): returns the $TestSetData$ Matrix
    - void setTestMatrix(Matrix< double >& $inMatrix$): loads Matrix $inMatrix$ into the $TestSetData$ Matrix
- *TwoSet* accessors for the test set:
    - bool setTestTwoSet(): Constructs an internal *TwoSet* object from the test set, populating the 'real' column of the *TwoSet* object with the output column from the test set *Matrix*, returns true if successful
    - TwoSet& getTestTwoSet(): Returns the constructed *TwoSet* object from the test set
    - bool TestTwoSetLoaded(): Returns true if the internal test *TwoSet* object was successfully constructed
    - bool saveTestTwoSet(string& $filename$): Takes $filename$ as argument (string), and saves the *TwoSet* object from the test set into a file, returns flag indicating if save operation was successful
- General methods:
    - void setInput(unsigned I): Set the number of input nodes to I
    - unsigned getInput(): Returns number of input nodes
    - void setOutput(unsigned O): Set the number of output nodes to O

* unsigned getOutput(): Returns number of output nodes
* unsigned getNumTrain(): Returns the number of examples in $TrainSetData$, the training set
* unsigned getNumTest(): Returns the number of examples in $TestSetData$, the test set
* void removeInputs(vector< unsigned >& v): Given a vector representing input positions within a dataset, removes the corresponding columns in Raw, $TrainSetData$, and $TestSetData$, thus effectively removing those inputs from a dataset

Note: In the following methods, a variate x is normalized according to:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)] \qquad (12.2)$$

where x_{norm} is the normalized value, $lbound$ is the lower limit of the normalized result, $hbound$ is the upper limit of the normalized result, min is the minimum value of x in the dataset, and max is the maximum value of x in the dataset.

– Methods related to normalizing dataset variates
  * void setInUpper(double x): Sets the upper limit of normalized input variates to x
  * double getInUpper(): Returns the upper limit of normalized input variates
  * void setInLower(double x): Sets the lower limit of normalized input variates to x
  * double getInLower(): Returns the lower limit of normalized input variates
  * void setOutUpper(double x): Sets the upper limit of normalized output variates to x
  * double getOutUpper(): Returns the upper limit of normalized output variates
  * void setOutLower(double x): Sets the lower limit of normalized output variates to x
  * double getOutLower(): Returns the lower limit of normalized output variates

– Methods related to identifying output as discrete (held to 0 or 1)
  * void setDiscrete(bool $flag$): Sets output to be discrete if $flag$ is true, continuous if false
  * bool getDiscrete(): Returns the state of discrete output
  * void setThreshold(double x): Sets the threshold value for discrete output to be x, any output >= x will be 1, and any output < x will be 0

    * double getThreshold(): Returns the threshold value
  - Methods related to transforming datasets:
    * bool raw2train(): Converts the raw dataset to a training set by normalizing its input variates, and if nondiscrete, its output variates, returning true if the operation was successful
    * bool randomize(unsigned n): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the number of exemplars n to be placed in the test set as an argument *(number of outputs must be only 1, and it must be discrete)*, returns true if successful
    * bool randomize(unsigned n, unsigned d): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the numerator n and denominator d of a fraction representing the portion of examplars to be placed in the test set as arguments, *(number of outputs must be only 1, and it must be discrete)*, returns true if successful
    * bool randomizeD(double x): Randomizes the raw dataset into training and test sets which are then normalized, preserving the frequency of outcomes in the training and test sets, takes the fraction x as a double value from 0 to 1 representing the portion of examplars to be placed in the test set as argument *(number of outputs must be only 1, and it must be discrete)*, returns true if successful
  - Methods related to calculating accuracy
    * void metricsReport(ostream& *streamObj*): Outputs all internal *TwoSet* metrics for a 1-output *Model*, takes ostream object *streamObj* as argument
  - Methods related to logging to the history file:
    * void setHistory(bool $flag$): Sets history logging, which *appends* all operations to the history file
    * bool getHistory(): Returns the current state of history logging
    * void setHistoryFilename(string& $filename$): Sets the name of the history file to string $filename$
    * bool addHistory(ostringstream& *outputStream*): appends the ostringstream object *outputStream* to the history file.

- *Private* members
  - Matrix< double > *Raw*: the raw dataset
  - Matrix< double > $TrainSetData$: the training set
  - Matrix< double > $TestSetData$: the test set

- TwoSet *TrainTwoSet*: the *TwoSet* object for the training set
- TwoSet *TestTwoSet*: the *TwoSet* object for the test set
- vector< double > *minima*: a vector containing the minimum values of the columns in *Raw*
- vector< double > *maxima*: a vector containing the maximum values of the columns in *Raw*
- unsigned *nInput*: the number of input nodes
- unsigned *nOutput*: the number of output nodes
- double *threshold*: the threshold for discrete output
- double *inUpperLimit*: the upper limit for a normalized input variate
- double *inLowerLimit*: the lower limit for a normalized input variate
- double *outUpperLimit*: the upper limit for a normalized output variate
- double *outLowerLimit*: the lower limit for a normalized output variate
- bool *discreteFlag*: a flag to indicate if the output is to be considered discrete (held to 0,1)
- bool *rawLoadedFlag*: a flag to indicate if *Raw* loaded
- bool *trainLoadedFlag*: a flag to indicate if *TrainSetData* loaded
- bool *testLoadedFlag*: a flag to indicate if *TestSetData* loaded
- bool *trainTwoSetFlag*: a flag to indicate if the training set *TwoSet* object has been successfully constructed
- bool *testTwoSetFlag*: a flag to indicate if the test set *TwoSet* object has been successfully constructed
- bool *historyFlag*: a flag to indicate if operations will be logged to a history file, this file is *appended*
- bool *minimaxFlag*: a flag to indicate if minima and maxima vectors have been set
- string *historyFilename*: the name of history file

- *Private* Utility functions
  - void minimax(Matrix< double >& *data*): computes the *minima* and *maxima* vectors from *Matrix data*
  - bool checkDiscrete(Matrix< double >& *data*): returns true if the output columns of *Matrix data* are discrete, false if otherwise
  - void normalize(Matrix< double >& *data*): normalizes input variates of *Matrix data*, and if nondiscrete, its output variates *(note: minima and maxima vectors must have been previously set)*
  - void clear(): clears *DataSet Matrix* members and flags

12.15 TwoSet

TwoSet is a specialized standalone object that is designed to calculate outcome metrics such as sensitivity, specificity, predictive value positive, predictive value negative and ROC area from a *one* output *Model*. The *TwoSet* data *Matrix* contains 2 columns, one for the known outcomes and one for the *Model's* outputs. The *Model* output *must be* discrete. The format of this data *Matrix* is:

$$\begin{array}{cc} real_0 & test_0 \\ real_1 & test_1 \\ real_2 & test_2 \\ \cdots & \cdots \\ real_{n-1} & real_{n-1} \end{array}$$

where *real* represents a known outcome, and *test* is the *Model's* output, for n examples in the data *Matrix*, e.g.:

```
0.000000 0.002851
0.000000 0.002978
0.000000 0.002857
1.000000 0.571710
```

For calculating ROC (receiver-operator characteristic) curve area, two complementary estimates are available. The empirical area is the exact Mann–Whitney ranking probability obtained by sweeping every distinct score; no threshold count or bin count is configured. Wickens[10] provides a statistical method to calculate ROC area. Here, $z = Z(A) = \Phi^{-1}(A)$ is linearly modeled for sensitivity and 1 - specificity, and this linear model is used to calculate ROC area. If enough distinct interior operating points are available to generate errors in the abscissa and ordinate of this line, a p-value reflects the probability of a least squares fit to that line, where a smaller p-value is *worse* (closer to 1 is more linear).

The method *ROCarea*(*ostream*&) may be used to automatically choose which method of calculating ROC curve are is used, depending on how many data points are available.

Two errors may be thrown from this class. The first is *DivisionByZero* where division by zero is encountered, generally in calculating positive and negative predictive value. The second is *twoSetErr*, which will contain errors generated in statistical calculation or linear modeling. *DivisionByZero* always contains the message "division by zero", whereas *twoSetErr*'s *what*() member will contain a message reflecting the type of error. An example of the latter would be:

```
string reply;
cout << "What is the name of the file? ";
```

[10] Wickens, Thomas D., Elementary Signal Detection Theory, Oxford University Press, New York, NY 2002, pp. 60–74

```
cin >> reply;
TwoSet data;
data.load( reply );
try {
   data.ROCarea( cout );
} catch ( TwoSet::twoSetErr& e ) { cout << e.what() << endl; }
```

The *TwoSet* object consists of:

- *Public* methods

  The constructor TwoSet(Matrix< double >& M) constructs and loads *Matrix* M into a *TwoSet* object

  - Methods for loading Matrices and files into a *TwoSet* object:
    - void setMatrix(Matrix< double >& M): Loads *Matrix* M into a *TwoSet* object, returns nothing
    - void load(string& $filename$): Takes $filename$ as argument (string), and loads a file into the *TwoSet* data *Matrix*, returns nothing
    - bool loaded(): Returns a flag to indicate if the *TwoSet* data *Matrix* is loaded
  - *TwoSet* accessors:
    - Matrix< double >& getData(): Returns the *TwoSet* data *Matrix*
    - bool save(string& $filename$): Takes $filename$ as argument (string), and saves the *TwoSet* object into a file, returns flag indicating if save operation was successful
    - unsigned getNumElements(): Returns the number of elements in the *TwoSet* data *Matrix*
    - TwoSet& insertReal(vector< double >& v): Insert vector v into the real column of the *TwoSet* data *Matrix*, the number of elements of v *must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double >& real(vector< double >& v): Populate vector v with the real column of the *TwoSet* data *Matrix*, the number of elements of v *must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double > real(): Get a new vector from the real column of the *TwoSet* data *Matrix*
    - TwoSet& insertTest(vector< double >& v): Insert vector v into the test column of the *TwoSet* data *Matrix*, the number of elements of v *must equal* the number of rows in the *TwoSet* data *Matrix*
    - vector< double >& test(vector< double >& v): Populate vector v with the test column of the *TwoSet* data *Matrix*, the number of elements of v *must equal* the number of rows in the *TwoSet* data *Matrix*

    * vector< double > test(): Get a new vector from the test column of the *TwoSet* data *Matrix*
    * double real(unsigned r): An accessor to set or retrieve the element in row r in the real column of the *TwoSet* data *Matrix*
    * double test(unsigned r): An accessor to set or retrieve the element in row r in the test column of the *TwoSet* data *Matrix*
- Accessors and methods for *TwoSet* calculated metrics
    * void setThreshold(double x): Sets threshold value to x, any test element >= x will be counted as 1, whereas any test element < x will be counted as 0
    * double getClassAcc(): Returns classification accuracy
    * double getSens(): Returns sensitivity, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * double getSpec(): Returns specificity, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * double getPVP(): Returns predictive value positive, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * double getPVN(): Returns predictive value negative, throws exception TwoSet::DivisionByZero& e, where e.what() is "division by zero"
    * void ClassTable(ostream& *outputStream*): Outputs to ostream& the classification table
- ROC area calculation by trapezoidal method
    * double getTrapROCarea(): Returns ROC area calculated by the exact empirical sweep over every distinct operating point; there is no configurable threshold count
- ROC area calculation by statistical method
    * double getStatROCarea(): Returns statistical ROC area, throws exception TwoSet::twoSetErr& e, where e.what() contains a statistical or linear regression error message
    * double getStatP(): Returns p-value for fitted line (smaller is worse)
    * double getStatChi2(): Returns chi-squared value for fitted line
- Methods and accessors which output to ostream a ROC report which uses either or both the trapezoidal and/or statistical method
    * void ROCarea(ostream& *outputStream*): Outputs to ostream& ROC area, and a report detailing method and results of calculation
    * void setROCReportFlag($flag$): Set $flag$ true if both statistical and trapezoidal methods are to be reported, false to use either the trapezoidal method if number of data points < threshold ($calcThresh$) or statistical method if >= threshold

    - bool getROCReportFlag(): Returns flag indicating if either (false) or both (true) of the statistical and trapezoidal methods are used and reported, according to the above description of setROCReportFlag(...)
    - void setROCthresh(unsigned n): Sets threshold for number of non-zero, non-one sensitivity and 1 - specificity data points under which statistics will not be used
    - unsigned getROCthresh(): Returns number of non-zero, non-one sensitivity and 1 - specificity data points under which statistics will not be used
    - void setROCSearchFlag(bool $flag$): Set $flag$ true if best p and ROC areas are searched and reported within a specified range of number of bins
    - bool getROCSearchFlag(): Returns flag indicating if best p and ROC areas are searched and reported within a specified range of bins
    - void setMinROCSearchBins(unsigned n): Sets minimum number of bins to search for best p and ROC areas (must be > 2)
    - unsigned getMinROCSearchBins(): Returns minimum number of bins to search for best p and ROC areas
    - void setMaxROCSearchBins(const unsigned n): Sets maximum number of bins to search for best p and ROC areas
    - unsigned getMaxROCSearchBins(): Returns maximum number of bins to search for best p and ROC areas
  - Accessors for the Kolmogorov-Smirnov test[11]
    - double getKSD(): Returns Kolmogorov-Smirnov statistic D
    - double getKSP(): Returns Kolmogorov-Smirnov p- value

- *Private* members
  - Matrix< double > A: the *TwoSet* data *Matrix*
  - unsigned n: the number of elements in the *TwoSet* data *Matrix*
  - unsigned tp: the number of true positives
  - unsigned tn: the number of true negatives
  - unsigned fp: the number of false positives
  - unsigned fn: the number of false negatives
  - unsigned statPoints: distinct interior operating points used by the most recent zROC fit
  - unsigned calcThresh: the number of data points under which statistics will not be used

[11] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 623–626

- double threshold: the threshold value
- double statP: p-value for fitted line for statistical ROC calculation
- double statChi2: chi-squared value for fitted line for statistical ROC calculation
- double KSD: Kolmogorov-Smirnov test D value
- double KSP: Kolmogorov-Smirnov test p-value
- bool loadedFlag: a flag to indicate if the *TwoSet* data *Matrix* is loaded
- bool thresholdFlag: a flag to indicate if the threshold is set
- bool nBinFlag: flag indicates if number of bins determines bin size
- bool binnedFlag: flag indicates if data was binned for statistical ROC calculation
- bool statROCcalcFlag: flag indicates if statistical ROC area calculated
- bool reportFlag: flag directs how to report statistical and/or trapezoidal ROC
- bool searchFlag: flag indicates if largest p, largest ROC is searched
- bool KScalcFlag: flag indicates if Kolmogorov-Smirnov test calculated

- *Private* Utility functions
  - void initialize(): Sets common initial values for constructors
  - void calculate(): Given a threshold, calculates tp, tn, fp, and fn
  - bool checkDiscrete(Matrix< double >& M): Returns true if the first column (representing real outcomes) of *Matrix* M is discrete, false if otherwise
  - void statReport(ostream& $outputStream$, unsigned $goodData$, unsigned $nBins$, double AUC, double $chi2$, double p): Utility method for ROCarea, outputs statistical report given ostream& $outputStream$, number of non-zero, non-one data points $goodData$, number of bins $nBins$, area under the curve AUC, chi-square value $chi2$, and p-value p
  - void KScalc(): Utility method to calculate Kolmogorov-Smirnov test[12]

[12] Adapted from W. H. Press et. al, Numerical Recipes in C, 2nd edition, Cambridge University Press, Cambridge, 1992, pp. 623–626

12.16 What you really need to know about programming a Model

12.16.1 Object Type

All concrete derived objects of *Model* inherit its protected member, (string) *objType*, which is the name of the object. Typically, the class constructor defines this member, for example:

```
class BareProp : public Network {
public:
   BareProp() { objType = "BareProp"; } // default constructor
```

Abstract classes *do not* define *objType*. This member is accessed by the *Model* method getType().

12.16.2 Copy constructor and overloaded = operator

All derived objects of *Model* **must** have a copy constructor and overloaded = operator. Typically, both call a protected utility function so that both copy the same members. The protected utility function typically begins with a call to the copy method of its immediate base class so that it inherits all of the members of its base class(es). For example, suppose a hypothetical neural network object named "LittleNet" is derived from *Network*, and contains the following members:

```
unsigned nNodes; // number of nodes

Matrix< double > W; // weight Matrix

vector< double > I, // input vector
                 y; // output vector
```

The following would be defined in the header file:

```
class LittleNet : public Network {
public:
   // Copy constructor
   LittleNet( const LittleNet& rhs );

   // Overloaded = operator
   LittleNet& operator= ( const LittleNet& rhs );

private: // or protected if the object is a base class
   // Copy utility
   void copy( const LittleNet& rhs );
```

Which would then be coded in the $C++$ file as:

```
// Copy constructor
LittleNet::LittleNet( const LittleNet& rhs )
{
   LittleNet::copy( rhs ); // use the copy utility
}

// Overloaded = operator
LittleNet& LittleNet::operator= ( const LittleNet& rhs )
{
   if ( &rhs != this ) // check for self-assignment
      LittleNet::copy( rhs ); // use the copy utility

   return *this; // enables A = B = C
}

// Copy utility
void LittleNet::copy( const LittleNet& rhs )
{
   Network::copy( rhs ); // call immediate base object copy
   nNodes = rhs.nNodes;
   W = rhs.W;
   I = rhs.I;
   y = rhs.y;
}
```

Remember: if you add any member to any existing class, you *must* insert a corresponding line of code to copy it in the copy utility function of that class.

12.16.3 Implementation of pure virtual methods in *Model*

Ultimately, all models map the inputs of each exemplar to the outputs. Thus, most of the *Model* base class methods pertain directly to this process. If you are writing a Model, you need to implement the following pure virtual methods:

setDataSet

The pure virtual method

```
virtual void setDataSet( DataSet& $dataObj$ )
```

loads a *DataSet* object into the derived *Model*. The protected member inherited from the base class *Model* is named *theData*, so the first line of

```
void YourModel::setDataSet( DataSet& dataObj )
```

where "YourModel" is the name of your derived *Model* object, should be

```
theData = dataObj; // set "theData" to incoming DataSet
```

The remainder of YourModel::setDataSet would then contain code that tailors your model to the incoming *DataSet* object. See SimpleProp::setDataSet and BareProp::setDataSet for examples.

train

The pure virtual method

```
virtual double train()
```

should train a *Model*, and return the final error (double) for that model. In the case of neural networks, training would involve finding the weight vectors and Matrices that minimize final error. See SimpleProp::train and BareProp::train for examples.

reportAccuracy

The pure virtual method

```
virtual void reportAccuracy( ostream& outputStream )
```

should report the accuracy of the trained model to an ostream outputStream. *If you are programming an object that extends* Network, *e.g. a neural network, you* **do not** *need to implement this virtual method, as* Network *implements reportAccuracy for you.*

outputHeader

The pure virtual method

```
virtual void outputHeader( ostream& outputStream )
```

should output a header to an ostream outputStream describing the architecture of the derived *Model*. See SimpleProp::outputHeader and BareProp::outputHeader for examples.

12.16.4 *Model* protected members

The following are the protected members in *Model* that your object needs to set or get:

theData

The protected member

```
DataSet theData
```

represents the primary dataset, in the form of a DataSet object. Your setDataSet method needs to set this member. See SimpleProp::setDataSet and BareProp::setDataSet for examples.

Train

The protected member

```
Matrix< double > Train
```

represents a Matrix that contains the training set input columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractInputMatrices(). See SimpleProp::setDataSet and BareProp::setDataSet for examples.

Test

The protected member

```
Matrix< double > Test
```

represents a Matrix that contains the test set input columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractInputMatrices(). See SimpleProp::setDataSet and BareProp::setDataSet for examples.

TrainOutput

The protected member

```
Matrix< double > TrainOutput
```

represents a Matrix that contains the training set output columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractOutputMatrices(). See BackProp::setDataSet for an example.

TestOutput

The protected member

```
Matrix< double > TestOutput
```

represents a Matrix that contains the test set output columns only, typically constructed by a call within your setDataSet method to the *Model* protected utility function extractOutputMatrices(). See BackProp::setDataSet for an example.

builtFlag

Set the protected member

```
bool builtFlag
```

to *true* when the Model has been formally constructed, and its architecture set. See SimpleProp::setHidden and BareProp::setHidden for examples.

historyFlag

Use the protected member

```
bool historyFlag
```

to decide if operations will be logged to a history file using the protected *Model* utility function addHistory. See SimpleProp and BareProp code for examples, as most methods use addHistory.

errorType

Use the protected member

```
bool errorType
```

to determine if the type of error will be least mean squared ($errorType = 0$) or cross-entropy ($errorType = 1$). Typically, this member is used as an argument in *errorFunction* constructors (see section 12.12.3, and SimpleProp::trainSet() and BareProp::trainSet() as examples.)

12.16.5 *Model* protected utility functions

The following are the protected utility functions in *Model* that your object will want to call:

extractInputMatrices

Call the utility function

```
void extractInputMatrices()
```

in your setDataSet method to extract the input columns of the training set into the protected *Model* Matrix $Train$, and the input columns of the test set into the protected *Model* Matrix $Test$.

extractOutputMatrices

Call the utility function

```
void extractOutputMatrices()
```

in your setDataSet method to extract the output columns of the training set into the protected *Model* Matrix $TrainOutput$, and the output columns of the test set into the protected *Model* Matrix $TestOutput$.

addHistory

Pass an ostringstream object to the utility function

```
bool addHistory( ostringstream $outputStream$ )
```

for that object to be added to the history file. This method should be called for every operation. See SimpleProp and BareProp code for examples, as most methods use addHistory.

copy

See section 12.16.2 for details.

12.17 Model

In the following specification, methods and members that pertain directly to coding a concrete derived *Model* implementation are shown in red .

Model is the base object. *Model* provides access to the dataset, and contains the virtual methods common to all modeling techniques. *Model* consists of:

- *Public* methods
  - Copy constructor and overloaded = operator, see section 12.16.2 for discussion:
    - Model& operator= (const Model& rhs)
    - Model(const Model& rhs)
  - Methods related to accessing the dataset to model:
    - virtual void setDataSet(DataSet& *data*): A pure virtual method that loads the *DataSet* object *data* into the *Model*
    - DataSet& getDataSet(): Returns a the *Model's* dataset in the form of a *DataSet* object
  - Methods related to training the model:
    - virtual double train(): A pure virtual method that trains a *Model*, and returns the final error (double) for that model
    - virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method that reports the accuracy of the trained model to an ostream *outputStream*
  - Methods related to logging to files or reporting to user:
    - virtual void outputHeader(ostream& *outputStream*): A pure virtual method which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
    - void setHistory(bool *flag*): Sets history logging, which *appends* all operations to the history file

* bool getHistory(): Returns the current state of history logging
* void setHistoryFilename(string& *filename*): Sets the name of the history file to string *filename*
* string getHistoryFilename(): Returns the name of the history file
* void setLastop(bool *flag*): Sets last operation logging, which *overwrites* a file to report the building (training in the case of *Iterative*) of a specific model
* bool getLastop(): Returns the current state of last operation logging
* void setLastopFilename(string& *filename*): Sets the name of the last operation logging file to *filename*
* string getLastopFilename(): Returns the name of the last operation logging file

– Methods related to setting the output error function

* void setLMSerror(): Sets the output error function to least mean squared, which is

$$E = \frac{1}{2}\sum_{i=1}^{m}(y_i - o_i)^2 \tag{12.3}$$

where E is the error, m the number of outputs, y is the known value and o the Model's output.

* void setXEerror(): Sets the output error function to be cross-entropy, which is

$$E = \sum_{i=1}^{m} -y\log(o) - (1-y)\log(1-o) \tag{12.4}$$

Note: for the cross-entropy error function, the output **must** *be* **discrete**.

- *Protected* members

– DataSet *theData*: the primary dataset, in the form of a *DataSet* object

– Matrix< double > *Train*: a Matrix that contains the training set input columns *only*, constructed by a call to the protected utility function *extractInputMatrices()*

– Matrix< double > *Test*: a Matrix that contains the test set input columns *only*, constructed by a call to the protected utility function *extractInputMatrices()*

– Matrix< double > *TrainOutput*: a Matrix that contains the training set output columns *only*, constructed by a call to the protected utility function *extractOutputMatrices()*

- Matrix< double > *TestOutput*: a Matrix that contains the test set output columns *only*, constructed by a call to the protected utility function *extractOutputMatrices()*
- bool *builtFlag*: a flag to indicate if the *Model* has been formally constructed
- bool *historyFlag*: a flag to indicate if operations will be logged to a history file, this file is *appended*
- bool *lastopFlag*: a flag to indicate if model building (training in the case of *Iterative*) will be logged to a file describing the last operation, this file is *overwritten*
- bool *errorType*: flag indicates type of error function, 0 if least mean squared, 1 if cross-entropy
- bool *builtFlag*: a flag to indicate the type of error function, 0 if least mean squared, 1 if cross-entropy
- string *objType*: the name of the object, see section 12.16.1
- string *historyFilename*: the name of history file
- string *lastopFilename*: the name of the file to log the last operation
- string *errorLabel*: a string containing the name of the output error function

- *Protected* Utility functions
  - void extractInputMatrices(): Utility function which is intended to be called by the derived *Model* in its *setDataSet* method, and places the input columns of the training set in Matrix *Train*, and the input columns of the test set in Matrix *Test*
  - void extractOutputMatrices(): Utility function which is intended to be called by the derived *Model* in its *setDataSet* method, and places the output columns of the training set in Matrix *TrainOutput*, and the output columns of the test set in Matrix *TestOutput*
  - bool addHistory(ostringstream& *outputStream*): Utility function to append the ostringstream object *outputStream* to the history file.
  - void copy(const Model& rhs): Copy utility function, see section 12.16.2 for discussion.

12.18 What you really need to know about programming an Iterative Model

Iterative inherits methods setDataSet, train, reportAccuracy and outputHeader from *Model*. If you are programming an *Iterative Model*, there are 2 pure virtual methods in *Iterative* that you must also implement:

trainSet

Iterative handles models that require multiple passes (iterations) to train. Objects derived from *Iterative* must thus provide a method to adjust a model by a single pass through the training set, and *Iterative* will call this method iteratively to reach the final trained state. Thus the method:

```
virtual double trainSet()
```

changes the components of a *Model* (e.g. weight Matrices and vectors in a neural network), and returns the set error. See SimpleProp::trainSet() and BareProp::trainSet() for examples.

classAccuracy

With each iteration, *Iterative* reports the set error and classification accuracy for the training and/or test sets if the outputs are discrete. Thus, the pure virtual method:

```
virtual void classAccuracy( ostream& outputStream )
```

must output to an ostream object the classification accuracy for the training and/or test sets. *If you are programming a* Network *object, you* **do not** *need to implement this virtual method, as* Network *implements classAccuracy for you.*

getGradMax

Most iterative algorithms employ some form of gradient descent, and the iterative class can report the maximum absolute gradient and use it to determine cessation of training. The pure virtual method:

```
virtual double getGradMax()
```

must be coded to return the maximum absolute gradient of the derived object. *If you are programming a* Network *object, you* **do not** *need to implement this virtual method, as* Network *implements getGradMax for you.* However, you must code Network::pack() to create the *Network* protected member *stackG* so that Network::getGradMax() returns the proper value.

runHeader

If you are adding parameters specific to an *Iterative* derived object that should be output to screen or log files prior to an *Iterative* run, you will need to implement a runHeader method in the derived class:

```
virtual void runHeader( ostream& outputStream)
```

which outputs to an ostream object the specific parameters to be displayed. *If you are programming a* Network *object, you will need to add code to this method in* Network.

As with any derivation of *Model*, a copy utility function, copy constructor and overloaded = operator must be implemented. See section 12.16.2 for details.

iteration

You may access the iteration index in your derived object by the protected member unsigned *iteration*.

boundsErrorFlag

If your object can generate a numerical bounds error (see section 12.12.3 for an example of a method which generates a numerical bounds error), then use the protected member bool *boundsErrorFlag* to indicate if such an error has been thrown.

gradMaxFlag

Use the protected member bool *gradMaxFlag* to determine if a gradient is to be constructed in your derived object.

12.19 Iterative

Iterative extends *Model*. Neural computational algorithms are iterative, and thus use *Iterative* as a base. *Iterative* manages methods and parameters related to iterating through a dataset.

In the following specification, methods and members that pertain directly to coding a concrete derived *Iterative* implementation are shown in red .

- *Public* methods
  - Copy constructor and overloaded = operator, see section 12.16.2 for discussion:
    - Iterative& operator= (const Iterative& rhs)
    - Iterative(const Iterative& rhs)
  - Methods pertaining to stopping training
    - void setMaxIterations(unsigned *maxIterations*): Sets the maximum number of iterations through the training set to *maxIterations*
    - unsigned getMaxIterations(): Returns the maximum number of iterations through the training set

    * void setMinStop(bool *minStopFlag*): Flag sets if a minimum error threshold will be used to stop training
    * bool getMinStop(): Returns flag indicating if a minimum error threshold will be used to stop training
    * void setMinError(double *minError*): Sets minimum error threshold value to *minError*
    * double getMinError(): Returns minimum error threshold value
    * void setChangeStop(bool *changeStopFlag*): Flag sets if training stops after error increases over 1 iteration
    * bool getChangeStop(): Returns flag indicating if change over 1 iteration will be used to stop training
    * void setChange(double Δ): Sets limit of change value to Δ
    * double getChange(): Returns limit of change value
    * void setWindowStop(bool *windowStopFlag*): Flag sets if training stops if set error increases over a specified window width
    * bool getWindowStop(): Returns flag indicating if set error increases over a window width will be used to stop training
    * void setWindow(unsigned *window*): Sets the value of the window width to *window*
    * unsigned getWindow(): Returns specified window width value
    * void setGradStop(bool *gradMaxFlag*): Flag sets if training will stop when the maximum absolute gradient decreases below the limit
    * bool getGradStop(): Returns flag indicating if training will stop when the maximum absolute gradient decreases below the limit
    * void setGradMaxLimit(double *limit*): Sets the value of the maximum absolute gradient limit to *limit*
    * double getGradMaxLimit(): Returns the value of the maximum absolute gradient limit

- Methods pertaining to printing while training
    * void setLogPrint(bool *logPrintFlag*): Sets if prints out by powers of 10 (1-10 by ones, 10-100 by 10s, 100-1000 by 100s, etc.)
    * bool getLogPrint(): Returns flag indicating if printing by powers of 10
    * void setPrintCount(unsigned *printCount*): Sets the linear print counter to *printCount*
    * unsigned getPrintCount(): Returns the linear print counter

- General methods
    * virtual void setDataSet(DataSet& *data*): A pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Model*

    * virtual double train(): A pure virtual method inherited from *Model* that trains a *Model*, and returns the final error (double) for that model
    * virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*
    * virtual void outputHeader(ostream& *outputStream*): A pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
    * virtual double trainSet(): A pure virtual method which trains one iteration through the training set, *Iterative Model* type dependant, and returns the set error
    * virtual void classAccuracy(ostream& *outputStream*): A pure virtual method which outputs classification accuracies to an ostream *outputStream* for an *Iterative* output table

- *Protected* members
  - unsigned *iteration*: the iteration index
  - unsigned *maxIterations*: the maximum number of iterations through the training set
  - bool *logPrintFlag*: flag indicates if printing by powers of 10
  - unsigned *printCount*: the value for the linear print counter
  - bool *minStopFlag*: flag indicates if a minimum error threshold will be used to stop training
  - double *minError*: minimum error threshold value
  - bool *changeStopFlag*: flag indicates if change in error over 1 iteration will be used to stop training
  - double *delta*: value of change over 1 iteration
  - bool *gradMaxFlag*: flag indicates if training will stop when maximum absolute gradient decreases below *gradMaxLimit*
  - double *gradMaxLimit*: value of the maximum absolute gradient limit
  - bool *windowStopFlag*: flag indicates if set error increases over a window width will be used to stop training
  - unsigned *window*: value of the window width
  - bool *boundsErrorFlag*: flag indicates if out of numerical bounds error encountered (e.g. log(0))

- *Protected* Utility functions

- void copy(const Iterative& rhs): Copy utility function, see section 12.16.2 for discussion.
- virtual void runHeader(ostream& *outputStream*): A pure virtual utility function that outputs parameters specific to derived objects preceeding an Iterative run
- virtual double getGradMax(): A pure virtual utility function that returns the maximum absolute gradient value

12.20 What you really need to know about programming a Network Model

12.20.1 Implementation of pure virtual methods in *Network*

Network inherits methods setDataSet, train, reportAccuracy and outputHeader from *Model*, and trainSet from *Iterative*, so you must code all of these methods. (See the respective methods in SimpleProp and BareProp as examples.) If you are programming a *Network Model*, there are 7 additional pure virtual methods in *Network* that you must implement in the concrete derived object:

df

The method

```
virtual unsigned df()
```

must be coded to return the number of degrees of freedom of your *Network* object, which is equal to the number of nodal connections. See SimpleProp::df and BareProp::df as examples.

randomize

The method

```
virtual void randomize()
```

must be coded to randomize the weights in your *Network* object. See SimpleProp::randomize and BareProp::randomize as examples.

load and save

The methods

```
virtual bool load( string& filename )
virtual bool save( string& filename )
```

must be coded to load and save your *Network* derived object from and to a file named "filename". Typically the method save would use your coded method outputHeader to write the *Network* object architecture to a file, then the Matrices and vectors representing the *Network* model. As expected, load should do the opposite. See SimpleProp::load, SimpleProp::save, BareProp::load and BareProp::save as examples.

innerTrainSet

The method

```
virtual double innerTrainSet()
```

must be coded in a *Network* object to calculate the gradient and error. The reason that another method in addition to the pure virtual method trainSet inherited from *Iterative* must be coded is to support automatic stepsize selection. The strategy proceeds as follows: trainSet is intended to call innerTrainSet to determine if the set error would increase, and set the learning rate ("step size") accordingly lower if it would. Thus, innerTrainSet calculates the gradient and error, but does not necessarily update the weights. If automatic stepsize *is not* selected, trainSet simply calls innerTrainSet to calculate the gradient and error, and updates the weights. If automatic stepsize *is* selected, trainSet calls innerTrainSet to calculate the gradient and error, and reduces the stepsize before updating the weights if the error is found to increase. See SimpleProp::innerTrainSet, BareProp::innerTrainSet and BackProp::innerTrainSet as examples of the way trainSet and innerTrainSet are coded to work together.

The search itself is written once, as *Network*::searchStepSize — a protected member template. Each concrete model's trainSet supplies two things and nothing else:

- its own **activation guard**, which is *not* the same in every model. *Logistic* searches whenever automaticStepSizeFlag is set; *SimpleProp*, *BareProp* and *BackProp* require batchEpochFlag as well, because the search compares the error of a whole pass against the previous pass and that only means something when a pass makes one weight update. The guard is written out at each call site rather than delegated to a policy method, so the asymmetry cannot be lost.
- its own **WeightSnapshot**, a private nested type holding just that model's weights: hW and oW for *SimpleProp* and *BareProp*, Weights for *BackProp*, W for *Logistic*. searchStepSize constructs one as a local of the search, so no model carries a duplicate of its weights between calls and nothing is copied at all when the search is off.

Network restores lastG and lastF itself, since those are its own members and are the history conjugate gradient descent and Shanno carry between iterations. Nothing else is restored: η, the gradients, the accumulators and currGradMax

are left as the trial passes leave them. Restoration is explicit rather than a destructor, so an exception out of innerTrainSet leaves trial weights installed exactly as it always has.

Richard Golden has suggested the following automatic stepsize selection algorithm, which he terms the "Crummy Back-tracking Line Search Algorithm":

- Let DELTAERROR be a small non-negative number (e.g., DELTAERROR = 1e-10 or 0)
- Let MAXLOOPS = a small positive integer (e.g., MAXLOOPS = 2 or 3 or 4)
- Let GAMMA be some positive number less than 1 (e.g., GAMMA = 0.5 or 0.8)
- Let LoopCounter = 0
- Set "candidate" learning rate equal to 1.0
- While Exit Loop is FALSE
  - LoopCounter = LoopCounter + 1
  - Compute change to weights using current candidate learning rate
  - Evaluate the new ERROR at the weights you obtained to get a "candidate error change"
  - If ("candidate error change" < DELTAERROR) OR (LoopCounter > MAXLOOPS), then EXITLOOP is set to TRUE
  - Else set current candidate learning rate = GAMMA [candidate learning rate]
  - End While Loop

Don't be fooled by the name–this algorithm is far from "crummy".

Note: This algorithm is only to be used with off-line or batch/epoch learning.

forward

The method

```
virtual void forward( Matrix< double >& M, const unsigned r )
```

must be coded to take a Matrix containing only the inputs of a training or test set as one argument (typically the *Train* or *Test* inherited protected members from *Model*), the position of the exemplar in the training or test set as a second argument, and derive the *Network's* output. *This is not returned by the method. This is very important: in single output* Networks, *the inherited* Network *protected member o would be set by this method, and in multiple*

output Networks, *the inherited* Network *protected member vector< double > output would be set by this method. Also very important, if sigmoidal output nodes are supported, the inherited* Network *protected member x would be set by this method, and in multiple output* Networks, *the inherited* Network *protected member vector< double > xs would be set by this method.*

removeInputs

The method

```
virtual void removeInputs( const vector< unsigned >& v )
```

must be coded to take as its argument a vector of unsigned representing positions of input nodes, and to remove those nodes from the *Network* derived object. Typically, this would be accomplished first by a call to *DataSet*::removeInputs(vector< unsigned >& *v*), followed by a call to *Model*::extractInputMatrices(), followed by code which removes input nodes in the structures representing weights in the *Network* object, typically *Matrix* columns. See SimpleProp::removeInputs and BareProp::removeInputs as examples.

12.20.2 Cloning and construction required for a new *Network*

A new concrete *Network* type must participate in the centralized `cloneNetwork()` dispatch in `netclone.*`; this is the clone used by stepwise regression and automatic optimizer probes. If the type may be created or loaded through a human interface it must also participate in `modelfactory.*`. *RegressNet* contains no type switch and must not be edited for each concrete network. See section 13.6.

12.20.3 *Network* protected members

The following are the protected members in *Network* that your object needs to set or get:

trainingType

The protected member

```
unsigned trainingType
```

determines the type of gradient descent algorithm of your neural network. $trainingType = 0$ is defined as canonical backpropagation. If you add other types of training to the *Network* class, you must define a unique *trainingType* for them. Currently coded algorithms include $trainingType =$

1. Conjugate gradient descent
2. Shanno algorithm

NOTE: Look closely at the code for Network::getGradMax(). You will find that this code assumes that if *trainingType* is not 0, then *stackG* will be constructed. If this is not true for your *trainingType*, then you must modify the if block in Network::getGradMax().

o

The protected member

```
double o
```

is the output for a single output network. *If your* Network *object supports single outputs, it must set this protected member,* typically in the method forward. Since the method trainSet will typically call the method forward, trainSet will generally access the member *o*. See SimpleProp::forward and BareProp::forward for examples in setting the protected member *o*, and SimpleProp::trainSet and BareProp::trainset for examples in accessing the protected member *o*.

x

The protected member

```
double x
```

contains the x term for the sigmoidal output o in a single output network:

$$o = 1/(1 + e^{-x})$$

If your Network *object supports single sigmoidal outputs, it must set this protected member,* typically in the method forward. See SimpleProp::forward and BareProp::forward for examples in setting the protected member x.

output

The protected member

```
vector< double > output
```

is the output for a multiple output network. *If your* Network *object supports multiple outputs, it must set this protected member,* typically in the method forward. Since the method trainSet will typically call the method forward, trainSet will generally access the member *output*.

xs

The protected member

```
vector< double > xs
```

contains the $\vec{\mathbf{x}}$ terms for the sigmoidal outputs $\vec{\mathbf{o}}$ in a multiple output network:

$$\vec{\mathbf{o}} = 1/(1 + e^{-\vec{\mathbf{x}}})$$

If your Network *object supports multiple sigmoidal outputs, it must set this protected member,* typically in the method forward. See BackProp::forward for an example in setting the protected member xs.

randomLimit

The protected member

```
double randomLimit
```

is used to specify the range of randomized weights, which will be between $-randomLimit$ and $+randomLimit$. Access this protected member in your randomize method. See SimpleProp::randomize and BareProp::randomize for examples.

weightsSetFlag

Your derived *Network* object must set the protected member

```
bool weightsSetFlag
```

to *true* when the weights are set, either by randomization or loading weights from a file. See SimpleProp::randomize, BareProp::randomize, SimpleProp::load and BareProp::load for examples of setting $weightsSetFlag$ to *true*, and SimpleProp::setHidden and BareProp::setHidden for examples of setting $weightsSetFlag$ to $false$.

biasFlag

Use the protected member

```
bool biasFlag
```

to determine if your *Network* object has biases.

eta

Use the protected member

```
double eta
```

for the learning rate η of your *Network* object.

batchEpochFlag

Use the protected member

```
bool batchEpochFlag
```

to determine if your *Network* object uses batch/epoch learning or not. (See the description of classic off-line ("batch") backprop in section 6.2.2.) *Pay special attention to how the learning rate is set.* It is expected that the driver will guide the user to the appropriate setting of the learning rate. Note also that "off- line" learning is synonymous with " batch" or "epoch" learning, whereas "on-line" learning refers to updating the weights with each exemplar. Here, if batchEpochFlag is *true*, "offline" learning is invoked, and if batchEpochFlag is *false*, "online" learning is used.

weightDecayFlag

Use the protected member

```
bool weightDecayFlag
```

to determine if your *Network* object uses weight decay. (See the discussion of equation 6.1 in section 6.1.2 for an explanation of weight decay.) The learning rate equivalent of equation 6.1 is

$$\begin{aligned} \vec{w}_{t+1} &= \vec{w}_t - \eta \left.\frac{\partial E'}{\partial w}\right|_{w_t} \\ &= \vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t} - 2\eta\lambda\vec{w}_t \\ &= (1 - 2\eta\lambda)\vec{w}_t - \eta \left.\frac{\partial E}{\partial w}\right|_{w_t} \end{aligned}$$

where $(1 - 2\eta\lambda)$ is "weight decay". λ may be set by either of 2 methods (N is the sample number size, see equation 6.1 in section 6.1.2 for a discussion of σ_w):

- The complexity of the population objective function >> sample objective function:

$$\lambda = \frac{1}{N 2\sigma_w^2} \tag{12.5}$$

- The complexity of the population objective function $\simeq$ sample objective function:

$$\lambda = \frac{1}{2\sigma_w^2} \tag{12.6}$$

It is anticipated that the driver will guide the user to set the appropriate value of λ, and *insure that the user knows what he or she's doing if they change it!*

decay

You can use the protected member

```
double decay
```

for the weight decay term 2λ of your *Network* object. Note the multiple of 2 so that calculation of decay with the gradient as in the right hand term of equation 6.10, $(1/\sigma_w^2)[\mathbf{W}\ ,\ \mathbf{b}]$, does not require the additional step of multiplication by 2.

regularizer

You can use the protected member

```
double regularizer
```

if you need to access λ directly, e.g. for of calculation of your error function in equation 6.1, which like $decayTerm$ only needs to be calculated once prior to iterative calls to the TrainSet method, and is thus placed in the member function runHeader, where the *Iterative* class will call it once prior to iteratively calling the TrainSet method.

decayTerm

In the protected member function runHeader in the *Network* class,

```
double decayTerm
```

is defined to be $1 - 2\eta\lambda$ (or $1 - \eta decay$), which only needs to be calculated once prior to iterative calls to the TrainSet method, and is thus placed in the member function runHeader, where the *Iterative* class will call it once prior to iteratively calling the TrainSet method. You may therefore use $decayTerm$ as the weight multiple for weight decay in your *Network* object if the decay is not calculated with the gradient. If the decay is calculated with the gradient as in equation 6.10, then *decay* is used. See BareProp::TrainSet and SimpleProp::TrainSet for examples using $decayTerm$.

stackG

The protected member

```
vector< double > stackG
```

is a single vector which contains the entire weight gradient of your *Network* derived object. It is used by the protected utility functions pack and unpack (see below).

12.20.4 *Network* protected utility functions

Copy utility function, constructor and overloaded = operator

As with any derivation of *Model*, a copy utility function, copy constructor and overloaded = operator must be implemented. See section 12.16.2 for details.

runHeader utility function

If you are adding parameters specific to a *Network* object that should be output to screen or log files prior to an *Iterative* run, you will need to add code to the runHeader method in *Network*. See section 12.18.

pack and unpack

The methods

```
virtual void pack()
virtual void unpack()
```

must be coded to convert the weight gradient structure of your *Network* derived object to the member vector *stackG* (pack), and vice versa (unpack). See SimpleProp::pack, SimpleProp::unpack, BareProp::pack and BareProp::unpack as examples.

12.20.5 Built-in gradient descent algorithms

One of the really neat things about *Networks* that construct the *stackG* vector is that they may use the built-in advanced gradient descent algorithms, Shanno's and conjugate gradient descent.[13] All the programmer needs to do is to construct the *stackG* vector, add the line of code

```
engine( trainingType, iteration );
```

after the *Network* object gradient structure is calculated and before the weights are updated, and these algorithms will automagically be available for the user. See SimpleProp::trainSet() and BareProp::trainSet() for examples.

Shanno algorithm

[14]

Briefly, Shanno's algorithm proceeds as follows: let the computed gradient be $\vec{g}$ (*stackG*), and the iteration be t (**Note:** the iteration is *not* the exemplar index through the training set. It is the actual iteration number for each

[13] Richard M. Golden, Mathematical Methods for Neural Network Analysis and Design, 1996, MIT Press, ISBN 0-262-07174-6, pp. 217–218, 221–222

[14] D. F. Shanno, "Conjugate gradient methods with inexact searches," *Mathematics of Operations Research* 3(3), 244–256, 1978, https://doi.org/10.1287/moor.3.3.244.

calculated gradient. It should also be noted that this algorithm, as well as conjugate gradient descent *is intended for off-line or batch/epoch learning.*) Let the learning rate be γ.

- Step 0: Let $t = 0$.
- Step 1: Set $\vec{f}(t) = -\vec{g}_t$. Go to step 3.
- Step 2: If t is a multiple of the degrees of freedom of the *Network*, then go to step 1. Otherwise, compute $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ and the scalars a_t, b_t, c_t such that
  - $a_t = \frac{\vec{f}(t-1)^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$,
  - $b_t = \frac{\vec{u}^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$,
  - $c_t = \gamma + \frac{|\vec{u}_t|^2}{\vec{f}(t-1)^T \vec{u}_t}$,

  and descent vector
  $$\vec{f}(t) = -\vec{g}_t + a_t \vec{u}_t + (b_t - c_t a_t)\vec{f}(t-1)$$
- Step 3: Update the weights by $\vec{w}(t+1) = \vec{w}(t) + \gamma \vec{f}(t)$. Let $t = t + 1$. Go to step 2.

Interested readers are directed to Richard Golden's book, pp. 217–218. It should be noted that in the highly optimized code in Network::shanno(unsigned *iteration*), *lastF* refers to $-\vec{f}(t-1)$.

Conjugate gradient descent

Conjugate gradient descent is a special case of the Shanno algorithm, and briefly proceeds as follows: let the computed gradient be $\vec{g}$ (*stackG*), and the iteration be t (**Note:** again, the iteration is *not* the exemplar index through the training set. It is the actual iteration number for each calculated gradient. It should also be noted that this algorithm, as well as Shanno's, *is intended for off-line or batch/epoch learning.*) Let the learning rate be γ.

- Step 0: Let $t = 0$.
- Step 1: Set $\vec{f}(t) = -\vec{g}_t$. Go to step 3.
- Step 2: If t is a multiple of the degrees of freedom of the *Network*, then go to step 1. Otherwise, compute $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ and the scalar b_t such that
  $$b_t = \frac{\vec{u}^T \vec{g}_t}{\vec{u}_t^T \vec{f}(t-1)}$$
  and descent vector
  $$\vec{f}(t) = -\vec{g}_t + b_t \vec{f}(t-1)$$

- Step 3: Update the weights by $\vec{w}(t+1) = \vec{w}(t) + \gamma\vec{f}(t)$. Let $t = t+1$. Go to step 2.

Interested readers are directed to Richard Golden's book, pp. 221–222. It should be noted that in the highly optimized code in Network::CGD(unsigned *iteration*), *lastF* refers to $-\vec{f}(t-1)$.

12.20.6 Calculation of the condition number

The condition number neuron reports is a *design* diagnostic: it answers whether the estimation problem is well posed, or whether the columns of the design are close enough to collinear that the coefficients are poorly identified. It is the ratio of the largest to the smallest absolute eigenvalue of the observed Fisher information of the *unpenalized* log likelihood,

$$\mathbf{X}^T\mathbf{V}\mathbf{X}, \qquad \mathbf{V} = \mathrm{diag}\left(\pi_i(1-\pi_i)\right)$$

where π_i is the fitted probability for exemplar i. This is the same matrix the Wald analysis inverts for the coefficient covariance (Hosmer & Lemeshow equation 2.8), and *Logistic::reportAccuracy* forms it exactly once and uses it for both. *Network::conditionOf(const Matrix<double>&)* takes that matrix and stores the two extreme absolute eigenvalues and their ratio; *Network::reportCondNum(ostream&)* prints what it stored and computes nothing. The *Gnu Scientific Library* symmetric eigenvalue routines are used.

The penalty is deliberately excluded. Weight decay adds $\lambda\mathbf{I}$ to the curvature, which improves the conditioning of *any* design by construction, so a penalized condition number could be driven downward simply by regularizing more heavily — concealing exactly the collinearity the diagnostic exists to expose. If a penalized-curvature condition number is ever wanted, it belongs alongside this one under its own name, not in place of it.

Historical note. Through 2026 this quantity was instead the outer product $\mathbf{B} = \mathbf{g} \cdot \mathbf{g}^T$ of per-exemplar gradient vectors, accumulated by the overloaded *storeGrads* methods under a $finalFlag$ that marked a *final training iteration* run for the purpose. That construction was retired in August 2026 for four independent reasons: it trained the model as a side effect of reporting it; under conjugate gradient descent and Shanno's algorithm it stored search *directions* rather than gradients, because the optimizer transformation had already been applied; under canonical backpropagation with gradient stopping disabled it read a matrix that had never been written; and it included the weight-decay penalty, so regularization improved the reported conditioning. *storeGrads*, $finalFlag$ and the gradient accumulator are gone with it.

See *Logistic* for example code, and see *SimpleProp* for comments which indicate where code would be placed in a neural network style implemented *Network* object.

12.21 Network

In the following specification, methods and members that pertain directly to coding a concrete derived *Network* implementation are shown in red .

Network extends *Iterative*. Algorithms with hidden layers, weights, learning rates, transfer functions and other general properties of neural networks are encapsulated in *Network*.

Note: It is expected that in objects derived from *Network*, the method setDataSet inherited from *Model* be called *before* the derived methods that specify object architecture, so that the derived object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the inherited *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - Network& operator= (const Network& rhs): see section 12.16.2 for discussion
  - Network(const Network& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): A pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Model*
  - virtual void outputHeader(ostream& *outputStream*): A pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the model, *Model* type dependant
  - void setEta(double η): Sets the learning rate to η
  - double getEta(): Returns the learning rate η
  - void setBias(bool $flag$): Sets bias nodes in *Network*
  - bool getBias(): Gets boolean value indicating if bias nodes specified in *Network*
  - void setBatchEpoch(bool $flag$): Sets if batch/epoch learning is used in a *Network* object (See the description of classic off-line ("batch") backprop in section 6.2.2)
  - bool getBatchEpoch(): returns if batch/epoch used
  - void setWeightDecay(bool $flag$): Sets if weight decay is used in a *Network* object (See the discussion of weight decay in section 12.20.3)
  - bool getWeightDecay(): returns if weight decay used
  - void setDecay(double 2λ): Sets weight decay term to 2λ in a *Network* object
  - double getDecay(): returns weight decay term λ

- void setTrainingType(unsigned T): Sets training type to T
- unsigned getTrainingType(): returns training type
- void setAutoStepSize(bool $flag$): Sets if automatic stepsize selection is used (See the discussion of automatic stepsize selection in section 12.20.1)
- bool getAutoStepSize(): returns if automatic stepsize selection is used
- virtual unsigned df(): Returns the number of degrees of freedom, equal to the number of nodal connections
- void setRandomLimit(double L): Sets the limit for the random weights $-L$ to $+L$
- virtual void randomize(): A pure virtual method which randomizes initial weights in a *Network* object
- getWeightsSet(): Returns a flag to indicate if weights set
- virtual bool save(string& $filename$): A pure virtual method which saves the *Network* object architecture and weights to a file named $filename$, returns true if successful
- virtual bool load(string& $filename$): A pure virtual method which loads the *Network* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): A pure virtual method which trains one iteration through the training set, $Network$ type dependant, and returns the set error
- virtual double innerTrainSet(): A pure virtual method which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): A pure virtual method which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*
- virtual void classAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Iterative* which outputs classification accuracies to an ostream *outputStream* for an *Iterative* output table
- virtual void removeInputs(vector< unsigned >& v): A pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *Network* object

- *Protected* members
  - unsigned *trainingType*: the type of gradient descent algorithm, $trainingType = 0$ is reserved for canonical backpropagation ($trainingType = 1$ has been coded for conjugate gradient descent, and $trainingType = 2$ for Shanno algorithm)
  - double o: the output for a single-output *Network*
  - double x: the x term for the sigmoidal output o in a single output network:
$$o = 1/(1 + e^{-x})$$
  - vector < double > *output*: the outputs vector for a multiple output *Network*
  - vector < double > xs: the $\vec{\mathbf{x}}$ terms for the sigmoidal outputs $\vec{\mathbf{o}}$ in a multiple output network:
$$\vec{\mathbf{o}} = 1/(1 + e^{-\vec{\mathbf{x}}})$$
  - double *randomLimit*: the limit for the random weights
  - bool *weightsSetFlag*: a flag to indicate if weights set
  - bool *biasFlag*: a flag to indicate if bias nodes specified in *Network*
  - bool *batchEpochFlag*: a flag to indicate if batch/ epoch training is used in the *Network* object (See the description of classic off-line ("batch") backprop in section 6.2.2)
  - bool *weightDecayFlag*: a flag to indicate if weight decay is used in the *Network* object (See the discussion of weight decay in section 12.20.3)
  - double *decay*: the weight decay term 2λ
  - double *regularizer*: the weight decay term λ
  - double *decayTerm*: the weight decay multiple $1 - 2\eta\lambda$ calculated in the protected member function runHeader (see section 12.20.3 for an explanation)
  - double *eta*: the learning rate η
  - bool *automaticStepSizeFlag*: a flag to indicate if automatic step size algorithm is used
  - double *o_errAccumulate*: stores average output error accumulation over the examplars used in innerTrainSet() method
  - double *oldErrorAccumulate*: stores average output error accumulation for previous run over the examplars, used in innerTrainSet() method
  - double *deltaError*: deltaError constant for automatic stepsize selection

- double *currGradMax*: current absolute maximum gradient for shanno/CGD
- double *gamma*: gamma constant for automatic stepsize selection
- unsigned *maxLoops*: maxLoops constant for automatic stepsize selection
- vector < double > *stackG*: a single vector which contains the entire weight gradient of the *Network* derived object
- vector < double > *lastG*: $\vec{g}(t-1)$ for shanno/CGD
- vector < double > *lastF*: $\vec{f}(t-1)$ for shanno/CGD
- bool *finalFlag*: a flag to indicate that the train methods (innerTrainSet() or trainSet()) are at the final iteration, for code to be run only during the final iteration
- Matrix < double > *grads*: a matrix to hold exemplar gradients formerly used to compute the **B** matrix for the condition number

- *Protected* Utility functions
  - void copy(const Network& rhs): Copy utility function, see section 12.16.2 for discussion.
  - virtual void runHeader(ostream& *outputStream*): A pure virtual utility function inherited from the Iterative class that outputs Network specific parameters preceeding an Iterative run
  - virtual void pack(): A pure virtual utility function that converts the entire weight gradient structure of the *Network* derived object to the protected member *stackG*
  - virtual void unpack(): A pure virtual utility function that converts the protected member *stackG* back to the weight gradient structure of the *Network* derived object
  - virtual double getGradMax(): A pure virtual utility function inhertied from *Iterative* that returns the maximum absolute gradient
  - void engine(unsigned *type*, unsigned *iteration*): an utility function which calls the appropriate training algorithm depending on training type *type*, and passes it the current iteration
  - void engine(unsigned *type*, unsigned *iteration*): chooses which kind of advanced gradient descent algorithm (CGD or shanno) based on *type*, and passes *iteration* to it
  - void CGD(unsigned *iteration*): implements conjugate gradient descent (see Golden pp. 221–222)[15]
  - void shanno(unsigned *iteration*): implements Shanno algorithm (see Golden pp. 217–218)

[15] Richard M. Golden, Mathematical Methods for Neural Network Analysis and Design, 1996, MIT Press, ISBN 0-262-07174-6

- void conditionOf(const Matrix<double>& *symmetric*): stores the largest and smallest absolute eigenvalues of the passed symmetric matrix and their ratio, the condition number. The caller supplies the matrix, because which matrix the condition number describes is a statistical decision belonging to the caller; *Logistic* passes $\mathbf{X}^T\mathbf{V}\mathbf{X}$ (see above). Replaced the overloaded storeGrads(...) methods and the *grads* accumulator in August 2026
- void reportCondNum(ostream& *outputStream*): utility method to report the stored condition number to *outputStream*. It prints; it does not compute

12.22 BareProp

The **BareProp** class is a *Network* object that serves to illustrate neural network design in neUROn2++. *BareProp* is a one output node, one hidden layer neural network without biases.

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *BareProp* method setHidden, so that the *BareProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - BareProp& operator= (const BareProp& rhs): see section 12.16.2 for discussion
  - BareProp(const BareProp& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *BareProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Bareprop Model*
  - void setHidden(unsigned H): Sets number of hidden nodes to H, which is all that is required to specify the architecture for a *BareProp* object
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights

  - virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *BareProp* object architecture and weights to a file named $filename$, returns true if successful
  - virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *BareProp* object architecture and weights from a file named $filename$, returns true if successful
  - virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
  - virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
  - virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
  - virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *BareProp* object

- *Private* members
  - unsigned $nHidden$: number of hidden nodes
  - Matrix< double > hW: hidden weight Matrix
  - Matrix< double > $hWup$: hidden weight update Matrix
  - Matrix< double > hG: hidden weight gradient Matrix
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > hO: hidden output vector
  - vector< double > oW: output weight vector
  - vector< double > oG: output weight gradient vector
  - vector< double > h_err: hidden error vector
  - double o_err: output error term

- *Private* Utility functions
  - void copy(const BareProp& rhs): Copy utility function, see section 12.16.2 for discussion.

- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

12.23 SimpleProp

The **SimpleProp** class is a *Network* object that serves to illustrate neural network design in neUROn2++. *SimpleProp* is a one output node, one hidden layer neural network with biases.

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *SimpleProp* method setHidden, so that the *SimpleProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - SimpleProp& operator= (const SimpleProp& rhs): see section 12.16.2 for discussion
  - SimpleProp(const SimpleProp& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *SimpleProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Bareprop Model*
  - void setHidden(unsigned H): Sets number of hidden nodes to H, which is all that is required to specify the architecture for a *SimpleProp* object
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
  - virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *SimpleProp* object architecture and weights to a file named $filename$, returns true if successful

- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *SimpleProp* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *SimpleProp* object

- *Private* members
  - unsigned $nHidden$: number of hidden nodes
  - unsigned nH: $nHidden - 1$, used for indexing with biases
  - Matrix< double > hW: hidden weight Matrix
  - Matrix< double > $hWup$: hidden weight update Matrix
  - Matrix< double > hG: hidden weight gradient Matrix
  - vector< double > in_bias: input bias vector to add to input Matrix
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > hO: hidden output vector
  - vector< double > oW: output weight vector
  - vector< double > oG: output weight gradient vector
  - vector< double > h_err: hidden error vector
  - double o_err: output error term

- *Private* Utility functions
  - void copy(const SimpleProp& rhs): Copy utility function, see section 12.16.2 for discussion.

- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

12.24 BackProp

The **BackProp** class is a *Network* object that allows multiple output nodes and multiple hidden layers in a neural network, and biases may or may not be present.

Note: It is expected that the *Model* method setDataSet be called *before* the *BackProp* method setHidden, so that the *BackProp* object may know the number of input nodes and size the Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - BackProp& operator= (const BackProp& rhs): see section 12.16.2 for discussion
  - BackProp(const BackProp& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *BackProp Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Backprop Model*
  - void setHidden(const vector< unsigned >& v): Sets the hidden layers, with the number of hidden nodes on each layer indicated by the corresponding vector v element
  - virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of nodal connections
  - virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
  - virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *BackProp* object architecture and weights to a file named $filename$, returns true if successful

- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *BackProp* object architecture and weights from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which forward propagates for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *BackProp* object

- *Protected* methods
  - const vector< Matrix< double > >& weightMatrices() const: returns the *authoritative* weight Matrices of this network by constant reference, for *observation only*.

    Why it exists. Every other *Network* already lets a derived class read its fitted parameters: *OneHiddenNet* keeps hW and oW protected, and *Logistic* publishes W through getBetas(). *BackProp* was the sole exception, and a subclass that cannot read a model's parameters cannot compute a parameter-state identity for it. That identity is what allows two training runs to be shown to have begun from the *same* weights, which is a precondition for comparing their speed at all: the optimizer benchmark (`tests/optimizer/`) refuses to certify a comparison whose arms cannot be proven to share a starting state.

    What it deliberately does not expose. Only $Weights$. The companion containers $WeightsUp$, $WeightsAccumulate$, $Gradient$ and *vpack* are training *workspace* rather than parameters and remain private. Widening the whole private section would have promised every future subclass mutable access to all of them, which is a far larger commitment than observation requires.

    Cost and contract. Constant, non-virtual, inline; it allocates nothing, copies nothing, and adds no dispatch. No production code path calls it, so it cannot appear in a per-exemplar or per-element loop. Because the reference is to a member of this object, it remains valid

only while the object does, and it must not be retained across a call to setHidden() or removeInputs(), both of which resize the Matrices it refers to.

- *Private* members
  - vector< unsigned > $nLayer$: describes the number of hidden nodes on each layer indicated by the corresponding vector element in $nLayer$
  - unsigned $nLayers$: number of hidden layers
  - vector< double > I: vector for single exemplar inputs
  - vector< double > y: vector containing dataset outputs
  - vector< double > o_err: vector containing output errors
  - vector< Matrix< double > > $Weights$: vector of weight Matrix for all hidden layers and output layer
  - vector< Matrix< double > > $WeightsUp$: vector of Matrices to hold updated weights one for each weights Matrix
  - vector< Matrix< double > > $WeightsAccumulate$: vector of Matrices holding the per-exemplar sum accumulated across a batch/epoch pass. Divided by the number of training exemplars it is the averaged *raw* batch gradient of Methodology equation 2.14. It is not the quantity the weight update consumes when a search algorithm is selected — see *Gradient* below.
  - vector< Matrix< double > > *Gradient*: vector of Matrices holding the gradient as a separate structure, which is required whenever an algorithm other than canonical backpropagation is chosen, or gradient stopping is armed. The averaged batch gradient is copied into it; *engine* then transforms it into the selected optimizer's search direction, by way of *pack* and *unpack*. **The weight update must therefore be taken from** *Gradient*, in batch/epoch mode as well as on-line. Taking it from $WeightsAccumulate$ discards the search direction and silently reduces conjugate gradient descent and Shanno to plain gradient descent (corrected 2026-08-01; see `tests/backprop/check_bpoptimizer.cpp`).
  - vector< vector< double > > *vpack*: per-layer holding vectors used by *unpack* to rebuild *Gradient* from *stackG*
  - vector< vector< double > > $HOutputs$: vector containing a vector of outputs for each hidden layer
  - vector< vector< double > > $HErrors$: vector containing a vector of errors for each hidden layer
- *Private* Utility functions
  - void copy(const BackProp& rhs): Copy utility function, see section 12.16.2 for discussion.

- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the entire weight gradient structure to the *Network* inherited protected member *stackG*
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure

12.25 Logistic

The **Logistic** class is a *Network* object that implements logistic regression. This form of data modeling is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function. As a result, logistic regression is implemented as a derived Network class. This strategy also allows the user to perform stepwise forward and reverse regression on the built logistic regression model using the RegressNet class.

Note that the error is cross-entropy by definition, that the gradient descent method is batch-epoch by definition, and a bias node (y-intercept) is present by definition. At the end of a run, the β weights and standard error are reported and p-values for the Wald test that the β weights are nonzero.[16]

Note: It is expected that the method setDataSet inherited from *Model* be called *before* the *Logistic* method randomize () (which sets initial weights to zero or random values, depending on the value of the flag *randInitCondFlag* which is set by the method setInitCond()), so that the *Logistic* object may know the number of input nodes and size the vectors and Matrices related to the dataset prior to completing its architecture. This sequence of loading a dataset using the *Model* method setDataSet prior to establishing network architecture will be invoked for all *Network* objects.

- *Public* methods
  - Logistic& operator= (const Logistic& rhs): see section 12.16.2 for discussion
  - Logistic(const Logistic& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *Logistic Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *Logistic Model*

[16] see Hosmer and Lemeshow, Applied Logistic Regression, 2nd edition, John Wiley & Sons, Inc., New York, 2000, pp 36–42

- virtual unsigned df(): Implementation of the pure virtual method inherited from *Network* which returns the number of degrees of freedom, equal to the number of inputs + 1 for the y- intercept
- virtual void randomize(): Implementation of the pure virtual method inherited from *Network* which randomizes initial weights
- virtual void reportAccuracy(ostream& *outputStream*): overrides (calls base *Network* reportAccuracy() method first) *Network* method inherited from *Iterative*, adding binary logistic regression specific footer information, including reporting β weights, standard error, and Wald test (p for β weights nonzero)
- virtual bool save(string& $filename$): Implementation of the pure virtual method inherited from *Network* which saves the *Logistic* object β weights and y-intercept to a file named $filename$, returns true if successful
- virtual bool load(string& $filename$): Implementation of the pure virtual method inherited from *Network* which loads the *Logistic* object β weights and y-intercept from a file named $filename$, returns true if successful
- virtual double trainSet(): Implementation of the pure virtual method inherited from *Iterative* which trains one iteration through the training set, and returns set error
- virtual double innerTrainSet(): Implementation of the pure virtual method inherited from *Network* which is called by trainSet() to calculate the gradient and return set error, used for automatic stepsize selection
- virtual void forward(Matrix< double >& M, unsigned n): Implementation of the pure virtual method inherited from *Network* which applies the sigmoidal function for one input row n in a Matrix of inputs M (e.g. either the inherited *Model* Matrix $Train$ or $Test$)
- virtual void removeInputs(vector< unsigned >& v): Implementation of the pure virtual method which takes as its argument a vector of unsigned v representing positions of input nodes, and removes those nodes from the *Logistic* object

- *Private* members
  - vector< double > y: vector containing dataset outputs
  - vector< double > I: vector for single exemplar inputs
  - vector< double > W: vector for β weights (last is y-intercept)
  - vector< double > G: vector for β weight gradients
  - double o_err: output error term
- *Private* Utility functions

- void copy(const Logistic& rhs): Copy utility function, see section 12.16.2 for discussion.
- virtual void pack(): Implementation of the pure virtual utility function inherited from *Network* that converts the β weight gradient structure to the *Network* inherited protected member *stackG*, as the β weight structure is already a single vector, this simply copies it
- virtual void unpack(): Implementation of the pure virtual utility function inherited from *Network* that converts the *Network* inherited protected member *stackG* back to the weight gradient structure, which, since it is a vector, just copies it

12.26 DFA

The **DFA** class is a *Model* object that encodes discriminant function analysis. It is an abstract class, whose derived concrete classes are *LDFA* for linear discriminant function analysis, and *QDFA* for quadratic discriminant function analysis.[17]

- *Public* methods
  - DFA& operator= (const DFA& rhs): see section 12.16.2 for discussion
  - DFA(const DFA& rhs): see section 12.16.2 for discussion
  - virtual void setDataSet(DataSet& *data*): Implementation of the pure virtual method inherited from *Model* that loads the *DataSet* object *data* into the *DFA Model*
  - virtual void outputHeader(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* which outputs a header to an ostream *outputStream* describing the architecture of the *DFA Model*
  - virtual double train(): A pure virtual method inherited from *Model* that trains a *Model*, and returns the final error (double) for that model
  - virtual void reportAccuracy(ostream& *outputStream*): A pure virtual method inherited from *Model* that reports the accuracy of the trained model to an ostream *outputStream*
- *Protected* members
  - Matrix< double > $D0$, $D1$: Matrices for inputs of 1-output datasets

[17] An excellent treatment of discriminant function analysis can be found in Richard O. Duda and Peter E. Hart, Pattern Classification and Scene Analysis, 1973, John Wiley & Sons, New York, ISBN 0-471-22361-1, pp. 24-32

- vector< Matrix< double > > D: a vector of Matrices for inputs of multiple output datasets
- vector< double > X: holds inputs for 1 exemplar
- vector< double > $U0$, $U1$: mean vectors for 1-output datasets
- vector< vector< double > > U: a vector of mean vectors for multiple output datasets
- double $P0$, $P1$: a priori probabilities for 1-output datasets
- vector< double > P: a priori probabilities for multiple output datasets
- double $K0$, $K1$: constants for 1-output datasets
- vector< double > K: constants for multiple output datasets
- double $d0$, $d1$: discriminant function results for 1-output datasets
- vector< double > d: discriminant function results for multiple output datasets
- vector< unsigned > *trainClasses*: outputs for multiple output training sets
- vector< unsigned > *testClasses*: outputs for multiple output test sets

- *Protected* Utility functions
  - void copy(const DFA& rhs): Copy utility function, see section 12.16.2 for discussion.

12.27 LDFA

The **LDFA** class is a *DFA* object that encodes linear discriminant function analysis.

- *Public* methods
  - LDFA& operator= (const LDFA& rhs): see section 12.16.2 for discussion
  - LDFA(const LDFA& rhs): see section 12.16.2 for discussion
  - virtual double train(): Implementation of the pure virtual method inherited from *Model* that trains a *LDFA Model*
  - virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained *LDFA* model to an ostream *outputStream*

- *Private* members
  - Matrix< double > C: the common covariance Matrix

- Matrix< double > S: inverse of the common covariance Matrix

- *Private* Utility functions

  - void copy(const LDFA& rhs): Copy utility function, see section 12.16.2 for discussion.

12.28 QDFA

The **QDFA** class is a *DFA* object that encodes quadratic discriminant function analysis.

- *Public* methods

  - QDFA& operator= (const QDFA& rhs): see section 12.16.2 for discussion
  - QDFA(const QDFA& rhs): see section 12.16.2 for discussion
  - virtual double train(): Implementation of the pure virtual method inherited from *Model* that trains a *QDFA Model*
  - virtual void reportAccuracy(ostream& *outputStream*): Implementation of the pure virtual method inherited from *Model* that reports the accuracy of the trained *QDFA* model to an ostream *outputStream*

- *Private* members

  - Matrix< double > $C0$, $C1$: covariance Matrices for 1-output datasets
  - vector< Matrix< double > > C: a vector of covariance Matrices for multiple output datasets
  - Matrix< double > $S0$, $S1$: covariance Matrix inverses for 1-output datasets
  - vector< Matrix< double > > S: a vector of covariance Matrices for multiple output datasets
  - double $Det0$, $Det1$: covariance Matrix determinants for 1-output datasets
  - vector< double > Det: covariance Matrix determinants for multiple output datasets

- *Private* Utility functions

  - void copy(const QDFA& rhs): Copy utility function, see section 12.16.2 for discussion.

Chapter 13

Stepwise regression

Statistical analysis of neural computational models is based on the observation that given 2 fully trained neural networks with final errors $E(\mathbf{y}_{reduced})$ and $E(\mathbf{y}_{full})$ that are sufficiently close may be discriminated by a chi-square probability distribution, where

$$G^2 = -2N[E(\mathbf{y}_{full}) - E(\mathbf{y}_{reduced})].$$

with G^2 the chi-square probability p with degrees of freedom the difference in free parameters (connections between nodes) between the 2 neural models, and N the number of exemplars in the dataset (see section 6.3.) E is expressed as the cross-entropy error (see equation 6.3):

$$e(\mathbf{t}^k, \mathbf{a}^{(m)}) = -\mathbf{t}^k . * \mathbf{log}[\mathbf{a}_k^{(m)}] - (\mathbf{1} - \mathbf{t}^k) . * \mathbf{log}[1 - \mathbf{a}_k^{(m)}]. \tag{13.1}$$

In stepwise regression, the 2 networks $\mathbf{y}_{reduced}$ and $\mathbf{y}_{full}$ are chosen to differ by 1 input variable (which may be represented by 1 or more input nodes.)

13.1 Stepwise reverse regression

Stepwise reverse regression proceeds by training the full network, and recording that network's final error. Input variables are removed 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the largest p-value (least significant) is then recorded with its p-value. Using the final error of the network which is deficient the one node with the largest p-value as a starting point, input variables are then removed 2 at a time, with each pair containing the 1 variable with the largest p-value and 1 of the remaining input variables. The pair of variables with the largest p-value (least significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 13.1 demonstrates stepwise reverse regression.

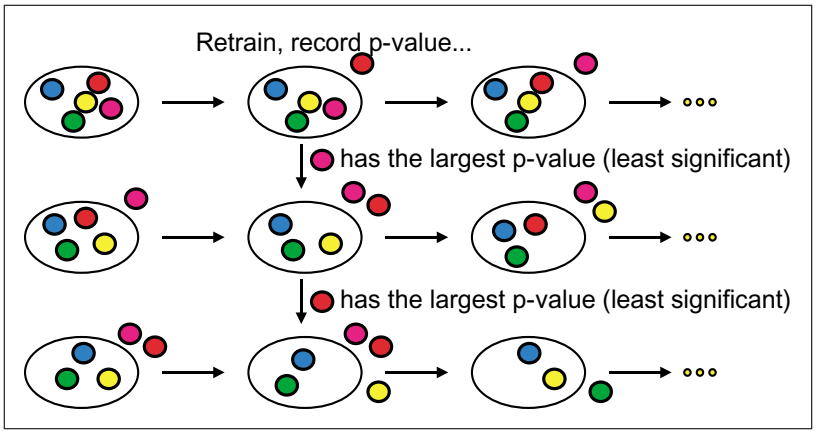

Figure 13.1: Stepwise reverse regression

13.2 Stepwise forward regression

Stepwise forward regression proceeds by training a baseline network with no input nodes, and recording that network's final error. Input variables are added 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the smallest p-value (most significant) is then recorded with its p-value. Using the final error of the network which is the one node with the smallest p-value as a starting point, input variables are then added 2 at a time, with each pair containing the 1 variable with the smallest p-value and 1 of the remaining input variables. The pair of variables with the smallest p-value (most significant) is then recorded with its p-value, and the process repeats until a chosen threshold p-value is reached.

Figure 13.2 demonstrates stepwise forward regression.

13.3 The RegressNet class

RegressNet is a compound class based on the *Network* object that implements stepwise regression analysis on neural computational models.

- *Public* methods
  - RegressNet& operator= (const RegressNet& rhs): see section 12.16.2 for discussion
  - RegressNet(const RegressNet& rhs): see section 12.16.2 for discussion
  - void setInputStructure(const vector< vector< unsigned > >& v): Sets a vector of vector of unsigned v as the structure of input nodes

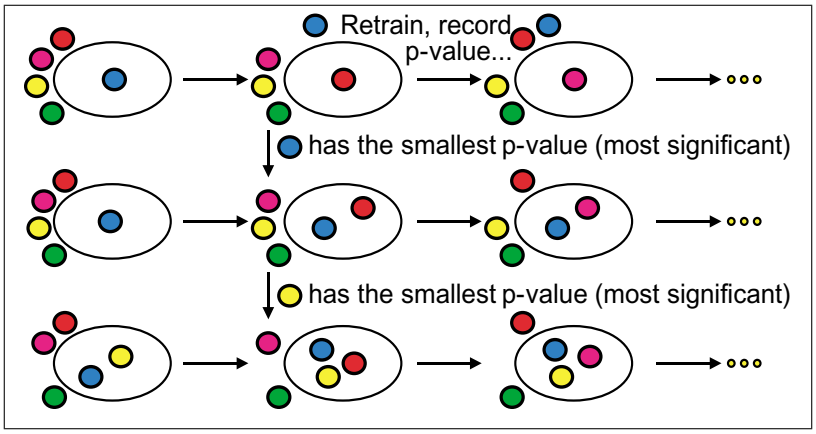

Figure 13.2: Stepwise forward regression

representing the variables for the stepwise regression. For example, in a 3 variable dataset, if variable 0 was represented by input nodes 0 and 1, variable 1 by input node 2, and variable 2 by input nodes 3, 4 and 5, vector[0] in v would contain (0,1), vector[1] in v would contain (2), and vector[3] in v would contain (3,4,5). All input nodes *must* be represented in v, and the groups must form an *exact partition* of them: *setNetwork* must have been called first, the structure and every group within it must be non-empty, every node index must belong to the network, and every input node must appear exactly once. Duplicates, overlaps, omissions — including an omission whose largest index still looks correct — and out-of-range nodes are each refused with their own message. The caller's order is preserved, between groups and within one: variable numbering in every report, progress announcement and audit-trail row is the caller's own, so normalizing here would renumber the answer. See section 10.8 for an utility method that parses a string description of input variables, and returns a vector of vector of unsigned.

  - void setNetwork(Network* *networkObj*, const double E): sets the *Network* object *networkObj* in the *RegressNet* object, and the final error E of the trained *networkObj*
  - void reverse_regress(): performs stepwise reverse regression
  - void forward_regress(): performs stepwise forward regression

- *Private* members
  - Network $*netPtr$: pointer to the incoming Network
  - Network $*netCopyPtr$: pointer to a copy of the incoming Network to generate subnetworks

- bool *historyFlag*: indicates logging to history file
- double *e_in*: incoming Network's error
- vector< vector< unsigned > > *variable_defs*: the data structure representing input variables
- typedef multimap< double, unsigned, less< double > > *ptable*: type definition of a table used to hold stepwise regression results, p-value first, followed by variable number
- ostringstream *out*: to simplify writing to screen and history file
- strstream *errorOut*: to build an exception message
- string *errorString*: the exception message

- *Private* Utility functions
  - void report(ostringstream& *o*): a utility function which outputs an ostringstream to screen and history file, takes ostringstream *o* as its argument
  - void print_input_structure(const vector< vector< unsigned > > & *v*): a utility function which prints the input variable structure to screen and history file, takes data structure representing input variables *v* as its argument
  - void print_regression_table(const ptable& *table*): a utility function which prints out a stepwise regression table, takes regression table multimap type *ptabletable* as argument
  - bool copy_network(): a utility function which copies the incoming Network object $*netPtr$ into $*netCopyPtr$ so that the latter can be made feature deficient for subnetworks. If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to this method.
  - string network_name() const: a utility function which returns the name of the type of *Network* object. If you are creating a regressable *Network* object (and chances are that you are,) you *must* add code to this method.
  - inline double Wilks(const double N, const double E_{full}, const double E_{sub}) const: a utility function which calculates the chi-square parameter for 2 networks with N exemplars and which differ by their number of input nodes, the cross-entropy error of the fully trained network with all input nodes being E_{full}, and the fully trained feature deficient subnetwork's error being E_{sub}
  - void copy(const RegressNet& rhs): copy utility function, see section 12.16.2 for discussion.

- *Protected* contract and shared scaffold

  The two selection procedures of sections 13.1 and 13.2 share their bookkeeping and nothing else. Each keeps its own outer and inner loops, its own nesting of the two models, its own Wilks argument order, its own degrees-of-freedom expression, its own choice of the largest or smallest p-value, its own stopping test, and its own way of arriving at the final variable set. Those are the published mathematics of the two procedures, and a shared descriptor carrying a comparator and an argument order would conceal exactly the difference a reader of this class needs to see.

  - static bool candidateFitEligible(Iterative::StopReason r, double E): may a candidate fit's error enter a Wilks comparison? Two independent conditions, and the second is not implied by the first: a stopping *rule* must have ended the run, *and* the error must be a real number. A saturated fit satisfies the first and fails the second, its gradients having underflowed to zero while its cross-entropy overflowed
  - void requireComparablePass(unsigned *step*): refuses a pass in which no candidate's p-value could be calculated at all, rather than naming a variable that was never compared. This is *not* the case of a pass whose p-values are all zero: zero is a p-value, meaning maximally significant, and the first candidate to produce one wins its pass. An exact partition of the input nodes rules out *gammq*'s invalid-argument refusals, but not its refusal when *gcf* or *gser* exhausts $ITMAX$, which happens on entirely valid degrees of freedom and chi-square, so this is a live numerical defense rather than an unreachable one
  - double beginAnalysis(const string& *banner*, const string& *question*): the cross-entropy refusal, the structure check, the first clone, the banner and the input-structure print; returns the p-value threshold
  - CandidateFit fitCandidate(..., double E_{prior}, unsigned *df*): trains one candidate subnetwork and records it, owning both progress announcements, the fit count, the record-*before*-judge ordering and the eligibility refusal. The caller has already cloned the network, removed the nodes and computed *df*, all three being direction-specific
  - double reportComparison(const CandidateFit& f, double E_{prior}, double G^2, unsigned *df*, unsigned v, ptable& t): reports one comparison and completes its audit-trail row. It is handed the statistic; it does not call Wilks, does not know which model was nested in which, and does not decide who wins. Returns the p-value, or not-a-number when the chi-square routine refused
  - void endAnalysis(const string& *direction*, const string& *verb*): the completion flag and the closing summary, reached only when a selection loop has decided

13.4 Exceptions

The *RegressNet* class throws exceptions of type *RegressNetErr*& *e*, where *e*.what() may contain:

- RegressNet error: maximum node value not *correct*, where *correct* is 1 less than the number of input nodes. This indicates that the vector of vector of unsigned representing the variable structure is incorrectly specified.
- RegressNet error: couldn't copy Network, unknown type. This indicates that the *Network* object has not been properly coded in RegressNet::copy_network().
- RegressNet error: stepwise regression not fully specified. This indicates that *Network* object and its final error have not been set in the *RegressNet* object.

13.5 Example

An example driver may thus appear as:

```
RegressNet regressionObj; // object to regress a Network

// Say you've already trained a Model pointed to by modelPtr, and
//    it was a Network Model, and the last error was lastError
Network* netPtr = dynamic_cast< Network* >( modelPtr );
regressionObj.setNetwork( netPtr, lastError );

vector< vector< unsigned > > variable_defs; // variable definitions
string variable_string; // for entry of variable definitions

cout << "Please enter a string representing the variables: ";
cin >> variable_string;
try {
   variable_defs = util::variable_parse( variable_string );
}
catch ( util::utilErr& e ) {
   cout << e.what() << endl;
}

if ( !variable_defs.empty() ) // parser returned with a good result
{
   try { // try to set the input structure in the stepwise regression
   object
      regressionObj.setInputStructure( variable_defs );
   }
```

```
   catch ( RegressNet::RegressNetErr& e ) {
      cout << e.what() << endl;
   }
}

try {
   regressionObj.reverse_regress();
}
catch ( RegressNet::RegressNetErr& e ) {
   cout << e.what() << endl;
}
```

13.6 Extending stepwise support today

RegressNet no longer contains a type-by-type copy switch, and no compiler-specific RTTI option is needed. Network cloning is centralized in `netclone.*`; construction from saved and requested model types is centralized in `modelfactory.*`. A new network type must participate in those shared interfaces and must preserve all architecture and training state when cloned.

Stepwise analysis works on clones and must never mutate the current trained model. Candidate fits use the same authoritative training loop as ordinary training, but run quietly and do not inspect the test set. Every candidate must converge before its likelihood may enter a Wilks comparison.

Index